MLR 570

Course Notes

My reference for the semester, organized chapter by chapter, part by part.

How to read this: red = key definitions/results blue = why it matters in ML amber = additions/clarifications not in my original notes.
Each part ends with its own quick-reference table.

Chapter 1 · Mathematical Background
1.1 Linear Algebra
1.2 Probability & Statistics
1.3 Calculus & Optimization

Chapter 2 · Data Fundamentals
2.1 Data Preprocessing
2.2 Similarity Measures
2.3 Model Evaluation

Chapter 3 · Supervised Learning
3.1 Hard-Margin SVM
3.2 Linear Regression: OLS, WLS & Ridge
3.3 Nonlinear Models, Kernels & k-NN

Contents

1 Mathematical Background **4**

1.1 Linear Algebra . . . 4

1.1.1 Matrices as Linear Transformations . . . 4

1.1.2 Rank . . . 6

1.1.3 Rank–Nullity and the Null Space . . . 8

1.1.4 Common Types of Matrices . . . 10

1.1.5 Eigenvectors and Eigenvalues . . . 11

1.1.6 Diagonalization . . . 12

1.1.7 Norms . . . 14

1.1.8 Singular Value Decomposition (SVD) . . . 17

1.1.9 Quick Reference . . . 20

1.2 Probability & Statistics . . . 22

1.2.1 A Single Random Variable . . . 22

1.2.2 Moments . . . 24

1.2.3 Two Random Variables: Joint Distributions . . . 27

1.2.4 Conditional Probability Distributions . . . 28

1.2.5 Independence . . . 30

1.2.6 Expectation with Two Random Variables . . . 30

1.2.7 Covariance and Correlation . . . 32

1.2.8 Conditional Expectation . . . 34

1.2.9 Putting It All Together: Discrete vs Continuous . . . 35

1.2.10 Quick Reference . . . 36

1.3 Calculus & Optimization . . . 37

1.3.1 Limits & Derivatives . . . 37

1.3.2 Partial Derivatives & Gradients . . . 39

1.3.3 Vector & Matrix Derivatives . . . 39

1.3.4 The Chain Rule . . . 41

1.3.5 Matrix Calculus Identities . . . 44

1.3.6 Convexity & Optimization . . . 45

1.3.7 Quick Reference . . . 48

2 Data Fundamentals **49**

2.1 Data Preprocessing . . . 49
2.1.1 Why Preprocess at All? . . . 49
2.1.2 Aggregation . . . 50
2.1.3 Sampling . . . 51
2.1.4 Normalization . . . 52
2.1.5 Encoding . . . 53
2.1.6 Discretization . . . 54
2.1.7 Dimensionality Reduction . . . 55
2.1.8 Principal Component Analysis (PCA) . . . 55
2.1.9 Feature Selection . . . 59
2.1.10 Quick Reference . . . 61
2.2 Similarity Measures . . . 62
2.2.1 Pearson Correlation . . . 62
2.2.2 Euclidean and Manhattan Distance . . . 63
2.2.3 The Curse of Dimensionality . . . 64
2.2.4 Cosine Similarity . . . 65
2.2.5 Jaccard Similarity . . . 66
2.2.6 Comparative Summary . . . 66
2.2.7 Quick Reference . . . 67
2.3 Model Evaluation . . . 68
2.3.1 Training, Validation, and Testing . . . 68
2.3.2 Underfitting and Overfitting . . . 69
2.3.3 K-Fold Cross-Validation . . . 70
2.3.4 Classification Metrics . . . 71
2.3.5 F1 and the Harmonic Mean . . . 72
2.3.6 The Decision Threshold . . . 72
2.3.7 Weighted F-beta . . . 73
2.3.8 Regression Metrics: MSE and MAE . . . 73
2.3.9 Quick Reference . . . 75

3 Supervised Learning 76
3.1 Hard-Margin SVM . . . 76
3.1.1 Why SVM? From “a separator” to the best separator . . . 76
3.1.2 Rebuilding the Line Geometry from First Principles . . . 77
3.1.3 Distance from a Point to a Hyperplane . . . 78
3.1.4 SVM Margin Geometry . . . 80
3.1.5 Why We Need the Hard-Margin Constraint . . . 81
3.1.6 Hard-Margin SVM Primal Problem . . . 82

3.1.7 Lagrange Multipliers: Putting the Constraints into One Function 82
3.1.8 From the Lagrangian to the Dual Problem 83
3.1.9 Why Does the Dual Maximize? 85
3.1.10 Final Hard-Margin Dual Problem 86
3.1.11 Support Vectors and the Geometry of the Final Classifier 86
3.1.12 Lecture Worked Example 87
3.1.13 Quick Reference 89
3.2 Linear Regression: OLS, WLS & Ridge 90
3.2.1 Ordinary Least Squares (OLS): What Are We Solving? 90
3.2.2 Deriving the OLS Gradient from First Principles 91
3.2.3 The Geometry of OLS: the Normal Equations Come from the Picture 92
3.2.4 OLS Numerical Example from the Lecture 94
3.2.5 Homoscedasticity vs. Heteroscedasticity 94
3.2.6 Weighted Least Squares (WLS) 95
3.2.7 Ridge Regression: a Different Problem 96
3.2.8 Ridge Objective and Closed-Form Solution 97
3.2.9 Ridge Worked Example from the Lecture 98
3.2.10 Quick Reference 100
3.3 Nonlinear Models, Kernels & k-NN 101
3.3.1 Why Linear Models Need Help 101
3.3.2 Nonlinear Feature Mappings 103
3.3.3 Polynomial Features Before Polynomial Kernels 103
3.3.4 The Polynomial Kernel, Built from First Principles 104
3.3.5 Why Dot Products Let Us Kernelize the Perceptron 106
3.3.6 RBF / Gaussian Kernel 107
3.3.7 Local versus Global Geometry 108
3.3.8 Polynomial versus RBF: What Each Kernel Assumes 109
3.3.9 Exponential / Laplacian Kernel 110
3.3.10 What Makes a Kernel Valid? 110
3.3.11 Kernel Least Squares / Kernel Ridge Regression 111
3.3.12 k-Nearest Neighbors (k-NN) 113
3.3.13 The Meaning of Bias and Variance in k-NN 115
3.3.14 Choosing k: the Elbow / Minimum-Error Region 116
3.3.15 k-NN Computational Cost and the Curse of Dimensionality 116
3.3.16 Quick Reference 118

CHAPTER 1

1 · Mathematical Background

Everything the rest of the course assumes you already know, gathered in one place, with the reason each piece matters for machine learning attached to it.

PART 1.1

Linear Algebra

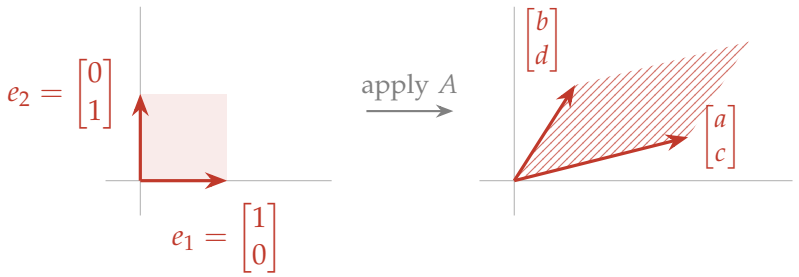
The flow: Matrix → Determinant → Det = 0 → Rank → Rank–Nullity → Null Space → Diagonal matrices → Eigenvectors → Diagonalization → Norms → SVD.

1.1.1 Matrices as Linear Transformations

Matrix

A matrix $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ is a **linear transformation**. It acts on the basis vectors

$$e_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad e_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$



before — after: area scaled by $|\det A|$

Determinant

So applying $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ tells us how much the grid is stretched by.

Determinant This stretching factor applies to the *whole grid*; it is called $|\det(A)|$.

Added note

For a 2×2 matrix: $\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$. The sign tells you whether the orientation flipped; the absolute value is the area stretch factor.

Case: $\det(A) = 0$

This means the transformed basis directions fail to span the full space. **Hence, a dimension was lost.**

Why $\det = 0$ means not invertible This is why a matrix with $\det = 0$ is **not invertible**: if a transformation collapses several different original points onto the same output, then, given the output, we can't determine which original point it came from.

Multiplying matrices = chaining transformations

The product AB means: apply B **first**, then A. For $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times p}$,

$$C = AB \in \mathbb{R}^{m\times p}, \qquad c_{ij} = \sum_k a_{ik} b_{kj}$$

(each entry combines a row of A with a column of B). It only works when the **inner dimensions match**: columns of A = rows of B.

Order matters In general $AB \neq BA$: transformations don't commute. Stretching then rotating is not the same grid as rotating then stretching.

Transpose

$(A^T)_{ij} = A_{ji}$: flip the matrix across its main diagonal, so rows become columns.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \Rightarrow \quad A^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Rules: $(A^T)^T = A$, $(A + B)^T = A^T + B^T$, and the one to be careful with:

$$\boxed{(AB)^T = B^T A^T}$$

The order **reverses**. If $A^T = A$, the matrix is **symmetric** (more on it in §1.1.4).

Matrix inverse

The inverse is the transformation that **undoes** A:

$$AA^{-1} = A^{-1}A = I.$$

When does it exist? A square A is invertible $\iff$ $\det(A) \neq 0$ $\iff$ full rank. Same story as above: if $\det = 0$, a dimension collapsed, several inputs landed on the same output, and no map can tell them apart again.

For a 2×2 matrix there is a direct formula:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad \Rightarrow \quad A^{-1} = \frac{1}{\det A} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Example

$A = \begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}$: $\det A = 24 - 14 = 10 \neq 0$, so

$$A^{-1} = \frac{1}{10}\begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix}. \qquad \text{Check: } AA^{-1} = \frac{1}{10}\begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix} = I. \checkmark$$

But $\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$ has $\det = 0 \to$ singular, no inverse.

Like the transpose, the inverse of a product **reverses the order**: $(AB)^{-1} = B^{-1}A^{-1}$ — to undo, undo the last transformation first. Also $(A^T)^{-1} = (A^{-1})^T$.

Added note

Only square, full-rank matrices have a true inverse. A rectangular matrix never does; the closest substitute is the Moore–Penrose **pseudoinverse**, built from the SVD.

This leads us to **Rank**.

1.1.2 Rank

Rank Rank is the # of independent directions represented by a matrix.

- A 3×3 matrix has rank $= 3$ if it preserves all 3 dimensions.

So $\to$ **Rank = # of independent dims that survive.**

Column space: what rank actually measures

Multiplying $A\mathbf{x}$ is really **mixing the columns** of A:

$$A\mathbf{x} = \begin{bmatrix} | & & | \\ \mathbf{a}_1 & \cdots & \mathbf{a}_n \\ | & & | \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = x_1\mathbf{a}_1 + \cdots + x_n\mathbf{a}_n.$$

So the set of every output A can produce is the set of all mixes of its columns.

Column space

$$\text{Col}(A) = \{\, A\mathbf{x} \,\} = \text{span of the columns of } A$$

It is everything the matrix can **reach**. And this is where rank formally lives:

$$\boxed{\text{rank}(A) = \dim \text{Col}(A) = \#\text{ of linearly independent columns}}$$

The row space has the same dimension — that's why counting independent rows or independent columns gives the same rank.

Example

$$L = \begin{bmatrix} 1 & 0 & 2 & 3 & 4 \\ 0.5 & 0 & 1 & 1.5 & 2 \\ 2 & 0 & 4 & 6 & 8 \\ 1 & 0 & 2 & 3 & 4 \\ 2 & 0 & 4 & 6 & 8 \end{bmatrix} \qquad \Rightarrow \qquad \text{rank}(L) = 1.$$

Every row is a multiple of the first ($R_2 = \frac{1}{2}R_1$, $R_3 = 2R_1$, $R_4 = R_1$, $R_5 = 2R_1$), so the whole matrix carries **one** pattern of info.

Solving $A\mathbf{x} = \mathbf{b}$

A system of linear equations is the question: *which mix of the columns of A gives* $\mathbf{b}$*?*

$$\boxed{A\mathbf{x} = \mathbf{b} \text{ has a solution } \iff \mathbf{b} \in \text{Col}(A)}$$

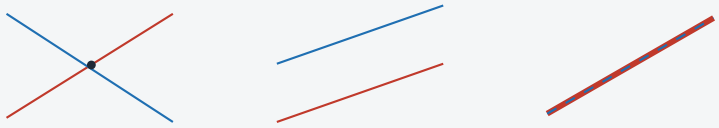
Three things can happen:

The three cases

1) Unique solution: $x + y = 3,\ 2x - y = 0$. Eliminate: $R_2 - 2R_1 \Rightarrow -3y = -6 \Rightarrow y = 2,\ x = 1$. The lines cross at one point.

2) No solution: $x + y = 2,\ 2x + 2y = 5$. Eliminate: $R_2 - 2R_1 \Rightarrow 0 = 1$, a contradiction. Parallel lines; $\mathbf{b}$ is outside $\text{Col}(A)$.

3) Infinitely many: $x + 2y = 4,\ 2x + 4y = 8$. The second equation is just $2\times$ the first. Let $y = t$: $\mathbf{x} = (4 - 2t,\ t)$ for any t — a whole line of solutions.



cross: one solution parallel: no solutions same line: infinitely many

Rank decides For a square A: full rank $\Rightarrow$ exactly one solution, $\mathbf{x} = A^{-1}\mathbf{b}$. Rank-deficient $\Rightarrow$ no solution ($\mathbf{b}$ outside $\text{Col}(A)$) or infinitely many ($\mathbf{b}$ inside, plus anything from the null space, §1.1.3).

From a data perspective

$$\begin{matrix} x\text{-axis} & y\text{-axis} \\ \downarrow & \downarrow \end{matrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Each column is a feature (an axis), each row is a sample.
Rank = # of independent patterns of info present (unique info).

Rank in ML

1. **Preventing redundancy.** If two feature columns are exact copies or multiples of each other, they don't add new info. Hence rank ↓.
2. **Checking model solvability.** If the matrix is full rank, the data has no useless duplication.

The model trains stably.

3. **Data compression and noise reduction.** Many real-world datasets have low rank because features overlap heavily. **PCA & SVD find low-rank approximations.**

Rule: maximum rank

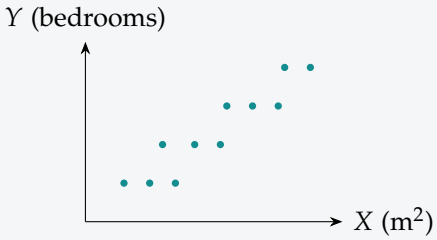
Rule

$$\text{Max rank} = \min(m, n)$$

The rank of a matrix can never be larger than its smallest dimension.

Examples

- Data on 10,000 houses (rows), 3 features (cols). Dimension: $10{,}000 \times 3$, so max possible rank $= 3$.
- In hopeful situations: let's say we have 50 features (columns) and 10,000 samples (rows) $\rightarrow$ max rank is 50. In a healthy dataset, the calculated rank is exactly $= 50$.
- 1,000 houses (rows), 2 features (cols): square meters (X) & bedrooms (Y). Max rank $= 2$.



If we plot the data: points are only plotted on 2D because we don't have a third feature — we are restricted to 2-D permanently.

Low rank: curse or blessing?

Curse: a low-rank square matrix is singular — det $= 0$, no inverse, and $A\mathbf{x} = \mathbf{b}$ has no solution or infinitely many.
Blessing: low rank means the matrix can be rebuilt from far fewer numbers. If $W \in \mathbb{R}^{1000\times1000}$ has rank 5, every row/column lives in a 5-dimensional subspace, so it factors as

$$W = UV, \qquad U \in \mathbb{R}^{1000\times5}, \quad V \in \mathbb{R}^{5\times1000}.$$

Storage drops from 10^6 numbers to 10,000, and $W\mathbf{x}$ becomes two cheap products $U(V\mathbf{x})$. Low rank = redundancy, and exploiting it is exactly what PCA & SVD compression do. The price: a rank-limited matrix can't represent rich relationships — cap the rank too hard and the model underfits.

1.1.3 Rank–Nullity and the Null Space

Rank–Nullity

Rank–Nullity Theorem

$$\text{rank} + \text{nullity} = n \qquad \text{with } n \text{ input dimensions.}$$

The input space splits into:

- directions that **survive** (rank),
- directions that **disappear** (nullity).

Null space

Null space

$$\text{Null}(A) = \{\, x \ \text{ s.t. } \ Ax = 0 \,\}$$

The null space tells us exactly what information the matrix is **blind to**.

Example

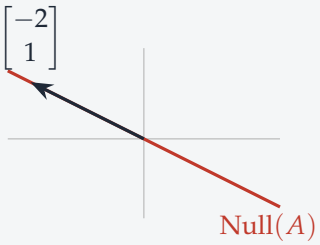
$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$.

$$Ax = 0 \ \Rightarrow \ \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = 0 \ \Rightarrow \ x_1 + 2x_2 = 0 \ \Rightarrow \ x_1 = -2x_2$$

Let $x_2 = t$:

$$x = \begin{bmatrix} -2t \\ t \end{bmatrix} = t \begin{bmatrix} -2 \\ 1 \end{bmatrix}$$

So any vector in the direction $\begin{bmatrix} -2 \\ 1 \end{bmatrix}$ is **erased** by A.
The null space here is a line, so nullity (dimension) $= 1$.
The null space basically means it crushes anything that enters its "space".



Added note

Check with rank–nullity: the second row of A is $2\times$ the first, so rank $= 1$. Then rank + nullity $= 1 + 1 = 2 = n$. ✓

Nullity can be bigger than 1

$A = \begin{bmatrix} 1 & 2 & -1 \\ 2 & 4 & -2 \end{bmatrix}$: rank $= 1$ (second row $= 2\times$ first), $n = 3$, so nullity $= 2$. Solving $x_1 + 2x_2 - x_3 = 0$ gives a whole **plane** of erased vectors, e.g. spanned by $(1, 0, 1)^T$ and $(0, 1, 2)^T$.

Null space in ML

The null space represents the **blind spot** of an ML model. It contains the directions in which your data *can* change, but the model will not see it.

- If a vector z is in the null space of matrix A, then multiplying them gives 0: $Az = 0$.
- Due to that, we can add this null-space noise z to your original data x, and the model output is exactly the same:

$$A(x+z) = Ax + Az = Ax + 0 = Ax$$

- To the ML model, x and $(x+z)$ are **identical**.

Why it matters in ML:

- **PCA:** when compressing data, we intentionally look for directions in the data that don't contribute much to the final prediction.
- **Rank–nullity balancing:** rank + nullity = total features, e.g.

$$\underbrace{7}_{\text{useful features}} + \underbrace{3}_{\text{dead zones, data change ignored}} = 10 \text{ features}$$

1.1.4 Common Types of Matrices

Diagonal matrix

$$D = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 7 & 0 \\ 0 & 0 & 2 \end{bmatrix}, \qquad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \qquad \Rightarrow \qquad Dx = \begin{bmatrix} 3x_1 \\ 7x_2 \\ 2x_3 \end{bmatrix}$$

D simply says: stretch direction 1 by 3, direction 2 by 7, direction 3 by 2. Every direction is handled on its own — nothing gets mixed.

This becomes important with the **eigendecomposition**: we want to find a coordinate system in which a complicated matrix behaves like a diagonal matrix. We look for:

1. **eigenvectors:** brand-new axes (directions),
2. **eigenvalues:** scale.

Diagonalization in ML

It is the ultimate clean-up tool for ML data. It takes a messy dataset where features are tangled up and alters perspectives so that every feature becomes perfectly independent.

Added note

Strictly, diagonalizing the data's *covariance* matrix makes the new features *uncorrelated* (this is exactly what PCA does). "Independent" is the intuition; "uncorrelated" is the precise term.

Other types worth recognizing

Identity matrix I**:** ones on the diagonal, zeros elsewhere. $AI = IA = A$ — the "do nothing" transformation.

Symmetric matrix: $A^T = A$.

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 3 & 4 \\ 0 & 4 & 5 \end{bmatrix}$$

The matrices ML cares about most are symmetric: **covariance matrices** and **Hessians**. Symmetry comes with two gifts: the eigenvalues are always **real**, and the eigenvectors can be chosen **orthonormal**, so

$$A = Q\Lambda Q^T$$

— diagonalization where the inverse is free, because $Q^{-1} = Q^T$ (next type).

Orthogonal / orthonormal matrix: $Q^TQ = I$. Columns are mutually perpendicular and unit length. For a square Q this forces

$$\boxed{Q^{-1} = Q^T}$$

the cheapest inverse there is. These matrices **preserve lengths and angles**: pure rotations/reflections, no stretching.

Example

$$Q = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \qquad Q^TQ = I,$$

and geometrically Q rotates the plane by 45°.

This is exactly what U and V are in the SVD (§1.1.8): the two "rotate" steps are orthogonal matrices, and $V^{-1} = V^T$ is why the formula ends with V^T.

Rank-one matrix: $A = \mathbf{u}\mathbf{v}^T$ (outer product of two vectors):

$$\mathbf{u} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \quad \Rightarrow \quad \mathbf{u}\mathbf{v}^T = \begin{bmatrix} 3 & 4 & 5 \\ 6 & 8 & 10 \end{bmatrix}.$$

Every row is a multiple of $\mathbf{v}^T$, so rank = 1: one pattern of info. The SVD layers $\sigma_i \mathbf{u}_i \mathbf{v}_i^T$ (§1.1.8) are exactly these.

Type	Definition	Where it shows up
Skew-symmetric	$A^T = -A$ (zero diagonal)	rotations/angular momentum in physics; purely imaginary eigenvalues
Upper/lower triangular	zeros below/above the diagonal	LU decomposition; solving systems by forward/backward substitution
Sparse	most entries are 0	recommender systems, bag-of-words, graph adjacency; cheap storage

1.1.5 Eigenvectors and Eigenvalues

A matrix is a transformation: for Ax, applying A to most vectors would change their **length and direction**.

However! Eigenvector directions do not change.

Definition A nonzero vector v is an eigenvector of A if

$$Av = \lambda v$$

Multiplying v by A would give v itself, just stretched by the scalar λ (the eigenvalue).

Why $\det(A - \lambda I) = 0$?

$$Av = \lambda v \;\rightarrow\; Av - \lambda v = 0 \;\rightarrow\; Av - \lambda Iv = 0 \;\rightarrow\; \boxed{(A - \lambda I)v = 0}$$

- We don't want $v = 0$, since eigenvectors must be non-zero.
- So, for what values of λ does the matrix $A - \lambda I$ have a nonzero vector in its null space?
- If it has a nonzero solution, then $A - \lambda I$ has a non-trivial null space. Hence it is **not full rank**. It is **not invertible**. Therefore:

$$\boxed{\det(A - \lambda I) = 0}$$

Added note
This is the whole chain of the notes connecting: non-trivial null space ⇒ not full rank ⇒ a dimension was lost ⇒ det = 0 (§1.1.1).

Importance in ML

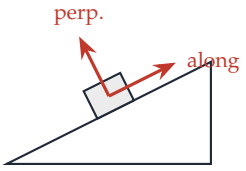
- Finding the true shape of data.
- Eigenvectors point at the highest variation in data (most dominant trend).

Added note
Precisely: the eigenvectors of the data's **covariance matrix** point along the directions of variation, and the one with the **largest eigenvalue** is the direction of highest variation. That's the first principal component in PCA.

1.1.6 Diagonalization

Diagonalization Switching the matrix's basis to a basis made of its eigenvectors. Diagonalization means switching to the eigenvector directions as our coordinate system.

Intuition: imagine a mechanics problem — we switch from the horizontal and vertical basis to *along the ramp* and *perpendicular to the ramp*. The problem is the same, but in the right axes it becomes simple.



Building P and D

Take $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ with eigens

$$v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix},\ \lambda_1 = 3 \qquad v_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix},\ \lambda_2 = 1$$

- $P = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ — just placing the eigenvectors into a matrix (as columns).
- $D = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$ — place their corresponding eigenvalues in another (diagonal) matrix.

Eigendecomposition

$$A = PDP^{-1}$$

- P: has the eigenvectors as columns.
- D: diagonal, with the corresponding eigenvalues.

Explanation of $A = PDP^{-1}$

Read it **from right to left**:

1. P^{-1}: change the matrix into its natural directions (eigenvector axes).
2. D: perform scaling.
3. P: change back to the regular directions.

Diagonal matrices make it easy for D^k to be computed:

$$D = \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix} \quad \Rightarrow \quad D^{10} = \begin{bmatrix} 4^{10} & 0 \\ 0 & 3^{10} \end{bmatrix}$$

So the idea is to take matrix A and factor it: $A = PDP^{-1}$.

e.g. computing A^4

$$A^4 = (PDP^{-1})(PDP^{-1})(PDP^{-1})(PDP^{-1}) = PD\underbrace{P^{-1}P}_{I}D\underbrace{P^{-1}P}_{I}D\underbrace{P^{-1}P}_{I}DP^{-1}$$

$$\boxed{A^4 = PD^4P^{-1}}$$

Worked example

E.g. $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$

Step 1) Find eigenvalues: $\det(A - \lambda I) = 0$

$$A - \lambda I = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} = \begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix}$$

$$(2-\lambda)^2 - 1 = 0 \qquad \Rightarrow \qquad \lambda_1 = 3, \quad \lambda_2 = 1$$

Step 2) Now find the corresponding eigenvectors:

- For $\lambda_1 = 3$: $(A - 3I)v = 0$

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0$$

$-x + y = 0 \Rightarrow \boxed{x = y}$
So any vector where $y = x$ works:
$\begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 2 \\ 2 \end{bmatrix}, \begin{bmatrix} 3 \\ 3 \end{bmatrix}, \ldots$ They all point in the same direction, so just take the simplest: $v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

- For $\lambda_2 = 1$: $(A - I)v = 0$

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 0$$

$x + y = 0 \Rightarrow x = -y$
Choose $v_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$.

Step 3) Assemble:

$$P = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \qquad D = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}, \qquad \boxed{A = PDP^{-1}}$$

Added note

Verification. $P^{-1} = \frac{1}{-2} \begin{bmatrix} -1 & -1 \\ -1 & 1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$. Then

$$PD = \begin{bmatrix} 3 & 1 \\ 3 & -1 \end{bmatrix}, \qquad PDP^{-1} = \frac{1}{2} \begin{bmatrix} 3 & 1 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = A. \checkmark$$

The order of columns in P must match the order of eigenvalues in D.

1.1.7 Norms

Norm A norm measures the **size** of a vector/matrix, e.g. of $x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$.

Norms in ML
We use norms to measure errors, compare vectors, and decide whether the model is overfitting/generalizing.

L_2 norm (Euclidean)

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$$

- Straight-line distance (Euclidean) from the origin to the point $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$.
- Distance between two points: $\|x - y\|_2$.

Why a circle? We ask: which points have exactly the same norm? All points on a circle are exactly the same distance away from the centre.

L_2 regularization

Perfect for keeping all weights **small but not zero**.

L_1 norm (Manhattan distance)

$$\|x\|_1 = \sum_i |x_i| \qquad \text{e.g. } x = \begin{bmatrix} 3 \\ 4 \end{bmatrix} : \quad \|x\|_1 = |3| + |4| = 7$$

Useful: imagine only being able to travel vertically or horizontally.

Added note

Same vector, different norms: $\|(3,4)\|_2 = \sqrt{9+16} = 5$, while $\|(3,4)\|_1 = 7$.

L_1 in ML

Important for **sparse models**, since the L_1 norm drives many weights to zero, performing **feature selection**.

Goal:

$$\min_w \Big[\underbrace{L(w)}_{\substack{\text{loss: diff between} \\ \text{actual vs predicted}}} + \underbrace{\lambda \|w\|_1}_{\substack{\text{charges the model for} \\ \text{using large weights}}} \Big]$$

λ controls how much we care.

L_∞, Frobenius, and what makes a norm

L_∞ norm (max norm): only the biggest coordinate matters:

$$\|x\|_\infty = \max_i |x_i|$$

One vector, three sizes

$$x = [3, -4] : \qquad \|x\|_1 = 3 + 4 = 7, \qquad \|x\|_2 = \sqrt{9+16} = 5, \qquad \|x\|_\infty = 4.$$

Frobenius norm (for matrices): flatten the matrix into one long vector and take its L_2 norm:

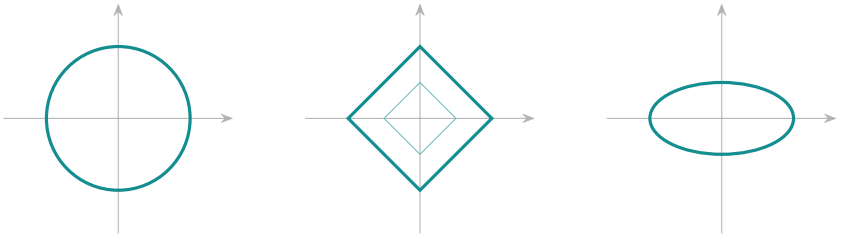
$$\|A\|_F = \sqrt{\sum_{i,j} a_{ij}^2} \qquad \text{e.g.} \quad A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} : \ \|A\|_F = \sqrt{1+4+9+16} = \sqrt{30}.$$

This is the $\|W\|_F^2$ that shows up when regularizing weight matrices (§1.3.5).

Added note

Every norm obeys three rules: $\|x\| \geq 0$, with $\|x\| = 0$ only for $x = 0$; $\|\alpha x\| = |\alpha| \, \|x\|$; and the triangle inequality $\|x + y\| \leq \|x\| + \|y\|$ — a detour is never shorter than the direct path.

Shapes of equal norm



L_2: $x_1^2 + x_2^2 = 1$ circle

L_1: $|x_1| + |x_2| = 1$ diamond

$x_1^2 + 4x_2^2 = 1$ ellipse

- **The plot of L_1 norms:** contours in a tilted plane (a diamond) — points that all lie equally distant from the origin. For L_2 it is circular.
- **Ellipses:** some directions are more expensive than others.

$$x^T Ax = 1 \qquad \text{e.g. } x_1^2 + 4x_2^2 = 1$$

Simply a circle, stretched.

Added note

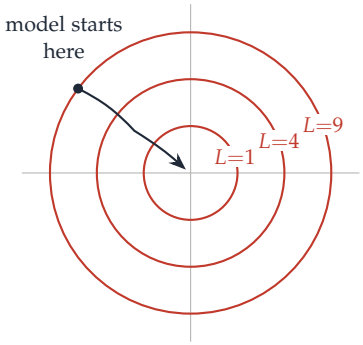
The ellipse example is $x^T Ax = 1$ with $A = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}$. Moving in the x_2 direction costs $4\times$ more, so the curve only reaches $x_2 = \pm\frac{1}{2}$ but $x_1 = \pm 1$. With a non-diagonal A, the ellipse is tilted — and its axes are the eigenvectors of A.
Why L_1 gives zeros: the diamond has sharp corners on the axes, so when the loss contours touch it, they tend to hit a corner, where one weight is exactly 0. The circle has no corners, so L_2 just shrinks weights.

Loss landscapes and contour plots

Suppose a model has params $w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$ and

$$L(w_1, w_2) = w_1^2 + w_2^2$$

- The 3D surface looks like a **bowl**.
- Imagine looking at it directly from above.
- If $L = 1$, then $w_1^2 + w_2^2 = 1$.
- So you see circles: $L = 1$, $L = 4$, $L = 9$, etc.
- Imagine your model is here (outer circle) and you are training it: it will go down to $L = 1$, like a ball rolling down.



Added note

The circles have radii $1, 2, 3$ (since $w_1^2 + w_2^2 = L$, radius $= \sqrt{L}$). Training keeps rolling inward past $L = 9 \to 4 \to 1$, all the way toward the minimum $L = 0$ at the centre.

Big picture A contour plot is a bird's-eye view of the loss landscape.

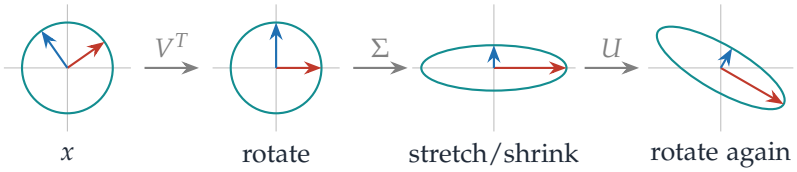
1.1.8 Singular Value Decomposition (SVD)

What SVD says

SVD says Any matrix can be understood by **stretching certain special input directions** by certain amounts and **sending them into corresponding output directions**.

$$A = U\Sigma V^T$$

Read **right to left**: rotate $\rightarrow$ stretch/shrink $\rightarrow$ rotate again.



Step 1) V^T helps rotate the coordinate system so that I am aligned with the directions that A wants to stretch.

Added note
Completing the remaining steps:
Step 2) Σ is diagonal: it stretches/shrinks along those aligned directions. Its diagonal entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ are the *singular values*.
Step 3) U rotates the result into its final orientation in the output space.
Compared to $A = PDP^{-1}$: SVD works for *any* matrix (even non-square), and U, V are orthogonal (pure rotations/reflections), so $V^{-1} = V^T$. The number of nonzero singular values = rank of A.

One direction at a time

E.g. the matrix

$$A = \begin{bmatrix} 4 & 1 & 2 & 0 \\ 1 & 4 & 0 & 2 \\ 2 & 0 & 4 & 1 \\ 0 & 2 & 1 & 4 \end{bmatrix} \qquad \text{SVD takes this matrix apart into} \qquad A = U\Sigma V^T.$$

Let's take it one direction at a time. The **strongest direction**:

$$v_1^T = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \end{bmatrix}, \qquad \sigma_1 = 7, \qquad u_1 = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

Reading one direction

- v_1 is a special **input direction**.
- $\sigma_1 = 7$ tells us **how strongly** A stretches that direction.
- u_1 is the **output direction** it becomes after the transformation.

$$v_1 \xrightarrow{A} \sigma_1 u_1 \qquad \text{in fact} \qquad \boxed{Av_1 = \sigma_1 u_1}$$

Added note
Quick check: every row of A sums to 7, so $Av_1 = \begin{bmatrix}3.5 & 3.5 & 3.5 & 3.5\end{bmatrix}^T = 7 \cdot \begin{bmatrix}0.5 & 0.5 & 0.5 & 0.5\end{bmatrix}^T = \sigma_1 u_1$. ✓

Assembling A piece by piece

Now let's start assembling A_1 (strongest direction), A_2 (second strongest), ...

Keeping one direction:

$$A_1 = \sigma_1 \, u_1 v_1^T$$

$$u_1 v_1^T = \begin{bmatrix}0.5\\0.5\\0.5\\0.5\end{bmatrix} \begin{bmatrix}0.5 & 0.5 & 0.5 & 0.5\end{bmatrix} = \begin{bmatrix}0.25 & 0.25 & 0.25 & 0.25\\0.25 & 0.25 & 0.25 & 0.25\\0.25 & 0.25 & 0.25 & 0.25\\0.25 & 0.25 & 0.25 & 0.25\end{bmatrix}$$

$$\sigma_1 u_1 v_1^T = 7 \cdot u_1 v_1^T = \begin{bmatrix}1.75 & 1.75 & 1.75 & 1.75\\1.75 & 1.75 & 1.75 & 1.75\\1.75 & 1.75 & 1.75 & 1.75\\1.75 & 1.75 & 1.75 & 1.75\end{bmatrix}$$

That's A_1, but $A \neq A_1$: **we are throwing info away**.

We say If I'm only allowed to keep **ONE** direction/pattern from A, keep its **strongest** one.

And so on. With A_2 we keep the strongest 2, etc.:

$$\begin{aligned}\text{rank 2:} \quad & A_2 = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T\\ \text{rank 3:} \quad & A_3 = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \sigma_3 u_3 v_3^T\end{aligned}$$

Eventually, if we keep **all** the singular components, we reconstruct the original A exactly:

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \sigma_3 u_3 v_3^T + \sigma_4 u_4 v_4^T$$

And that is

$$A = U\Sigma V^T \qquad \text{or} \qquad A = \sum_i \sigma_i \, u_i v_i^T$$

Each $\sigma_i u_i v_i^T$ is a rank-1 "layer" of A, sorted from strongest to weakest.

Added note
The remaining pieces for this A (it's symmetric with positive eigenvalues, so here $u_i = v_i$ and the singular values equal the eigenvalues):

$$\sigma_2 = 5,\ v_2 = \tfrac{1}{2}\begin{bmatrix}1\\-1\\1\\-1\end{bmatrix} \qquad \sigma_3 = 3,\ v_3 = \tfrac{1}{2}\begin{bmatrix}1\\1\\-1\\-1\end{bmatrix} \qquad \sigma_4 = 1,\ v_4 = \tfrac{1}{2}\begin{bmatrix}1\\-1\\-1\\1\end{bmatrix}$$

So $\Sigma = \text{diag}(7,5,3,1)$, and $A_4 = A$. Since the weakest layer has $\sigma_4 = 1$, dropping it (A_3) loses little — this is exactly how SVD does compression and noise reduction (§1.1.2): keep the few largest σ_i, drop the rest.

Added note

Three more facts that tie this part together:

- U and V are **orthogonal** matrices (§1.1.4): pure rotations. That's why the recipe reads rotate → stretch → rotate, and why the formula ends in V^T ($V^{-1} = V^T$).
- Where they come from: $A^TA = V\Sigma^2V^T$ and $AA^T = U\Sigma^2U^T$. So the columns of V are the eigenvectors of A^TA, the columns of U are the eigenvectors of AA^T, and the σ_i^2 are the shared eigenvalues.
- **Eckart–Young theorem:** A_k is the *best* rank-k approximation there is — no rank-k matrix comes closer — and the error is exactly the discarded layers:

$$\|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \cdots + \sigma_r^2}.$$

  The cost of compression is known before you pay it.

SVD in ML

A_k is the best rank-k approximation of A: the strongest k patterns, with the weak ones (often noise) thrown away. This is the low-rank approximation that PCA and SVD use for compression.

1.1.9 Quick Reference

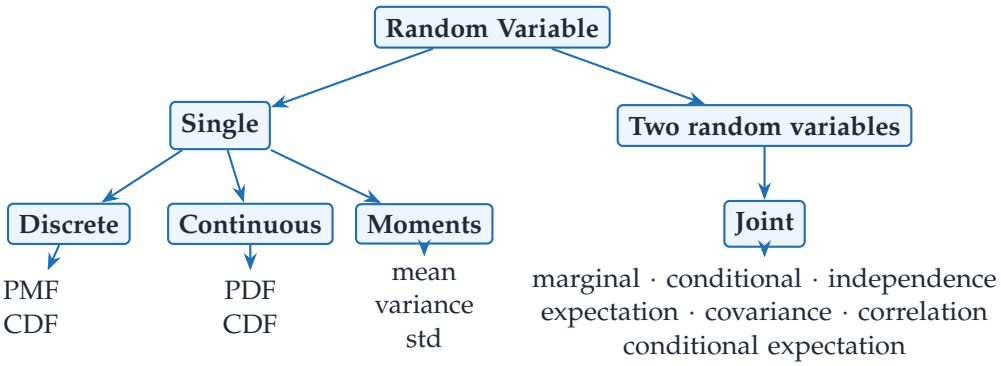
Concept	Formula / definition	Meaning
Matrix	$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$	a linear transformation of the grid
Determinant	$\lvert \det A \rvert$	how much the whole grid is stretched
	$\det A = 0$	basis fails to span the full space	a dimension was lost $\Rightarrow$ not invertible
Rank	# independent directions	# independent dims that survive
Max rank	$\min(m, n)$	never larger than the smallest dimension
Null space	$\{x : Ax = 0\}$	what the matrix is blind to
Rank–nullity	rank + nullity $= n$	survive + disappear = input dims
Eigenvector	$Av = \lambda v,\ v \neq 0$	direction doesn't change, only scales
Eigenvalues	$\det(A - \lambda I) = 0$	$A - \lambda I$ must have a non-trivial null space
Diagonalization	$A = PDP^{-1}$	P = eigenvectors, D = eigenvalues
Powers	$A^k = PD^kP^{-1}$	D^k is easy: power each diagonal entry
L_2 **norm**	$\sqrt{\sum x_i^2}$	Euclidean distance; circle; small weights
L_1 **norm**	$\sum \lvert x_i \rvert$	Manhattan distance; diamond; sparse weights
Ellipse	$x^T Ax = 1$	some directions more expensive
Regularized loss	$\min_w [L(w) + \lambda \lVert w \rVert]$	λ controls how much we care
SVD	$A = U\Sigma V^T$	rotate $\rightarrow$ stretch/shrink $\rightarrow$ rotate again
One SVD direction	$Av_i = \sigma_i u_i$	input dir v_i, stretched by σ_i, becomes u_i
SVD as layers	$A = \sum_i \sigma_i u_i v_i^T$	keep top k layers $\Rightarrow$ A_k (rank-k approx.)

Concept	Formula / definition	Meaning
Column space	$\mathrm{Col}(A) = \{A\mathbf{x}\}$ = span of columns	everything A can reach; rank = its dimension
Solving $A\mathbf{x} = \mathbf{b}$	solvable $\iff$ $\mathbf{b} \in \mathrm{Col}(A)$	full rank: one solution; low rank: none or ∞
Transpose	$(AB)^T = B^T A^T$	flip across the diagonal; order reverses
Inverse	$AA^{-1} = I$; exists $\iff$ $\det \neq 0$	undoes A; $(AB)^{-1} = B^{-1}A^{-1}$
Symmetric	$A^T = A$	real eigenvalues; $A = Q\Lambda Q^T$
Orthogonal	$Q^T Q = I$, so $Q^{-1} = Q^T$	pure rotation; preserves lengths/angles
Rank-one	$A = \mathbf{u}\mathbf{v}^T$	one pattern; what SVD layers are made of
L_∞ **/ Frobenius**	$\max_i \lvert x_i \rvert$; $\sqrt{\sum a_{ij}^2}$	biggest coordinate; L_2 of a flattened matrix
Eckart–Young	$\lVert A - A_k \rVert_F = \sqrt{\sum_{i>k} \sigma_i^2}$	A_k is the best rank-k approximation

PART 1.2

Probability & Statistics

The flow: Random variable → PMF / PDF → CDF → Moments (mean, variance, std) → Two random variables → Joint → Marginal → Conditional → Independence → Expectation of two variables → Covariance → Correlation → Conditional expectation → Law of total expectation.



1.2.1 A Single Random Variable

Sample space and random variables

The **sample space** Ω is the set of all possible outcomes of a random experiment: tossing two coins gives $\Omega = \{HH, HT, TH, TT\}$; measuring a temperature gives $\Omega = \mathbb{R}$.

A **random variable** X assigns a number to each outcome, so we can work with numbers instead of raw outcomes. Notation: capital X is the random variable, lowercase x is a particular value it took.

A random variable X is either **discrete** (it takes countable values, like $0, 1, 2$) or **continuous** (it takes any value in a range). Each one has its own way of describing probability.

Discrete: PMF

PMF

$$p_X(x) = P(X = x) \qquad \sum_x p_X(x) = 1$$

The PMF gives the probability **at one value**.

Discrete: CDF

CDF

$$F_X(x) = P(X \leq x)$$

So it accumulates probability as x increases.

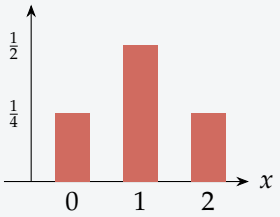
Example

x	$p_X(x)$
0	$\frac{1}{4}$
1	$\frac{1}{2}$
2	$\frac{1}{4}$
sum	1

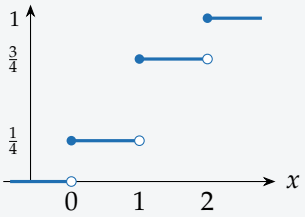
CDF at 1:

$$P(X \le 1) = P(X = 0) + P(X = 1)$$

$$\text{so} \quad P(X \le 1) = \tfrac{1}{4} + \tfrac{1}{2} = \tfrac{3}{4}$$



PMF: probability at each value



CDF: accumulates, ends at 1

Continuous: PDF

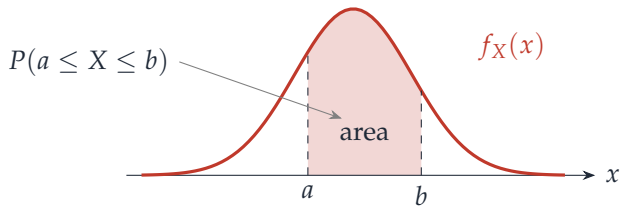
PDF

$$P(a \le X \le b) = \int_a^b f_X(x)\,dx \qquad \int_{-\infty}^{\infty} f_X(x)\,dx = 1$$

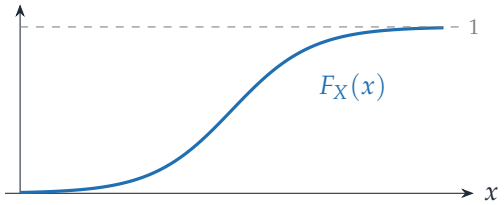
Rule For a continuous random variable, at a specific point:

$$P(X = x) = 0$$

So $f_X(x)$ is **not** interpreted as a probability. Probability comes from an **area**.



PDF: probability = area under the curve



CDF: area accumulated so far

Continuous: CDF

CDF

$$F_X(x) = P(X \le x) = \int_{-\infty}^{x} f_X(t)\,dt$$

It is the area accumulated from far left **up to** x.

PDF ↔ CDF

Since the CDF is the integral of the PDF, going back is just differentiating:

$$f_X(x) = \frac{d}{dx} F_X(x)$$

$$\text{PDF} \xrightarrow{\text{integrate}} \text{CDF}$$
$$\text{CDF} \xrightarrow{\text{differentiate}} \text{PDF}$$

1.2.2 Moments

Moments summarize a distribution with a few numbers: where it is centred, and how spread out it is.

1) Mean $E[X]$

Mean

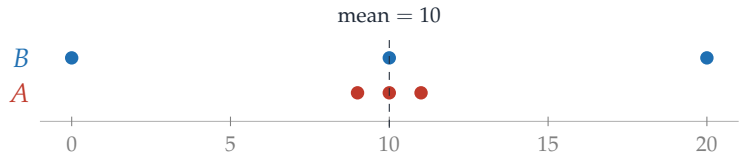
$$E[X] = \sum_x x\, p_X(x) \quad \text{(discrete)} \qquad E[X] = \int_{-\infty}^{\infty} x\, f_X(x)\, dx \quad \text{(continuous)}$$

Each term is **value** × **how likely it occurs**. We write $E[X] = \mu$.

The mean alone is not enough. Consider

$$A = \{9, 10, 11\} \qquad B = \{0, 10, 20\}$$

Both have mean = 10, but they are spread out differently.



To find deviations we can do (for A):

$$9 - 10 = -1, \qquad 10 - 10 = 0, \qquad 11 - 10 = 1 \qquad \Rightarrow \qquad \frac{-1 + 0 + 1}{3} = 0$$

The positives and negatives always cancel:

$$E[X - \mu] = 0 \quad \text{always}$$

→ So the solution is to **square the deviations**.

2) Variance

Variance

$$\text{Var}(X) = E\left[(X - \mu)^2\right]$$

Average squared distance from the mean.

Shortcut formula. Expand the square: $(X-\mu)^2 = X^2 - 2\mu X + \mu^2$

$$\begin{aligned}\mathrm{Var}(X) &= E\left[X^2 - 2\mu X + \mu^2\right] \\ &= E[X^2] - 2\mu \underbrace{E[X]}_{=\mu} + \mu^2 \\ &= E[X^2] - \mu^2\end{aligned}$$

$$\boxed{\mathrm{Var}(X) = E[X^2] - \left(E[X]\right)^2}$$

Back to A and B

$$\mathrm{Var}(A) = \frac{(-1)^2 + 0^2 + 1^2}{3} = \frac{2}{3} \qquad \mathrm{Var}(B) = \frac{(-10)^2 + 0^2 + 10^2}{3} = \frac{200}{3}$$

Same mean, but now the spread shows up: B is way more spread out.

3) Standard deviation

Std. deviation

$$\sigma = \sqrt{\mathrm{Var}(X)}$$

Since variance gives values in their units **squared**, the square root brings us back to the original units.

For the example: $\sigma_A = \sqrt{2/3} \approx 0.82$ and $\sigma_B = \sqrt{200/3} \approx 8.16$.

Moments in ML

- Mean and std are what we use to **standardize features**: $z = \dfrac{x-\mu}{\sigma}$, so every feature is centred at 0 with spread 1.
- The MSE loss is a variance-style quantity: the average squared distance between predicted and actual.

Common distributions

The named distributions are just ready-made PMFs/PDFs for situations that keep recurring.

Discrete:

Distribution	$P(X=k)$	$E[X]$, $\mathrm{Var}(X)$	Models
Bernoulli(p)	p if k=1; $1-p$ if k=0	p, $p(1-p)$	one yes/no outcome: a coin flip, a spam indicator
Binomial(n, p)	$\binom{n}{k}p^k(1-p)^{n-k}$	np, $np(1-p)$	# successes in n independent trials: defects in a batch
Poisson(λ)	$\dfrac{\lambda^k e^{-\lambda}}{k!}$	λ, λ	# events per window: arrivals per hour, packets per second
Multinomial	generalizes binomial	—	counts over K categories

Poisson has mean = variance = λ, which is a useful check: count data spread much more widely than its mean is not Poisson.

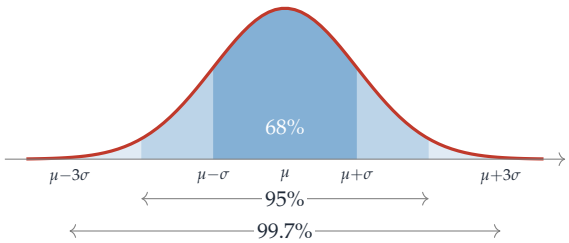
Example

$X \sim \text{Bernoulli}(0.3)$: $E[X] = 0.3$ and $\text{Var}(X) = E[X^2] - (E[X])^2 = 0.3 - 0.09 = 0.21$.

Continuous:

Distribution	Density	Models
Uniform(a, b)	constant $\frac{1}{b-a}$ on $[a, b]$	every value in the range equally likely
Normal(μ, σ^2)	bell curve centred at μ	noise, measurement errors, heights
Exponential(λ)	$\lambda e^{-\lambda x}$, $x \geq 0$	waiting time between Poisson events; memoryless: $P(X > s+t \mid X > s) = P(X > t)$
Multivariate Normal	normal in d dimensions	default model for continuous feature vectors

The 68–95–99.7 rule For a Normal(μ, σ^2): about 68% of the probability lies within $\mu \pm \sigma$, about 95% within $\mu \pm 2\sigma$, and about 99.7% within $\mu \pm 3\sigma$.



Sample mean and sample variance

Everything above assumes we *know* the distribution. In practice we have n i.i.d. samples $X_1, \ldots, X_n$ from an unknown distribution with true mean μ and variance σ^2, and we **estimate**:

$$\boxed{\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i} \qquad \boxed{S_n^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X}_n)^2}$$

- $E[\bar{X}_n] = \mu$: the sample mean is an unbiased estimate of the true mean.
- $\text{Var}(\bar{X}_n) = \sigma^2/n$: averaging n samples shrinks the noise by a factor of n. More data → steadier estimate.
- The $n-1$ (not n) makes S_n^2 unbiased: the samples sit slightly closer to their own mean $\bar{X}_n$ than to the true μ, and dividing by $n-1$ corrects exactly for that.

Example

Sample $(3, 5, 2, 4)$, $n = 4$: $\bar{X} = \dfrac{14}{4} = 3.5$ and

$$S^2 = \frac{(-0.5)^2 + (1.5)^2 + (-1.5)^2 + (0.5)^2}{3} = \frac{5}{3} \approx 1.67.$$

1.2.3 Two Random Variables: Joint Distributions

Now every outcome gives us **two** values (X, Y), and we care about how they behave *together*.

Joint PMF (discrete)

Joint PMF

$$p_{X,Y}(x,y) = P(X = x,\ Y = y) \qquad \sum_x \sum_y p_{X,Y}(x,y) = 1$$

It can be imagined as a **table**: rows are values of X, columns are values of Y, and every cell is a joint probability.

$X \backslash Y$	1	2	3
1	$p(1,1)$	$p(1,2)$	$p(1,3)$
2	$p(2,1)$	$p(2,2)$	$p(2,3)$
3	$p(3,1)$	$p(3,2)$	$p(3,3)$

Example: 80 students

	Studied	Did not study	Total
Passed	36	14	50
Did not pass	9	21	30
Total	45	35	80

- **Joint:** $P(\text{passed \& studied}) = \dfrac{36}{80}$
- **Marginal:** $P(\text{passed}) = \dfrac{50}{80}$
- **Conditional:** $P(\text{failed} \mid \text{studied}) = \dfrac{9}{45}$

Marginalization

Each cell gives only the joint distribution $p_{X,Y}(X = x,\ Y = y)$ → it lives **inside** the cells.

But…suppose we want only $P(X = 1)$: what is the probability that a randomly selected student studied? (Let $X = 1$ mean studied, $Y = 1$ mean passed.)

$X = 1$ happens either when $Y = 0$ or $Y = 1$. So sum up the row/column with $X = 1$:

$$P(X = 1) = P(X = 1, Y = 1) + P(X = 1, Y = 0) = \frac{36}{80} + \frac{9}{80} = \frac{45}{80}$$

That's exactly the column total. Marginal → usually looks at **row totals or column totals** (they're written in the margins of the table, hence the name).

Marginalization

$$P(X = x) = \sum_y P(X = x,\ Y = y)$$

To find the probability of $X = x$, add all joint probabilities containing $X = x$, **regardless of what Y does**. Summing across y, we **sum out** Y.



Continuous: joint PDF and marginals

For two continuous variables, the joint distribution is a **density** $f_{X,Y}(x,y)$. Marginalization works the same way, except we replace sums with integrals.

Continuous marginals To get the density of X alone, integrate over every possible value of y:

$$f_X(x) = \int_{-\infty}^{\infty} f_{X,Y}(x,y)\,dy \qquad f_Y(y) = \int_{-\infty}^{\infty} f_{X,Y}(x,y)\,dx$$

Integrate out the unwanted variable.

Same rule as before: a density is not a probability, probability comes from an area. With one variable that's $P(a \le X \le b) = \int_a^b f_X(x)\,dx$. Similarly, for two variables it's a **volume** over a region R:

$$P\big((X,Y) \in R\big) = \iint_R f_{X,Y}(x,y)\,dx\,dy$$

$$\text{e.g.} \qquad P(0 < X < 1,\ 2 < Y < 4) = \int_0^1 \int_2^4 f_{X,Y}(x,y)\,dy\,dx$$

1.2.4 Conditional Probability Distributions

- **Marginal** → "Only care about X, ignore Y."
- **Conditional** → "Given something about Y, how does that change what I believe about X?"

Conditional probability

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \qquad \text{PMF notation:} \quad p_{X|Y}(x \mid y) = \frac{p_{X,Y}(x,y)}{p_Y(y)}$$

Joint divided by marginal.

Example

$$P(X = 1 \mid Y = 1) = \frac{P(X = 1,\ Y = 1)}{P(Y = 1)}$$

We know $P(X = 1, Y = 1) = 0.4$ and $P(Y = 1) = 0.6$. So

$$P(X = 1 \mid Y = 1) = \frac{0.4}{0.6} = \frac{2}{3} \qquad \text{and} \qquad P(X = 0 \mid Y = 1) = \frac{1}{3}$$

$\frac{2}{3} + \frac{1}{3} = 1$. This **must** happen, because when we condition on $Y = 1$, we now have a **new probability distribution** over the possible values of X.

Conditionals sum to 1 So for a fixed $Y = y$:

$$\sum_x p_{X|Y}(x \mid y) = 1$$

Joint = Conditional × Marginal

Rearranging the definition:

$$P(X = x \mid Y = y) = \frac{P(X = x,\ Y = y)}{P(Y = y)} \quad \Longrightarrow \quad \underbrace{P(X = x \mid Y = y)}_{\text{conditional}} \cdot \underbrace{P(Y = y)}_{\text{marginal}} = \underbrace{P(X = x,\ Y = y)}_{\text{joint}}$$

Product rule

$$P(X, Y) = P(X \mid Y)\, P(Y) = P(Y \mid X)\, P(X)$$

Setting the two sides equal and dividing by $P(X)$ gives **Bayes' theorem**, which flips a conditional around:

$$\boxed{P(Y \mid X) = \frac{P(X \mid Y)\, P(Y)}{P(X)}}$$

Law of total probability

The same split-into-subgroups idea as marginalization: if $B_1, \ldots, B_k$ cut the sample space into pieces,

$$\boxed{P(A) = \sum_i P(A \mid B_i)\, P(B_i)}$$

Weight each subgroup's answer by how big the subgroup is. This is exactly what the denominator of Bayes' theorem is.

Bayes in action: a diagnostic test

Disease prevalence $P(D) = 0.01$, sensitivity $P(+ \mid D) = 0.95$, false-positive rate $P(+ \mid \bar{D}) = 0.05$.

$$P(+) = \underbrace{0.95(0.01)}_{\text{sick}} + \underbrace{0.05(0.99)}_{\text{healthy}} = 0.059 \qquad \Rightarrow \qquad P(D \mid +) = \frac{0.95(0.01)}{0.059} \approx 0.161$$

A positive test moves the probability from 1% to only ≈16% — nowhere near the 95% the sensitivity suggests. When a condition is rare, most positives come from the much larger healthy group.

Continuous conditionals

Same idea with densities:

$$f_{X|Y}(x \mid y) = \frac{f_{X,Y}(x, y)}{f_Y(y)} \qquad \text{so} \qquad f_{X,Y}(x, y) = f_{X|Y}(x \mid y) \cdot f_Y(y)$$

Conditionals in ML

A classifier is really learning a conditional distribution: $P(\text{label} \mid \text{features})$. "Given these features, what do I believe about the label?"

1.2.5 Independence

Independence

$$P(X = x \mid Y = y) = P(X = x)$$

This means that, given the value of y, that information does **not** change the probability distribution of x. Hence they are independent.

Formula for independence (derivation):

$$P(X = x \mid Y = y) = \frac{P(X = x,\ Y = y)}{P(Y = y)}$$

$$\text{if } X, Y \text{ independent:} \quad P(X = x \mid Y = y) = P(X = x)$$

$$\text{so} \quad P(X = x) = \frac{P(X = x,\ Y = y)}{P(Y = y)}$$

Test for independence

$$P(X = x,\ Y = y) = P(X = x) \cdot P(Y = y)$$

$$\textbf{or} \quad P(X \mid Y) = P(X) \qquad \textbf{or} \quad P(X, Y) = P(X)P(Y)$$

Continuous: $f_{X,Y}(x, y) = f_X(x) \cdot f_Y(y)$.

Example

	$Y = 0$	$Y = 1$	$P(X)$
$X = 0$	0.15	0.15	0.3
$X = 1$	0.35	0.35	0.7
$P(Y)$	0.5	0.5	1

Last row/column are the marginals.
If $X = 0, Y = 0$:
$P(X = 0, Y = 0) = 0.15$
Check: $P(X)P(Y) = 0.3 \times 0.5 = 0.15$ ✓
If you check every other cell, it matches (e.g. $0.7 \times 0.5 = 0.35$ ✓). Hence X, Y are **independent**.

One cell failing is enough to break independence. All cells have to match.

Independence in ML

Naive Bayes assumes the features are independent given the class, so the joint splits into a product: $P(x_1, \ldots, x_n \mid y) = \prod_i P(x_i \mid y)$. That's what makes it so cheap to compute.

Conditional independence

$X \perp Y \mid Z$ means: once you know Z, learning Y tells you nothing more about X:

$$P(X, Y \mid Z) = P(X \mid Z)\, P(Y \mid Z).$$

E.g. two fair coin flips are independent outright: every cell has $p(x, y) = \frac{1}{4} = \frac{1}{2} \cdot \frac{1}{2}$. Conditional independence is the version Naive Bayes above actually assumes: features independent *given the class*, not independent in general.

1.2.6 Expectation with Two Random Variables

Suppose every outcome gives us two values.

Expectation of X and Y individually

$$E[X] = \sum_x x\, p_X(x) \qquad E[Y] = \sum_y y\, p_Y(y)$$

Since $p_X(x) = \sum_y p_{X,Y}(x,y)$ (marginalization), hence

$$E[X] = \sum_x x \cdot \sum_y p_{X,Y}(x,y) \qquad \text{same as} \qquad \boxed{E[X] = \sum_x \sum_y x\, p_{X,Y}(x,y)}$$

Note: we go through every (x,y) pair, but only multiply the probability by x, because X is what we are averaging. Similarly:

$$E[Y] = \sum_y \sum_x y\, p_{X,Y}(x,y)$$

Expectation of a function of X, Y

Let $Z = g(X,Y)$, e.g. $Z = X + Y$ or $Z = XY$. So the expected value is:

Expectation of $g(X,Y)$

$$E[g(X,Y)] = \sum_x \sum_y g(x,y)\, p_{X,Y}(x,y)$$

So, as always: **value** · **probability**, summed over every cell.

Example: $E[XY]$

$X \backslash Y$	$Y=1$	$Y=2$
$X=1$	0.2	0.3
$X=2$	0.1	0.4

We want $E[XY]$, so for each cell find $xy \cdot P(X=x, Y=y)$:

$$\begin{aligned} E[XY] &= (1)(1)(0.2) + (1)(2)(0.3) \\ &\quad + (2)(1)(0.1) + (2)(2)(0.4) \\ &= 0.2 + 0.6 + 0.2 + 1.6 \\ E[XY] &= 2.6 \end{aligned}$$

$E[XY]$ is an **important quantity in covariance** (§1.2.7).

Expectation of a sum

$$\begin{aligned} E[X+Y] &= \sum_x \sum_y (x+y)\, p(x,y) \\ &= \underbrace{\sum_x \sum_y x \cdot p(x,y)}_{E[X]} + \underbrace{\sum_x \sum_y y \cdot p(x,y)}_{E[Y]} \end{aligned}$$

Linearity of expectation

$$E[X+Y] = E[X] + E[Y]$$

This is **regardless of independence**. It always works.

Expectation of a multiplication

Product of independent variables

$$X \perp Y \implies E[XY] = E[X]\,E[Y]$$

Only when X, Y are independent.

Why? Independence means $p_{X,Y}(x,y) = p_X(x)\,p_Y(y)$, so

$$E[XY] = \sum_x \sum_y x \cdot y \; p_X(x)\,p_Y(y)$$

which separates into

$$\Big(\sum_x x\,p_X(x)\Big) \cdot \Big(\sum_y y\,p_Y(y)\Big) \qquad \text{thus} \qquad E[XY] = E[X]\,E[Y]$$

When they are *not* independent, $E[XY]$ and $E[X]E[Y]$ differ, and that gap is exactly what leads to **covariance**.

Continuous version

With a joint PDF, sums become integrals:

$$E\big[g(X,Y)\big] = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} g(x,y)\; f_{X,Y}(x,y)\,dx\,dy \qquad E[XY] = \iint xy\; f_{X,Y}(x,y)\,dx\,dy$$

Mental model

$$(X,Y) \longrightarrow \text{joint distribution } p_{X,Y}(x,y) \longrightarrow E\big[g(X,Y)\big] = \sum_x \sum_y g(x,y)\,p(x,y)$$

$$\longrightarrow \quad E[X+Y] = E[X] + E[Y] \qquad \text{and} \qquad E[XY]$$

1.2.7 Covariance and Correlation

Covariance

Covariance measures how two variables move **together**: is X above its mean when Y is above its mean?

Covariance

$$\mathrm{Cov}(X,Y) = E\big[(X-\mu_X)(Y-\mu_Y)\big] = E[XY] - E[X]\,E[Y]$$

- This is the gap from the last section. If $X \perp Y$, then $E[XY] = E[X]E[Y]$, so $\mathrm{Cov}(X,Y) = 0$.
- $\mathrm{Cov}(X,X) = E[X^2] - (E[X])^2 = \mathrm{Var}(X)$. Variance is just covariance of a variable with itself.
- Careful: Cov $= 0$ does *not* mean independent. It only means no *linear* relationship.

Continuing the $E[XY] = 2.6$ example

Marginals from the table: $P(X=1) = 0.5$, $P(X=2) = 0.5$ and $P(Y=1) = 0.3$, $P(Y=2) = 0.7$.

$$E[X] = 1(0.5) + 2(0.5) = 1.5 \qquad E[Y] = 1(0.3) + 2(0.7) = 1.7$$

$$\mathrm{Cov}(X,Y) = E[XY] - E[X]E[Y] = 2.6 - (1.5)(1.7) = 2.6 - 2.55 = 0.05$$

Positive, so they tend to increase together (slightly).

Correlation

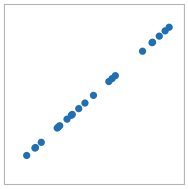
The problem with covariance is that its magnitude depends on the **units** of X and Y (measure height in cm instead of m and the covariance jumps by $100\times$). So we divide by their std. dev.:

Correlation

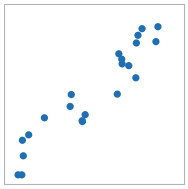
$$\rho_{X,Y} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y} \qquad \sigma_X = \sqrt{\mathrm{Var}(X)}, \quad \sigma_Y = \sqrt{\mathrm{Var}(Y)}$$

Forcing correlation into

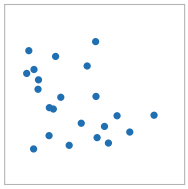
$$-1 \le \rho_{X,Y} \le 1$$



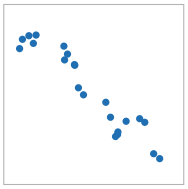
$\rho = 1$
linear correlation



$\rho > 0$
increase together



$\rho = 0$
no linear relation



$\rho < 0$
one up, other down

- $\rho > 0 \to$ increase together. $\rho = 1 \to$ perfect linear correlation.
- $\rho < 0 \to$ one goes up, the other goes down. $\rho = -1 \to$ perfect negative linear.
- $\rho = 0 \to$ no linear relationship.

Same example

$$\mathrm{Var}(X) = E[X^2] - (E[X])^2 = \big(1(0.5) + 4(0.5)\big) - 1.5^2 = 2.5 - 2.25 = 0.25 \;\Rightarrow\; \sigma_X = 0.5$$

$$\mathrm{Var}(Y) = \big(1(0.3) + 4(0.7)\big) - 1.7^2 = 3.1 - 2.89 = 0.21 \;\Rightarrow\; \sigma_Y \approx 0.458$$

$$\rho_{X,Y} = \frac{0.05}{(0.5)(0.458)} \approx 0.22$$

A weak positive correlation. Now it's a unit-free number we can actually compare.

Covariance & correlation in ML

- Two features with $|\rho|$ close to 1 carry almost the same info, which is the redundancy that lowers rank (§1.1.2).
- PCA works on the **covariance matrix** of the data: its eigenvectors are the directions of highest variation (§1.1.5).

1.2.8 Conditional Expectation

- **Normal expectation:** $E[X]$ is the avg value across the **entire population**.
- **Conditional expectation:** what is $E[X]$ given $Y = y$?

Conditional expectation

$$E[X \mid Y = y] = \sum_x x \, P(X = x \mid Y = y)$$

"The avg of X inside the subgroup where $Y = y$."

Compared with

$$E[X] = \sum_x x \, P(X = x)$$

the only difference is we replace the distribution with the **conditional distribution**.

For continuous:

$$E[X \mid Y = y] = \int_{-\infty}^{\infty} x \, f_{X|Y}(x \mid y) \, dx \qquad \text{where} \qquad f_{X|Y}(x \mid y) = \frac{f_{X,Y}(x, y)}{f_Y(y)}$$

Law of total expectation

Law of total expectation

Overall average = weighted average of subgroup averages

$$E[X] = \sum_y E[X \mid Y = y] \, P(Y = y) = E\big[E[X \mid Y]\big]$$

Example

X = exam score, Y = studied or not.

- $E[X \mid Y = 1] = 80 \rightarrow$ avg score of those who studied
- $E[X \mid Y = 0] = 60 \rightarrow$ avg score of those who didn't study

Suppose $P(Y = 1) = 0.75$ and $P(Y = 0) = 0.25$. Then

$$E[X] = 0.75(80) + 0.25(60) = 60 + 15 = 75$$

Each subgroup average gets weighted by how big that subgroup is.

Conditional expectation in ML

A regression model predicting y from x is trying to learn $E[Y \mid X = x]$: the average target value for inputs like this one. Under MSE loss, that's exactly the best possible prediction.

1.2.9 Putting It All Together: Discrete vs Continuous

Every idea in this part has a discrete version (sums) and a continuous version (integrals). Same logic, different tool.

	Discrete	Continuous
Single	PMF: $p_X(x) = P(X = x)$	PDF: $P(a \le X \le b) = \int_a^b f_X(x)\,dx$
CDF	$F_X(x) = \sum_{t \le x} p_X(t)$	$F_X(x) = \int_{-\infty}^{x} f_X(t)\,dt$
Mean	$E[X] = \sum_x x\, p_X(x)$	$E[X] = \int x\, f_X(x)\,dx$
Joint	$P(X, Y)$	density $f_{X,Y}(x, y)$
Marginals	$P(X) = \sum_y P(X, y)$ $P(Y) = \sum_x P(x, Y)$	$f_X(x) = \int f_{X,Y}(x, y)\,dy$ $f_Y(y) = \int f_{X,Y}(x, y)\,dx$
Conditionals	$P(X \mid Y) = \dfrac{P(X, Y)}{P(Y)}$	$f_{X\|Y}(x \mid y) = \dfrac{f_{X,Y}(x, y)}{f_Y(y)}$
Independence	does conditioning on Y change X? if not: $P(X \mid Y) = P(X)$, $P(X, Y) = P(X)P(Y)$	$f_{X,Y}(x, y) = f_X(x) \cdot f_Y(y)$
$E[g(X, Y)]$	$\sum_x \sum_y g(x, y)\, p(x, y)$	$\int\int g(x, y)\, f_{X,Y}(x, y)\,dx\,dy$

1.2.10 Quick Reference

Concept	Formula / definition	Meaning
PMF	$p_X(x) = P(X = x),\ \sum p_X(x) = 1$	probability at one value
PDF	$P(a \le X \le b) = \int_a^b f_X\,dx$	probability = area; $P(X = x) = 0$
CDF	$F_X(x) = P(X \le x)$	accumulated probability up to x
PDF ↔ CDF	$f_X(x) = \frac{d}{dx}F_X(x)$	integrate ⇄ differentiate
Mean	$E[X] = \sum x\,p_X(x)$	value × how likely
Variance	$E[(X-\mu)^2] = E[X^2] - (E[X])^2$	avg squared distance from mean
Std. deviation	$\sigma = \sqrt{\mathrm{Var}(X)}$	spread in original units
Joint	$p_{X,Y}(x,y) = P(X = x, Y = y)$	lives inside the cells
Marginal	$P(X = x) = \sum_y P(X = x, Y = y)$	sum out Y; row/column totals
Conditional	$P(X \mid Y) = P(X,Y)/P(Y)$	new distribution, sums to 1
Product rule	$P(X,Y) = P(X \mid Y)P(Y)$	joint = conditional × marginal
Bayes	$P(Y \mid X) = P(X \mid Y)P(Y)/P(X)$	flip a conditional
Independence	$P(X,Y) = P(X)P(Y)$	knowing Y doesn't change X
$E[g(X,Y)]$	$\sum\sum g(x,y)\,p(x,y)$	value · probability, every cell
Linearity	$E[X+Y] = E[X] + E[Y]$	always, regardless of independence
Product	$E[XY] = E[X]E[Y]$	only if $X \perp Y$
Covariance	$E[XY] - E[X]E[Y]$	move together? depends on units
Correlation	$\rho = \mathrm{Cov}/(\sigma_X\sigma_Y)$	unit-free, $-1 \le \rho \le 1$
Cond. expectation	$E[X \mid Y = y] = \sum x\,P(x \mid y)$	avg inside the subgroup $Y = y$
Total expectation	$E[X] = \sum_y E[X \mid Y = y]P(Y = y)$	weighted avg of subgroup avgs
Named dists	$E = p,\ np,\ \lambda$	yes/no; # successes; # events per window
Normal rule	68–95–99.7	within $\mu \pm \sigma$, 2σ, 3σ
Total probability	$P(A) = \sum_i P(A \mid B_i)P(B_i)$	weight subgroup answers by subgroup size
Sample mean/var	$\bar{X} = \frac{1}{n}\sum X_i;\ S^2 = \frac{\sum(X_i-\bar{X})^2}{n-1}$	$\mathrm{Var}(\bar{X}) = \sigma^2/n$; $n-1$ makes S^2 unbiased

PART 1.3

Calculus & Optimization

The flow: Limit → derivative → partial derivative → gradient → Jacobian → vector/matrix derivatives → chain rule → matrix identities → convexity → optimization.

The goal of this part is not to relearn an entire calculus course. We only need the pieces that explain how a machine-learning loss changes when we change its parameters. In ML, derivatives are fundamentally **sensitivity measurements**.

Why calculus matters in ML

A model has parameters $\boldsymbol{\theta}$ and a loss $L(\boldsymbol{\theta})$. Training asks: "If I change each parameter a tiny amount, which way makes the loss go down?" The derivative/gradient answers exactly that question.

1.3.1 Limits & Derivatives

Added note

Limits are listed in the course outline but are not developed in the Parr–Howard matrix-calculus paper. Only the amount needed to understand the derivative definition is included here.

Limits

A limit describes what a function approaches as its input approaches some value:

$$\boxed{\lim_{x \to a} f(x) = L}$$

It asks what happens *near* a; the value $f(a)$ itself can even be missing. Limits let us define an instantaneous rate of change by shrinking a finite change down toward zero.

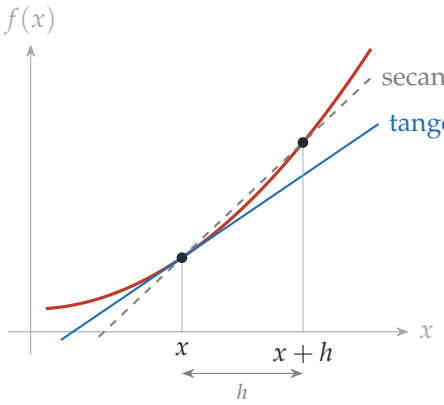
Derivative = local rate of change

For $y = f(x)$, the derivative is

Derivative

$$f'(x) = \frac{df}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

It is the slope of the tangent line, or equivalently the **sensitivity of the output to a tiny change in the input**.



shrink $h \to 0$: the secant turns into the tangent

For a small change Δx,

$$\boxed{f(x+\Delta x) \approx f(x) + f'(x)\,\Delta x}$$

So if $f'(3) = 6$, then near $x = 3$ a change $\Delta x = 0.01$ changes the output by roughly $6(0.01) = 0.06$.

Example

For $f(x) = x^2$,

$$\frac{df}{dx} = 2x.$$

At $x = 3$, the slope is 6. The derivative is not just a formula; it says how strongly the output reacts *at that point*.

Scalar derivative rules

Rule	Formula	Example
Constant	$\frac{d}{dx}c = 0$	$\frac{d}{dx}99 = 0$
Constant multiple	$\frac{d}{dx}[cf] = cf'$	$\frac{d}{dx}(3x) = 3$
Power	$\frac{d}{dx}x^n = nx^{n-1}$	$\frac{d}{dx}x^3 = 3x^2$
Sum/difference	$(f \pm g)' = f' \pm g'$	$\frac{d}{dx}(x^2 + 3x) = 2x + 3$
Product	$(fg)' = f'g + fg'$	$\frac{d}{dx}(x^2 x) = 3x^2$
Quotient	$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$	$\frac{d}{dx}\frac{1}{x} = -\frac{1}{x^2}$
Chain	$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$	$y = \sin(x^2) \to 2x\cos(x^2)$

ML interpretation

$\dfrac{\partial L}{\partial w_i}$ means: if weight w_i moves a tiny amount, how strongly and in which direction does the loss L move?

1.3.2 Partial Derivatives & Gradients

Neural-network functions depend on many parameters, so one ordinary derivative is no longer enough.

Partial derivatives

For $f(x, y)$, we differentiate with respect to **one variable at a time**, treating the others as constants:

$$\frac{\partial f}{\partial x} = \text{sensitivity to } x \text{ while } y \text{ is fixed}, \qquad \frac{\partial f}{\partial y} = \text{sensitivity to } y \text{ while } x \text{ is fixed}.$$

Example: $f(x, y) = 3x^2y$

With respect to x, treat $3y$ as constant:

$$\boxed{\frac{\partial f}{\partial x} = 6xy}$$

With respect to y, treat $3x^2$ as constant:

$$\boxed{\frac{\partial f}{\partial y} = 3x^2}$$

Gradient

For one scalar output $f(\mathbf{x})$ with many inputs, collect all partial derivatives into one vector.

Gradient

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} & \cdots & \frac{\partial f}{\partial x_n} \end{bmatrix}$$

Following the Parr–Howard paper's **numerator-layout convention**, the gradient of a scalar output is written as a **row** vector. Some ML texts use a column gradient instead; that is simply the transpose. Be consistent with shapes.

For $f(x, y) = 3x^2y$,

$$\boxed{\nabla f = [\, 6xy,\ 3x^2 \,]}$$

Geometric meaning

The gradient points in the direction of **steepest increase**. Therefore $-\nabla f$ points in the direction of steepest local **decrease**. This is the central idea behind gradient descent.

For a small vector change $\Delta\mathbf{x}$,

$$\boxed{\Delta f \approx \nabla f \, \Delta\mathbf{x}}$$

(in the paper's row-gradient convention). This is the multivariable version of $\Delta f \approx f'(x)\,\Delta x$.

1.3.3 Vector & Matrix Derivatives

From derivative → gradient → Jacobian

Mapping	Derivative object	Shape
scalar → scalar	ordinary derivative	1×1
vector → scalar	gradient	$1 \times n$
vector → vector	Jacobian	$m \times n$

Suppose $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{y} = \mathbf{f}(\mathbf{x}) \in \mathbb{R}^m$, where each output $y_i = f_i(\mathbf{x})$ is a scalar function.

Jacobian

$$J = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

Each **row** is the gradient of one output. Therefore

$$J \in \mathbb{R}^{m \times n} : \qquad \text{one row per output, one column per input.}$$

Example

Let

$$f(x,y) = 3x^2 y, \qquad g(x,y) = 2x + y^8.$$

Then

$$\nabla f = [\,6xy,\ 3x^2\,], \qquad \nabla g = [\,2,\ 8y^7\,],$$

so

$$\boxed{J = \begin{bmatrix} 6xy & 3x^2 \\ 2 & 8y^7 \end{bmatrix}}$$

For the identity function $\mathbf{y} = \mathbf{x}$,

$$\frac{\partial x_i}{\partial x_j} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases} \qquad \Rightarrow \qquad \boxed{\frac{\partial \mathbf{x}}{\partial \mathbf{x}} = I}$$

Element-wise operations ⇒ diagonal Jacobians

If each output y_i depends only on the matching input x_i, then

$$\frac{\partial y_i}{\partial x_j} = 0 \qquad (i \neq j),$$

so all off-diagonal Jacobian entries vanish.

Element-wise diagonal condition

$$y_i \text{ depends only on } x_i \quad \Rightarrow \quad J \text{ is diagonal.}$$

Useful cases:

$$\boxed{\frac{\partial(\mathbf{w}+\mathbf{x})}{\partial \mathbf{w}} = I, \qquad \frac{\partial(\mathbf{w}+\mathbf{x})}{\partial \mathbf{x}} = I}$$

$$\boxed{\frac{\partial(\mathbf{w}\odot\mathbf{x})}{\partial \mathbf{w}} = \operatorname{diag}(\mathbf{x}), \qquad \frac{\partial(\mathbf{w}\odot\mathbf{x})}{\partial \mathbf{x}} = \operatorname{diag}(\mathbf{w})}$$

Here $\odot$ is element-wise (Hadamard) multiplication. This ties back to diagonal matrices (§1.1.4): each direction is handled on its own, nothing gets mixed.

Scalar expansion

A scalar interacting with a vector is implicitly copied across all entries:

$$\mathbf{x}+z = \begin{bmatrix} x_1+z \\ \vdots \\ x_n+z \end{bmatrix}, \qquad z\mathbf{x} = \begin{bmatrix} zx_1 \\ \vdots \\ zx_n \end{bmatrix}.$$

Hence

$$\boxed{\frac{\partial(\mathbf{x}+z)}{\partial \mathbf{x}} = I, \qquad \frac{\partial(\mathbf{x}+z)}{\partial z} = \mathbf{1}} \quad \text{and} \quad \boxed{\frac{\partial(z\mathbf{x})}{\partial \mathbf{x}} = zI, \qquad \frac{\partial(z\mathbf{x})}{\partial z} = \mathbf{x}}$$

Vector reduction: vector $\to$ scalar

Summing a vector gives a scalar:

$$y = \sum_i x_i.$$

Every x_i contributes with derivative 1, so

$$\boxed{\frac{\partial}{\partial \mathbf{x}} \sum_i x_i = \mathbf{1}^T}$$

The dot product is a sum of element-wise products:

$$y = \mathbf{w}^T\mathbf{x} = \sum_i w_i x_i.$$

Therefore

$$\boxed{\frac{\partial(\mathbf{w}^T\mathbf{x})}{\partial \mathbf{x}} = \mathbf{w}^T, \qquad \frac{\partial(\mathbf{w}^T\mathbf{x})}{\partial \mathbf{w}} = \mathbf{x}^T}$$

under the paper's row-gradient convention.

> **Shape check**
> Never treat dimensions as decoration. If $\mathbf{y} \in \mathbb{R}^m$ and $\mathbf{x} \in \mathbb{R}^n$, then $\partial\mathbf{y}/\partial\mathbf{x}$ **must** be $m \times n$. Shape checking catches many matrix-calculus mistakes immediately.

1.3.4 The Chain Rule

The chain rule handles **nested computations**. Deep networks are just long chains of nested transformations, so this is the mathematical core of backpropagation.

1) Single-variable chain rule

If

$$u = g(x), \qquad y = f(u),$$

then

Chain rule

$$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$$

Example: $y = \sin(x^2)$

Let $u = x^2$, $y = \sin u$. Then

$$\frac{du}{dx} = 2x, \qquad \frac{dy}{du} = \cos u,$$

so

$$\boxed{\frac{dy}{dx} = 2x\cos(x^2)}$$

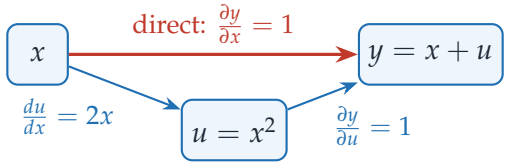
The practical recipe is: **break into simple intermediate variables → differentiate each local step → multiply the local derivatives.**

2) Total derivative: when there are multiple paths

Sometimes x affects the output in more than one way. Example:

$$y = x + x^2.$$

Let $u = x^2$, so $y = x + u$. There is a **direct** path $x \to y$ and an **indirect** path $x \to u \to y$.



Total-derivative idea Add the contribution from **every path** by which x affects the output:

$$\frac{dy}{dx} = \frac{\partial y}{\partial x} + \frac{\partial y}{\partial u}\frac{du}{dx}$$

For $y = x + x^2$: $1 + (1)(2x) = 1 + 2x$.

More generally, if f depends directly on x and indirectly through $u_1, \ldots, u_n$,

$$\boxed{\frac{df}{dx} = \frac{\partial f}{\partial x} + \sum_{i=1}^{n} \frac{\partial f}{\partial u_i}\frac{du_i}{dx}}$$

3) Vector chain rule

Let

$$\mathbf{u} = \mathbf{g}(\mathbf{x}), \qquad \mathbf{y} = \mathbf{f}(\mathbf{u}).$$

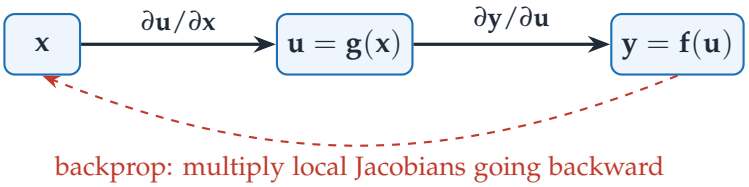
Then the scalar multiplication of derivatives becomes **matrix multiplication of Jacobians**:

Vector chain rule

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{x}}$$

Shape check:

$$(m \times k)(k \times n) = m \times n.$$



Backpropagation

A neural network is a computation graph. Backpropagation starts from the scalar loss and repeatedly applies the chain rule **backward**, multiplying by each local derivative. Modern libraries do this automatically; matrix calculus explains what the autograd engine is computing.

For a simple neuron

$$z = \mathbf{w}^T \mathbf{x} + b, \qquad a = \sigma(z), \qquad L = L(a),$$

the parameter gradient follows the chain

$$\boxed{\nabla_{\mathbf{w}} L = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \mathbf{x}}$$

(using the common column-gradient ML notation here). The important idea is not memorizing this one expression: it is **local derivatives multiplied through the graph**.

1.3.5 Matrix Calculus Identities

Added note
The Parr–Howard paper derives the Jacobian, element-wise, scalar-expansion, reduction, dot-product, and chain-rule identities above. The following compact table also includes standard identities commonly needed immediately afterward in ML derivations.

For this table, $\nabla_{\mathbf{x}} f$ is written in the common ML **column-gradient** convention. A and vectors not being differentiated are treated as constants.

Expression	Derivative / gradient
c	$\nabla_{\mathbf{x}} c = \mathbf{0}$
$\mathbf{a}^T\mathbf{x}$	$\nabla_{\mathbf{x}}(\mathbf{a}^T\mathbf{x}) = \mathbf{a}$
$\mathbf{x}^T\mathbf{x} = \|\mathbf{x}\|_2^2$	$\nabla_{\mathbf{x}}(\mathbf{x}^T\mathbf{x}) = 2\mathbf{x}$
$\mathbf{x}^T A\mathbf{x}$	$\nabla_{\mathbf{x}}(\mathbf{x}^T A\mathbf{x}) = (A + A^T)\mathbf{x}$
$\mathbf{x}^T A\mathbf{x},\ A = A^T$	$\nabla_{\mathbf{x}}(\mathbf{x}^T A\mathbf{x}) = 2A\mathbf{x}$
$\|A\mathbf{x} - \mathbf{b}\|_2^2$	$\nabla_{\mathbf{x}} = 2A^T(A\mathbf{x} - \mathbf{b})$
$\mathbf{y} = A\mathbf{x} + \mathbf{b}$	$J_{\mathbf{x}}\mathbf{y} = A$
$\mathbf{y} = g(\mathbf{x})$ element-wise	$J = \operatorname{diag}\big(g'(x_1), \ldots, g'(x_n)\big)$
$\mathbf{a}^T W\mathbf{b}$	$\nabla_W(\mathbf{a}^T W\mathbf{b}) = \mathbf{a}\mathbf{b}^T$
$\operatorname{tr}(W^T A)$	$\nabla_W \operatorname{tr}(W^T A) = A$
$\|W\|_F^2 = \sum_{ij} W_{ij}^2$	$\nabla_W \|W\|_F^2 = 2W$

Why the quadratic-loss identity matters

Least squares uses

$$L(\mathbf{x}) = \|A\mathbf{x} - \mathbf{b}\|_2^2.$$

Its gradient is

$$\boxed{\nabla_{\mathbf{x}} L = 2A^T(A\mathbf{x} - \mathbf{b})}$$

The term $(A\mathbf{x} - \mathbf{b})$ is the **residual/error**; multiplying by A^T maps that output-space error back to parameter space. This is the squared L_2 norm from §1.1.7 being used as a loss.

Trace trick

A scalar can always be written as its own trace, and trace lets us cyclically rotate matrix products:

$$\boxed{\operatorname{tr}(ABC) = \operatorname{tr}(BCA) = \operatorname{tr}(CAB)}$$

This is useful for rewriting scalar losses into a form where the matrix being differentiated is easy to isolate.

Practical rule

If a matrix derivative looks terrifying, reduce the expression to **scalar entries**, differentiate those with ordinary partial-derivative rules, and then reassemble the vector/matrix. This is also the strategy emphasized in the matrix-calculus paper.

1.3.6 Convexity & Optimization

Added note

Convexity is part of the course outline but is outside the scope of the Parr–Howard paper. This subsection gives the standard ML background needed to interpret loss landscapes and optimization.

Optimization problem

Training usually means

$$\boxed{\min_{\boldsymbol{\theta}} L(\boldsymbol{\theta})}$$

where $\boldsymbol{\theta}$ contains the model parameters.

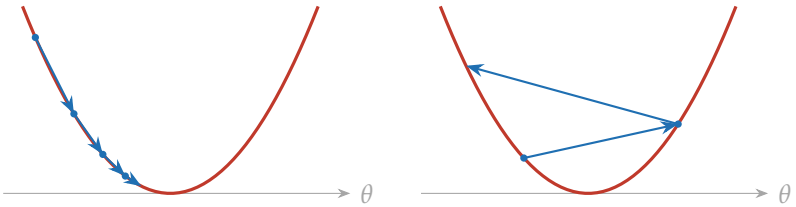
Gradient descent

Because ∇L points uphill, move in the opposite direction:

Gradient descent update

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \, \nabla L(\boldsymbol{\theta}_t)$$

$\eta > 0$ is the **learning rate** (step size).



η good: steady progress η too large: overshoots

- η too small $\rightarrow$ training is slow.
- η too large $\rightarrow$ overshooting/instability is possible.
- $\nabla L = \mathbf{0} \rightarrow$ a **stationary point**, but not automatically a minimum.

Second derivatives and the Hessian

For one variable, at a stationary point:

$$f''(x) > 0 \;\Rightarrow\; \text{local minimum}, \qquad f''(x) < 0 \;\Rightarrow\; \text{local maximum}.$$

For many variables, collect second partial derivatives into the **Hessian**:

Hessian

$$H_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

The Hessian describes **local curvature**.

At a stationary point, roughly:

- $H \succ 0$ (positive definite) $\rightarrow$ curves upward in every direction $\rightarrow$ local **minimum**.
- $H \prec 0 \rightarrow$ curves downward in every direction $\rightarrow$ local **maximum**.
- mixed positive/negative curvature $\rightarrow$ **saddle point**.

Positive definite just means all the Hessian's eigenvalues are positive (§1.1.5): every eigen-direction curves up.

Convex functions

Convexity A function f is convex if, for any $\mathbf{x}, \mathbf{y}$ and $0 \leq \lambda \leq 1$,

$$f\big(\lambda \mathbf{x} + (1-\lambda)\mathbf{y}\big) \leq \lambda f(\mathbf{x}) + (1-\lambda) f(\mathbf{y}).$$

Geometrically: the straight line joining two points on the graph never falls below the function.

For twice-differentiable f,

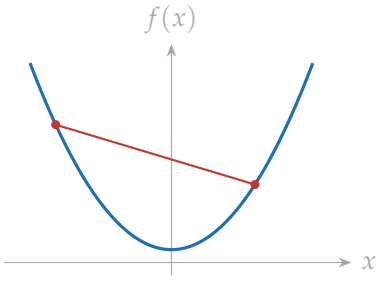
$$\boxed{f \text{ convex} \iff H_f(\mathbf{x}) \succeq 0 \text{ everywhere.}}$$

A **convex set** is a set with no dents: for any two points in it, the whole line segment between them stays inside. (A convex function needs a convex domain; in constrained optimization, a convex constraint region is what keeps "local = global" alive.)

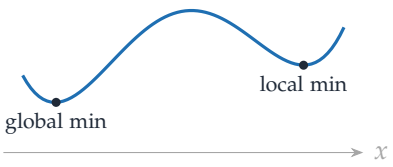
First-order test. For differentiable f, convexity is equivalent to

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^T (\mathbf{y} - \mathbf{x}) \qquad \text{for all } \mathbf{x}, \mathbf{y}$$

— the graph sits **above every tangent**. So any point with $\nabla f = \mathbf{0}$ (a flat tangent) is instantly a **global** minimum. That is the convexity magic in one line.



convex bowl: the chord stays above



non-convex: several minima

Why convexity matters

In a convex problem, **every local minimum is global**, so optimization is much easier to reason about. Linear regression with squared loss is convex. Deep neural networks are generally **non-convex**, so their landscapes can contain saddles and many different minima.

Loss landscapes and contours

For two parameters w_1, w_2, a loss such as

$$L(w_1, w_2) = w_1^2 + w_2^2$$

is a 3D bowl. A contour plot is the bird's-eye view (same picture as §1.1.7): each contour contains parameter values with the same loss. The negative gradient points **perpendicular** to the contour toward decreasing loss.

Simple optimization example

For $L(\mathbf{w}) = \mathbf{w}^T\mathbf{w}$, the gradient is $\nabla L = 2\mathbf{w}$. Gradient descent gives

$$\mathbf{w}_{t+1} = \mathbf{w}_t - 2\eta\mathbf{w}_t = (1 - 2\eta)\,\mathbf{w}_t,$$

so, for a suitable step size, the weights repeatedly shrink toward the minimum $\mathbf{w} = \mathbf{0}$.
The "suitable" step size here is $0 < \eta < 1$, so that $|1 - 2\eta| < 1$ and each step really does shrink $\mathbf{w}$. With $\eta > 1$, the factor is below -1 and the weights flip sign and grow: exactly the overshooting picture above.

1.3.7 Quick Reference

Concept	Formula / definition	Meaning
Limit	$\lim_{x\to a} f(x) = L$	what f approaches near a
Derivative	$f'(x) = \lim_{h\to 0} \frac{f(x+h)-f(x)}{h}$	local slope / sensitivity
Local linearization	$\Delta f \approx f'(x)\Delta x$	tiny input change → approx. output change
Partial derivative	$\partial f/\partial x_i$	sensitivity to one variable, others fixed
Gradient	$\nabla f = [\partial f/\partial x_1, \dots]$	all partials of one scalar output
Jacobian	$J = \partial \mathbf{y}/\partial \mathbf{x}$	all output-vs-input sensitivities
Jacobian shape	$m \times n$	m outputs, n inputs
Identity derivative	$\partial \mathbf{x}/\partial \mathbf{x} = I$	each x_i only depends on itself
Element-wise op.	off-diagonal partials $= 0$	Jacobian becomes diagonal
Scalar expansion	$\partial(z\mathbf{x})/\partial \mathbf{x} = zI$	scalar copied across vector
Vector sum	$\partial \sum_i x_i/\partial \mathbf{x} = \mathbf{1}^T$	every element contributes once
Dot product	$\partial(\mathbf{w}^T\mathbf{x})/\partial \mathbf{x} = \mathbf{w}^T$	weighted sum sensitivity
Scalar chain rule	$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$	multiply local derivatives
Total derivative	direct path + all indirect paths	sum every route from input to output
Vector chain rule	$J_{f\circ g} = J_f J_g$	chain rule becomes matrix multiplication
Quadratic form	$\nabla_{\mathbf{x}}(\mathbf{x}^T A\mathbf{x}) = (A + A^T)\mathbf{x}$	core matrix identity
Squared residual	$\nabla\|A\mathbf{x} - \mathbf{b}\|^2 = 2A^T(A\mathbf{x} - \mathbf{b})$	least-squares gradient
Gradient descent	$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\nabla L$	move opposite steepest increase
Hessian	$H_{ij} = \partial^2 f/(\partial x_i \partial x_j)$	local curvature
Convex	$f(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) \le \lambda f(\mathbf{x}) + (1-\lambda)f(\mathbf{y})$	local minima are global
First-order test	$f(\mathbf{y}) \ge f(\mathbf{x}) + \nabla f(\mathbf{x})^T(\mathbf{y} - \mathbf{x})$	graph sits above every tangent
Convexity test	$H \succeq 0$ everywhere	nonnegative curvature in all directions

Big picture In ML, the whole part collapses to one storyline:

loss $L(\boldsymbol{\theta})$ → gradient ∇L → chain rule/backprop → update parameters to reduce L.

Everything else in this section is machinery that makes that computation possible for vectors and matrices.

Source basis. Sections on partial derivatives, gradients, Jacobians, element-wise operations, scalar expansion, vector reductions, and chain rules follow Terence Parr and Jeremy Howard, *The Matrix Calculus You Need For Deep Learning* (2018). Items marked "Added note" provide standard background required by the MLR 570 section outline but not developed in that paper.

CHAPTER 2

2 · Data Fundamentals

Everything that happens before you fit a model and everything that happens after: preparing data, measuring similarity, and judging honestly whether the result works.

PART 2.1

Data Preprocessing

The flow: Why preprocess at all → aggregation → sampling → normalization → encoding → discretization → dimensionality reduction → PCA → feature selection.

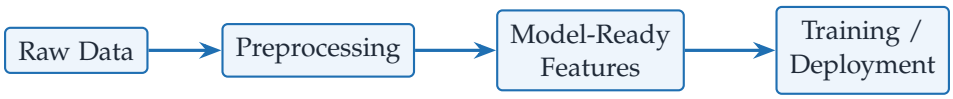
Raw data is rarely ready to give directly to a machine-learning model. Measurements may be noisy, incomplete, differently scaled, or represented in incompatible formats. **Preprocessing** turns that raw data into a cleaner, more consistent representation that the model can actually use.

Why preprocessing matters in ML

Good preprocessing helps a model learn meaningful patterns, generalize to unseen data, converge more efficiently during training, and avoid problems such as one feature dominating simply because its numerical scale is much larger than another.

2.1.1 Why Preprocess at All?

The basic pipeline is



The major operations in this part are:

- **Aggregation:** combine several observations into a summary.
- **Sampling:** select a representative subset of a larger dataset.
- **Normalization:** rescale numerical features to comparable ranges.
- **Encoding:** convert categorical values into numerical representations.
- **Discretization:** convert continuous values into a finite set of bins.
- **Dimensionality reduction / feature selection:** reduce the number of dimensions while keeping useful information.

Scale can distort geometry

Suppose Feature 1 has values around 10^4 while Feature 2 has values around 10. A distance-based method can become dominated by Feature 1, not because Feature 1 is more informative, but simply because its numbers are much larger. Scaling can restore a geometry in which both features are comparable.

E.g. for two points differing by 1000 in Feature 1 and by 5 in Feature 2:

$$\|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{1000^2 + 5^2} \approx 1000.01$$

Feature 2 contributes almost nothing to the distance (§1.1.7).

2.1.2 Aggregation

Aggregation Summarizes or combines data points in order to reduce size or complexity and expose higher-level structure such as trends.

Common summaries are

$$\text{sum} = \sum_{i=1}^{n} x_i, \qquad \text{mean} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \text{median} = \text{middle ordered value.}$$

The **mean** is simple and efficient but sensitive to outliers. The **median** is more robust to extreme values, although it throws away more fine-grained variation. Aggregation can also be spatial or windowed, e.g. hourly temperatures → daily average temperatures.

Mean vs median

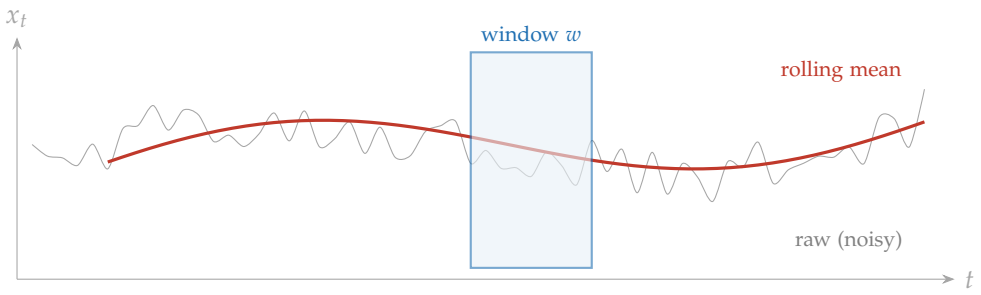
For $\{2, 3, 3, 4, 100\}$: mean $= 112/5 = 22.4$, median $= 3$.
One outlier drags the mean far away from where most of the data sits; the median doesn't move.

Rolling / windowed aggregation

For a univariate time series $\{x_t\}_{t=1}^{T}$ and a right-aligned window of size w,

$$\boxed{\bar{x}_t = \frac{1}{w}\sum_{i=0}^{w-1} x_{t-i}} \qquad \boxed{\tilde{x}_t = \text{median}(x_{t-w+1}, \ldots, x_t)}$$

The window uses the current and previous $w-1$ samples; **no future sample is used**. Therefore the first $w-1$ outputs are undefined until a complete window exists.



What rolling aggregation does

Averaging suppresses high-frequency noise and makes the slower trend easier to see. A rolling median does the same while being less affected by isolated spikes.

Added intuition — the bias–variance trade-off here

A **small** window reacts quickly to changes, so it has less lag (lower bias) but follows noise more strongly (higher variance). A **large** window is smoother (lower variance) but contains more old values, so it lags behind a changing signal (higher bias). Think: *responsive but noisy* $\leftrightarrow$ *smooth but laggy.*

Larger w therefore gives stronger smoothing and trend extraction, but it can hide short-term dynamics. There is no universally optimal window size; it depends on the signal-to-noise ratio, the domain, and the downstream task.

2.1.3 Sampling

Sampling Selects a subset of a larger dataset or population. The goal is usually to reduce computational cost while keeping the subset **representative**; sampling can also be used to address class imbalance.

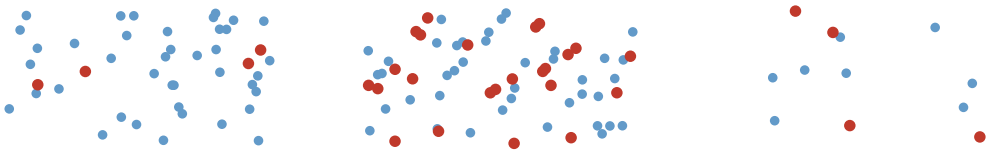
Method	Main idea	Main risk / limitation
Random	Every observation has equal probability of selection.	Rare subgroups may be missed.
Stratified	Split into known strata and sample proportionally from each.	Requires strata / class labels.
Oversampling	Increase the minority class by duplication or synthetic generation.	Overfitting or unrealistic synthetic samples.
Undersampling	Remove observations from the majority class.	Useful information can be discarded.
Systematic	Select every kth observation after a random start.	Hidden periodicity can introduce bias.

Stratified sampling

If only 5% of a medical dataset belongs to a rare positive class, a stratified train/validation split keeps roughly 5% positives in each subset instead of leaving that proportion to chance.

Imbalanced classification

Oversampling makes the minority class larger; undersampling makes the majority class smaller. Both can produce a more balanced training set, but they change the data in different ways. In sensitive applications, synthetic minority samples should only be used when their realism is defensible.



original: 40 vs 4 oversample minority undersample majority

Advanced strategies — recognize the names

- **Importance sampling:** chooses observations according to task-specific importance.
- **Cluster sampling:** selects groups rather than individuals.
- **Reservoir sampling:** maintains a random sample from a stream of unknown length.
- **Adaptive sampling:** focuses on informative regions.
- **Synthetic methods** such as SMOTE or GAN-based methods: generate new samples.

2.1.4 Normalization

Normalization / scaling Puts numerical features on comparable scales so that a large numerical magnitude is **not mistaken for greater importance**.

This matters particularly for distance-based methods such as k-NN and K-means, and for gradient-based optimization where poor scaling can slow or destabilize convergence (§1.3.6).

Min–Max scaling

Usually maps a feature to $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

The minimum becomes 0, the maximum becomes 1, and values in between are mapped proportionally. It is simple and intuitive, but because $x_{\min}$ and $x_{\max}$ define the entire scale, **extreme outliers can distort the transformation**.

Example

Ages $\{20, 30, 40, 60\}$: $x_{\min} = 20$, $x_{\max} = 60$, so $30 \mapsto \dfrac{30 - 20}{60 - 20} = 0.25$.

Add one outlier age of 100 and now $30 \mapsto \dfrac{10}{80} = 0.125$: every normal value gets squeezed toward 0.

Z-score standardization

Centers and rescales using the feature mean and standard deviation (§1.2.2):

$$x' = \frac{x - \mu}{\sigma}$$

After standardization the feature has **mean 0 and standard deviation 1** in the data used to fit the scaler. A standardized value of $x' = 2$ means the observation lies two standard deviations above the fitted mean.

Why scaling matters

Age may be measured on the order of 10^1, income on the order of 10^5, and credit score below 10^3. Without scaling, the largest-magnitude feature can dominate geometric calculations even if it is not the most predictive feature.

Practical rule: fit on training data only Compute $x_{\min}, x_{\max}$ or μ, σ from the **training set**, then use those same fitted values to transform training, validation, and test data.

For example,

$$x'_{\text{test}} = \frac{x_{\text{test}} - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

not a new test-set mean and standard deviation. The validation/test sets are meant to behave like **unseen data**; computing statistics on them leaks information about the test set into training.

Further methods mentioned in the lecture include robust scaling (median/IQR), log or power transforms, L2 unit-vector normalization, batch/layer normalization for neural-network activations, and whitening.

2.1.5 Encoding

Encoding Converts categorical values into numerical representations that a machine-learning model can process.

Label encoding

Assign each category an integer, e.g.

$$\text{Red} \to 0, \qquad \text{Green} \to 1, \qquad \text{Blue} \to 2.$$

The danger is that this may create an **artificial ordinal relationship**: the model can interpret $2 > 1 > 0$ as meaningful even when the categories have no natural order.

One-hot encoding

Create one binary column for each category:

$$\text{Red} \to [1,0,0], \qquad \text{Green} \to [0,1,0], \qquad \text{Blue} \to [0,0,1].$$

This works well for low-cardinality nominal categories. The main limitation is **dimensionality**: thousands of categories create thousands of sparse columns.

Added intuition — why one-hot is preferred in linear-algebra terms

For k unordered categories, one-hot encoding represents the categories as the **standard basis vectors** of $\mathbb{R}^k$:

$$\text{Red} = e_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \qquad \text{Green} = e_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \qquad \text{Blue} = e_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Different basis vectors are orthogonal,

$$e_i^T e_j = 0 \qquad (i \neq j),$$

and every pair is equally separated,

$$\|e_i - e_j\|_2 = \sqrt{2} \qquad (i \neq j).$$

So the representation introduces **no fake ordering or magnitude**. It also gives a linear model one independent coefficient per category:

$$\boxed{\mathbf{w}^T e_j = w_j}$$

The active one-hot coordinate simply selects the parameter belonging to that category.

Compare label encoding: $|0-2| = 2$ but $|0-1| = 1$, so Red looks "twice as far" from Blue as from Green, which means nothing.

Frequency encoding

Replace a category by how often it appears in the dataset. This can reduce dimensionality for high-cardinality features, but frequency itself does not necessarily imply a useful relationship with the target.

Binary encoding

Assign categories integers, convert those integers to binary, then expand the binary digits into columns. It can use fewer columns than one-hot encoding when the number of categories is large, but it is less interpretable.

Column count

For k categories: one-hot needs k columns, binary needs about $\lceil \log_2 k \rceil$.
E.g. $k = 1000$: one-hot $\rightarrow$ 1000 columns, binary $\rightarrow$ 10 columns ($2^{10} = 1024$).

2.1.6 Discretization

Discretization Converts a continuous variable into a finite set of intervals or bins. It trades numerical precision for a simpler, often more robust or interpretable representation.

Equal-width binning

Divide the numerical range into k intervals of identical width:

$$\boxed{\text{width} = \frac{x_{\max} - x_{\min}}{k}}$$

Every interval spans the same numerical distance, but the **number of observations in each bin can be very different**.

Equal-frequency (quantile) binning

Sort the values and choose boundaries so that each bin contains approximately the same number of observations. Thus

similar counts per bin, different numerical widths.



K-means binning

Use K-means to partition the values into k clusters by minimizing within-cluster variation. The bin boundaries therefore **adapt to the empirical data distribution**.

Method	What is controlled?	Main trade-off
Equal-width	Same interval width	Simple, but counts can be very uneven.
Equal-frequency	Roughly same count per bin	Balanced counts, but widths vary.
K-means	Data-driven cluster compactness	Adaptive, but less transparent.

2.1.7 Dimensionality Reduction

Dimensionality reduction Maps high-dimensional data into a lower-dimensional space while trying to preserve useful structure.

Main motivations are lower computational/storage cost, noise reduction, visualization in 2D/3D, and improved generalization by removing redundant variation.

The lecture mentions PCA, LDA, t-SNE, UMAP, and random projection, but develops **PCA** as the principal technique in this section.

2.1.8 Principal Component Analysis (PCA)

PCA Finds a new set of **orthogonal axes**, called **principal components**, such that the projected data retains as much variance as possible.

Given

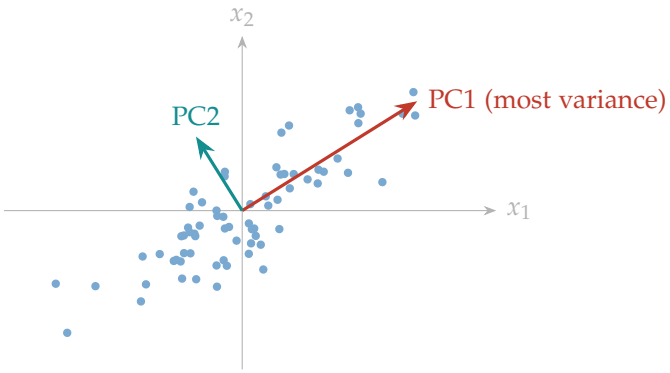
$$X \in \mathbb{R}^{n \times d}$$

with n samples and d features, PCA finds $k < d$ orthogonal directions

$$W_k \in \mathbb{R}^{d \times k}$$

and produces

$$\boxed{Y = X_c W_k \in \mathbb{R}^{n \times k}}$$



Geometric picture

PCA **rotates the coordinate system** so that PC1 points along the direction where the data cloud varies most. PC2 is orthogonal to PC1 and captures the next-largest remaining variance, and so on.

Step 1 — center the data

Subtract the mean of each feature:

$$\boxed{X_c = X - \boldsymbol{\mu}}$$

PCA cares about variation around the mean, so centering moves the data cloud to the origin without changing its shape.

Step 2 — build the covariance matrix

$$\boxed{\Sigma = \frac{1}{n-1} X_c^T X_c \in \mathbb{R}^{d \times d}}$$

For two features,

$$\Sigma = \begin{bmatrix} \mathrm{Var}(x_1) & \mathrm{Cov}(x_1, x_2) \\ \mathrm{Cov}(x_2, x_1) & \mathrm{Var}(x_2) \end{bmatrix}.$$

The **diagonal** tells us how much each feature varies; the **off-diagonal** terms tell us how features move together (§1.2.7). Thus Σ summarizes the orientation and spread of the data cloud. It is always symmetric, since $\mathrm{Cov}(x_1, x_2) = \mathrm{Cov}(x_2, x_1)$.

Added intuition — why $X_c^T X_c$?

Multiplying X_c^T by X_c compares feature columns with one another. Diagonal terms contain sums of squares such as $\sum_i x_{i1}^2$; off-diagonal terms contain cross-products such as $\sum_i x_{i1} x_{i2}$. Because the features are centered, dividing these quantities by $n-1$ gives variances and covariances.

Step 3 — eigenvalue decomposition

Solve

$$\boxed{\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i}$$

Each eigenvector $\mathbf{v}_i$ gives a principal-component **direction**; its eigenvalue λ_i gives the **variance along that direction** (§1.1.5). Therefore

$$\boxed{\text{eigenvector} = \text{direction}, \qquad \text{eigenvalue} = \text{variance captured}.}$$

Sort the eigenvalues so that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$.

Step 4 — keep the top k directions and project

Form

$$W_k = [\, \mathbf{v}_1 \; \mathbf{v}_2 \; \cdots \; \mathbf{v}_k \,]$$

and compute

$$\boxed{Y = X_c W_k}$$

With samples stored as rows, the ith sample transforms as

$$\boxed{\mathbf{y}_i = \mathbf{x}_i W_k}$$

Each entry of $\mathbf{y}_i$ is the component of the original centered sample along one PCA axis.

> **Added intuition — resolving a force vector**
> In mechanics, a force is resolved into components along chosen axes, e.g. horizontal and vertical. PCA does the same geometric operation: it resolves each data vector into components along PC1, PC2, ... The difference is that PCA **learns the axes from the data** by choosing directions of maximum variance.

Why this becomes dimensionality reduction

If all d components are kept, PCA is mainly a **change of coordinate system**. If only the strongest $k < d$ components are retained, low-variance directions are discarded:

$$\boxed{d \text{ original features} \longrightarrow k \text{ PCA coordinates.}}$$

This is closely related to **diagonalization** (§1.1.6): PCA uses the eigenvectors of the covariance matrix as the natural coordinate axes. In that basis,

$$W^T \Sigma W = \Lambda,$$

where Λ is diagonal. A diagonal covariance means the new PCA features are uncorrelated with each other.

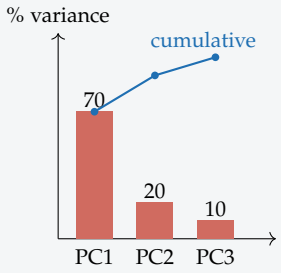
Explained variance

The fraction of total variance explained by component i is

$$\boxed{\text{EVR}_i = \frac{\lambda_i}{\sum_j \lambda_j}}$$

Example

If eigenvalues are $70, 20, 10$, then PC1 explains 70%, PC2 explains 20%, and PC3 explains 10%.
Keeping PC1 and PC2 preserves **90%** of the total variance.



Choosing k

The lecture gives three common approaches:

- **Elbow method:** choose the point after which extra components add little variance.
- **Variance threshold:** keep enough components to capture a target such as 95% cumulative variance.
- **Cross-validation:** choose the k that performs best on the downstream task.

PCA via SVD

Instead of explicitly forming the covariance matrix, PCA can be computed from

$$\boxed{X_c = USV^T}$$

where the columns of V are the principal-component directions. This directly connects PCA to the SVD studied in Chapter 1 (§1.1.8).

Added note

Why it works:

$$X_c^T X_c = (USV^T)^T(USV^T) = VS^TU^TUSV^T = VS^2V^T \qquad (\text{since } U^TU = I)$$

$$\Rightarrow \quad \Sigma = V\,\frac{S^2}{n-1}\,V^T$$

which is exactly an eigendecomposition of Σ: the columns of V are its eigenvectors, and $\lambda_i = \sigma_i^2/(n-1)$.

Scale before PCA

PCA is variance-based, so a feature measured on a much larger numerical scale can dominate the components. It is therefore standard to **standardize features before PCA** when their scales differ substantially (§2.1.4).

Limitations

PCA captures **linear** structure, can be sensitive to outliers, depends on scaling, and often reduces interpretability because each principal component is a linear combination of the original features.

2.1.9 Feature Selection

Feature selection Keeps the most useful subset of the **original** features. Unlike PCA, it does not create new synthetic features.

For example,

$$[x_1, x_2, x_3, x_4, x_5] \longrightarrow [x_1, x_3, x_5].$$

The goals are improved generalization, lower training/inference cost, and better interpretability.

	PCA	Feature selection
Output	new features (linear combinations)	a subset of the original features
Interpretability	lower	kept
Uses the target?	no	filter: sometimes; wrapper: yes

Filter methods

Filter methods score features using statistical criteria **independently of a predictive model**.

Variance threshold.

$$\mathrm{Var}(X_j) = \frac{1}{n}\sum_{i=1}^{n}(X_{ij} - \mu_j)^2.$$

Very low variance suggests the feature changes little across samples. Limitation: variance alone ignores the target.

Correlation with the target.

$$\rho(X_j, y) = \frac{\mathrm{Cov}(X_j, y)}{\sigma_{X_j}\sigma_y}.$$

Large $|\rho|$ indicates strong **linear** association, but nonlinear dependence can be missed (§1.2.7).

Mutual information.

$$I(X_j; y) = \sum_{x,y} p(x,y)\log\frac{p(x,y)}{p(x)\,p(y)}.$$

High mutual information means knowing X_j strongly reduces uncertainty about the target; it can capture **nonlinear as well as linear** dependence.

If X_j and y are independent, then $p(x,y) = p(x)p(y)$ in every cell (§1.2.5), so every log term is $\log 1 = 0$ and $I = 0$. Mutual information measures how far the joint is from that independent case.

Filter-method trade-off

They are fast, scalable, interpretable, and model-independent, but they can miss interactions between features and may retain redundant features.

Wrapper methods

Wrapper methods evaluate feature subsets by **repeatedly training and validating a predictive model**. They can capture feature interactions, but they are computationally expensive.

- **Recursive Feature Elimination (RFE):** train on all features → rank importance → remove the least important → repeat.
- **Forward selection:** start with no features and repeatedly add the feature that improves validation performance the most.
- **Backward elimination:** start with all features and repeatedly remove the least useful feature.

The lecture groups feature-selection methods into **filter, wrapper, and embedded** approaches, but develops filter and wrapper methods in detail here.

2.1.10 Quick Reference

Concept	Formula / definition	Meaning
Aggregation	$\bar{x} = \frac{1}{n}\sum_i x_i$	Combine observations into a simpler summary.
Rolling mean	$\bar{x}_t = \frac{1}{w}\sum_{i=0}^{w-1} x_{t-i}$	Smooth a time series using recent history.
Sampling	Select a subset of observations	Reduce cost or preserve/adjust population structure.
Min–Max	$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$	Map to a fixed range, usually $[0, 1]$.
Z-score	$x' = \frac{x - \mu}{\sigma}$	Center at 0 and scale by standard deviation.
Fit scaler	Fit on train, reuse on val/test	Keep preprocessing based only on training information.
One-hot	category $j \to e_j$	Separate unordered categories without fake magnitude/order.
Discretization	continuous $\to$ finite bins	Trade precision for simpler categories.
Equal-width	width $= \frac{x_{\max} - x_{\min}}{k}$	Equal numerical intervals.
Equal-frequency	approx. equal counts per bin	Variable widths, balanced counts.
PCA covariance	$\Sigma = \frac{1}{n-1} X_c^T X_c$	Summarize variance/covariance structure.
PCA eigenproblem	$\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i$	$\mathbf{v}_i$ = direction, λ_i = captured variance.
PCA projection	$Y = X_c W_k$	Resolve samples onto the top k principal axes.
Explained variance	$\lambda_i / \sum_j \lambda_j$	Fraction of total variance captured by PC i.
PCA via SVD	$X_c = USV^T$	Columns of V = principal directions.
Feature selection	keep original feature subset	Reduce dimensions without creating new features.
Correlation filter	$\rho(X_j, y)$	Rank linear association with target.
Mutual information	$I(X_j; y)$	Rank general statistical dependence with target.
Wrapper method	train/validate feature subsets	Captures interactions but costs more computation.

Big picture Preprocessing is not just "cleaning." It changes the **representation** on which the learning algorithm operates. The core question throughout this part is: *what information should we preserve, suppress, rescale, or re-express before asking a model to learn?*

Source note. The core content and ordering in these notes follow the MLR570 Chapter 2 revised lecture notes. Amber boxes mark added intuitive explanations developed during study, so they can be distinguished from the lecture-source material.

PART 2.2

Similarity Measures

The flow: introduction → Pearson correlation → Euclidean distance → Manhattan distance → curse of dimensionality → cosine similarity → Jaccard similarity → comparative summary.

Similarity and distance measures give us a numerical way to describe how alike or different two observations are. The key idea is that "similar" does not always mean the same thing: sometimes we care about **linear co-movement**, sometimes **geometric distance**, sometimes **direction**, and sometimes **set overlap**.

Why this matters in ML

Many algorithms make decisions by comparing observations. k-NN and K-means rely on distance, feature analysis often uses correlation, and text or embedding systems commonly use cosine or Jaccard similarity. The measure we choose determines the **geometry the algorithm "sees."**

2.2.1 Pearson Correlation

Pearson correlation Measures the strength and direction of the **linear** relationship between two variables (§1.2.7):

$$\rho(A, B) = \frac{\mathrm{Cov}(A, B)}{\sigma_A \, \sigma_B}$$

Its value lies between -1 and 1:

$\rho = 1$	perfect positive linear relationship
$\rho = 0$	no linear relationship detected
$\rho = -1$	perfect negative linear relationship

Linear example

Let $A = [1, 2, 3, 4, 5]$ and $B = [2, 4, 6, 8, 10]$. Since $B = 2A$, the variables lie exactly on a straight line and $\rho = 1$.

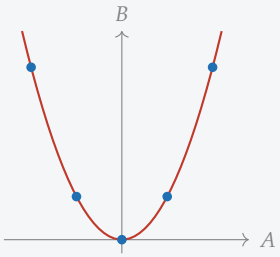
Nonlinear example

Let $B = A^2$ with values of A symmetric around zero. B is **completely determined** by A, but Pearson correlation can still be 0 because the increasing and decreasing linear trends cancel.

E.g. $A = [-2, -1, 0, 1, 2]$, $B = [4, 1, 0, 1, 4]$. Here $E[A] = 0$, so

$$\text{Cov}(A, B) = E[AB] - E[A]E[B] = \frac{-8-1+0+1+8}{5} - 0 = 0$$

$$\Rightarrow \ \rho = 0.$$



Added clarification

Important: $\rho = 0$ does **not** mean "no relationship." It means Pearson detected no *linear* relationship. Nonlinear dependence may still be very strong.

Applications: feature selection, detecting strongly correlated variables, and analyzing linear relationships in regression.
Limitation: Pearson misses nonlinear relationships and is sensitive to outliers.

2.2.2 Euclidean and Manhattan Distance

Both are **distance** measures: smaller distance means the points are closer. The difference is the geometry used to measure that closeness.

Euclidean distance

Euclidean distance The ordinary straight-line distance between two points. It is the L_2 distance from Chapter 1 (§1.1.7).

$$d(A, B) = \sqrt{\sum_{i=1}^{n} (A_i - B_i)^2}$$

Numerical example

$A = [1, 2, 3]$, $B = [4, 5, 6]$. Then

$$d(A, B) = \sqrt{(1-4)^2 + (2-5)^2 + (3-6)^2} = \sqrt{27} \approx 5.196.$$

Typical applications include k-NN classification and K-means clustering. Its main weaknesses are **sensitivity to feature scaling** (§2.1.4) and reduced discriminative power in very high dimensions.

Manhattan distance

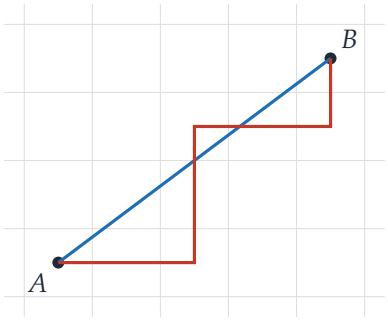
Manhattan distance The sum of the absolute coordinate-wise differences. It is the L_1 distance from Chapter 1 (§1.1.7).

$$d(A, B) = \sum_{i=1}^{n} |A_i - B_i|$$

Same points

$A = [1, 2, 3]$, $B = [4, 5, 6]$. Then

$$d(A, B) = |1 - 4| + |2 - 5| + |3 - 6| = 9.$$



Euclidean: straight line

Manhattan: along the axes

(every staircase path from A to B has the same Manhattan length)

Geometry

Euclidean = straight-line path. Manhattan = move only along coordinate axes, like navigating a city grid.

Intuition

Because Euclidean distance **squares** each coordinate difference, one unusually large difference can dominate strongly. Manhattan grows only **linearly** with that difference, so it is often more robust to outliers.

E.g. differences $[1, 1, 10]$: Euclidean $= \sqrt{1 + 1 + 100} \approx 10.1$ (almost all from the 10), Manhattan $= 1 + 1 + 10 = 12$.

2.2.3 The Curse of Dimensionality

Curse of dimensionality As the number of dimensions grows, distance-based comparisons can become **less informative**. Euclidean distances between different points tend to become more similar to one another, making "nearest" and "farther" harder to distinguish.

In low dimensions, one neighbor can clearly be much closer than another. In very high dimensions, every pair differs across many coordinates, so distances accumulate many contributions.

Added intuition

Mental picture: as dimensionality grows, the **gap between the nearest and farthest points can shrink relative to their overall distance**. This is why methods such as k-NN and clustering

can struggle in high-dimensional spaces.

$$\frac{d_{\max} - d_{\min}}{d_{\min}} \longrightarrow 0 \qquad \text{as } d \text{ grows}$$

If every point is roughly the same distance away, "nearest neighbor" stops meaning much.

The lecture notes emphasize that Manhattan distance can sometimes be more stable than Euclidean distance in high-dimensional settings because it treats coordinate differences linearly rather than squaring them.

What we do about it

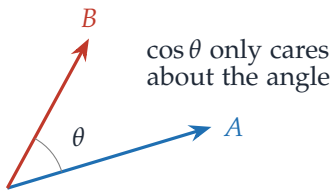
This is one more reason for dimensionality reduction and feature selection (§2.1.7–2.1.9): fewer, more informative dimensions keep distances meaningful.

2.2.4 Cosine Similarity

Cosine similarity Measures the cosine of the **angle** between two vectors. It asks whether they point in the same direction rather than how far apart their endpoints are.

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \, \|B\|}$$

$\cos = 1$	same direction
$\cos = 0$	perpendicular directions
$\cos = -1$	opposite directions



Perfect alignment

$A = [1, 2, 3]$ and $C = [2, 4, 6] = 2A$. C is twice as large, but it points in exactly the same direction:

$$\cos(A, C) = \frac{2 + 8 + 18}{\sqrt{14} \cdot \sqrt{56}} = \frac{28}{\sqrt{14} \cdot 2\sqrt{14}} = \frac{28}{28} = 1.$$

Key distinction

Euclidean asks: "How far apart are these vectors?"
Cosine asks: "How aligned are these vectors?"
Two vectors can be far apart in magnitude and still have cosine similarity 1.

Applications: document similarity, recommender systems, and high-dimensional embeddings such as word or image embeddings. Cosine is useful when direction/pattern matters more than magnitude.

Link to Pearson: if you first center both vectors (subtract their means), cosine similarity becomes exactly Pearson correlation. So Pearson is cosine on mean-centered data.

2.2.5 Jaccard Similarity

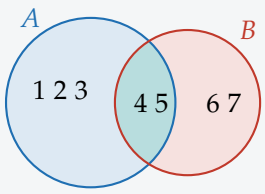
Jaccard similarity Measures how much two **sets** overlap.

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Set example

$A = \{1,2,3,4,5\}$, $B = \{4,5,6,7\}$.
Intersection $= \{4,5\}$ has size 2; union $= \{1,2,3,4,5,6,7\}$ has size 7. Therefore

$$J(A,B) = \frac{2}{7} \approx 0.286.$$



Binary-vector example

$A = [1,0,1,0,1]$, $B = [1,1,0,0,1]$. Treat the 1-entries as set membership. Two positions are shared (positions 1 and 5) and four positions occur in the union (1, 2, 3, 5), so

$$J = \frac{2}{4} = 0.5.$$

Position 4 is 0 in both, so it counts in neither: Jaccard ignores shared absences.

Interpretation: $J = 1$ means identical sets; $J = 0$ means no overlap. Jaccard is intuitive for documents, categorical data, and binary feature vectors, but it **ignores frequency and magnitude**.

2.2.6 Comparative Summary

The most important idea is that each measure answers a **different question**. "Similarity" only makes sense after deciding what aspect of the data should count as similar.

Measure	What it asks	Best suited for	Main limitation
Pearson	Do the variables move together linearly?	feature relationships, regression	misses nonlinear dependence; outlier-sensitive
Euclidean	How far apart in a straight line?	low-dimensional numeric data, k-NN, K-means	scale-sensitive; weaker in high dimensions
Manhattan	How far apart axis-by-axis?	grid-like or high-dimensional data	less intuitive geometrically
Cosine	Do the vectors point in the same direction?	text, embeddings, recommender systems	ignores magnitude differences
Jaccard	How much set membership overlaps?	sets, binary/categorical data	ignores frequency/magnitude

Mental map

Relationship between variables	→	**Pearson**
Numerical/geometric distance	→	**Euclidean** or **Manhattan**
Direction or pattern, not size	→	**Cosine**
Shared membership or overlap	→	**Jaccard**

Why the choice matters

$A = [1, 2, 3]$ and $B = [2, 4, 6]$ have cosine similarity 1 because they point in the same direction, yet their Euclidean distance is $\sqrt{1+4+9} = \sqrt{14} \approx 3.74$, not zero, because their magnitudes differ. The same pair can therefore be **similar under one definition and separated under another**.

2.2.7 Quick Reference

Concept	Formula / definition	Meaning
Pearson correlation	$\rho = \mathrm{Cov}(A, B)/(\sigma_A \sigma_B)$	linear association; -1 to 1
$\rho = 0$	no linear correlation	does not rule out nonlinear dependence
Euclidean	$\sqrt{\sum (A_i - B_i)^2}$	straight-line / L_2 distance
Manhattan	$\sum \lvert A_i - B_i \rvert$	axis-wise / L_1 distance
Curse of dimensionality	distance contrast decreases as d grows	nearest/farthest become less distinguishable
Cosine	$\dfrac{A \cdot B}{\lVert A \rVert \, \lVert B \rVert}$	orientation; ignores overall scale
Jaccard	$\lvert A \cap B \rvert / \lvert A \cup B \rvert$	set/binary overlap

Source note. The core content and ordering in these notes follow the MLR570 Chapter 2 revised lecture notes. Amber boxes mark intuitive clarifications developed during study so they remain distinguishable from the lecture-source material.

PART 2.3

Model Evaluation

The flow: training / validation / testing → underfitting and overfitting → K-fold cross-validation → confusion matrix → accuracy, precision, recall, F1 → decision threshold → weighted F-beta → regression metrics (MSE and MAE) → quick reference.

Model evaluation asks whether a trained model has learned a pattern that **generalizes** beyond the data it saw during training. Training performance alone is not enough; we need held-out data and metrics that match the task.

Why this matters in ML

A model can achieve extremely low training loss and still perform badly on new data. Evaluation is therefore not just "calculate one number"; it is a **workflow** for separating learning, tuning, and final judgment.

2.3.1 Training, Validation, and Testing

Training set	Used to **fit model parameters** by minimizing training error. The model directly learns from these examples.
Validation set	Used during development for **model selection, hyperparameter tuning**, and monitoring generalization.
Test set	Used **only after** training / validation to estimate final performance on unseen data.

Lecture example: 1000 samples

A possible split is 70% training (700 samples), 15% validation (150), and 15% testing (150). The exact ratio is task-dependent; the conceptual separation of roles is what matters.



Memory aid **Train** = learn. **Validation** = tune / choose. **Test** = final judge.

Added analogy

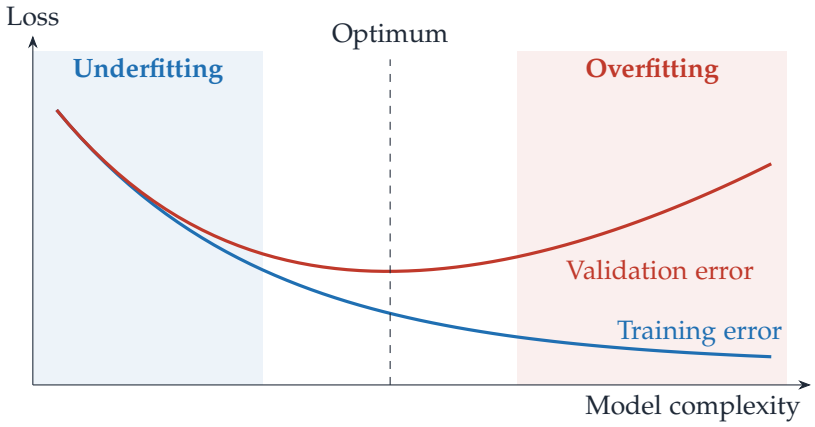
Think of training as **practice questions**, validation as a **mock exam** used to adjust your strategy, and the test set as the **final exam**. Looking at the final exam while preparing would destroy its role as an unbiased final check.

This is the same rule as fitting a scaler on training data only (§2.1.4): nothing about the test set may leak into decisions made during development.

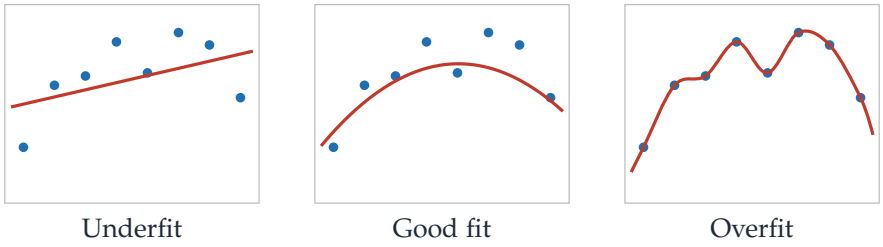
2.3.2 Underfitting and Overfitting

Underfitting Model is **too simple** (or insufficiently trained) to capture the structure. Training error is **high** and validation / test error is also **high**.

Overfitting Model fits the training data **too specifically**, including noise. Training error is **low**, but validation / test error is **much higher**.



Lecture Figure 3.1 concept: training error tends to decrease as complexity increases, while validation error eventually rises again. The useful operating point is near the **minimum validation error**.



Lecture Figure 3.2 concept: too-simple fit, useful fit, and overly flexible fit. The same idea applies to regression curves and classification decision boundaries.

Added connection

Bias–variance connection: underfitting is associated with **high bias** (too rigid); overfitting is associated with **high variance** (too sensitive to the particular training sample). Same trade-off as the rolling-window size in §2.1.2.

Lecture example: choosing a model

Model	Training error	Validation error	Interpretation
1	12%	11%	best validation result
2	17%	18%	worse than Model 1
3	**6%**	19%	very low train error, but overfits
4	23%	23.5%	underfits / weak overall
5	11%	17%	some overfitting

The lecture example deliberately shows why **minimum training error is not the selection criterion**. Model 3 has the lowest training error, but Model 1 has the lowest validation error and is the better choice.

Quick read: a big gap between train and validation → overfitting (Models 3, 5). Both high → underfitting (Model 4).

2.3.3 K-Fold Cross-Validation

K-fold cross-validation Splits the data into K subsets (folds), trains the model K times, and **rotates** which fold is held out for validation. Final performance is the **average** across folds.

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
Run 1	VAL	Train	Train	Train	Train	$\rightarrow \text{Error}_1$
Run 2	Train	VAL	Train	Train	Train	$\rightarrow \text{Error}_2$
Run 3	Train	Train	VAL	Train	Train	$\rightarrow \text{Error}_3$
Run 4	Train	Train	Train	VAL	Train	$\rightarrow \text{Error}_4$
Run 5	Train	Train	Train	Train	VAL	$\rightarrow \text{Error}_5$

Simplified reconstruction of the lecture $K = 5$ diagram: each fold is the validation fold once, while the other four folds are used for training.

Average validation error:

$$\boxed{\text{CV error} = \frac{1}{K}\sum_{i=1}^{K}\text{Error}_i}$$

Why use it?

It reduces dependence on one lucky or unlucky split, uses limited data efficiently (every sample is used for validation exactly once), and gives a more stable estimate of validation performance.

Added clarification

Larger K means more training runs and therefore more computation. Each held-out fold is also smaller. The point is not that more folds are always "better"; it is another **reliability-versus-cost trade-off**.

For classification with imbalanced classes, folds are usually made **stratified** so each fold keeps the same class proportions (§2.1.3).

2.3.4 Classification Metrics

All of the basic binary classification metrics start from the **confusion matrix**.

	Predicted +	**Predicted −**
Actual +	**TP** correct positive	**FN** missed positive
Actual −	**FP** false alarm	**TN** correct negative

Lecture Figure 3.4 concept. TP/TN lie on the correct diagonal; FP and FN are the two different kinds of error.

Metric	Formula	Question it answers
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Out of everyone, how many predictions were correct?
Precision	$\frac{TP}{TP+FP}$	When I predict positive, how often am I right?
Recall	$\frac{TP}{TP+FN}$	Of all real positives, how many did I find?
F1	$\frac{2PR}{P+R}$	Are precision and recall both reasonably good?

Memory **Precision:** denominator = everything **predicted** positive ($TP+FP$) → a column of the matrix.
Recall: denominator = everything **actually** positive ($TP+FN$) → a row of the matrix.

Added clarification

Accuracy warning: on an imbalanced dataset, predicting only the majority class can produce high accuracy while completely failing on the minority class.

E.g. 95 negatives, 5 positives, model always says "negative": accuracy $= 95/100 = 95\%$, but recall $= 0/5 = 0$.

Precision and recall are conditional probabilities (§1.2.4): precision $= P(\text{actual}+ \mid \text{predicted}+)$, recall $= P(\text{predicted}+ \mid \text{actual}+)$. Same two events, conditioned in opposite directions.

2.3.5 F1 and the Harmonic Mean

F1 F1 is the **harmonic mean** of precision and recall, so a high value requires **both** metrics to be reasonably high.

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why harmonic mean?

If Precision $= 0.1$ and Recall $= 0.9$, the arithmetic mean is 0.5, which looks deceptively moderate. The harmonic mean gives

$$F_1 = \frac{2(0.1)(0.9)}{0.1 + 0.9} = 0.18,$$

strongly exposing the weak precision. The lecture also notes

$$\text{Harmonic Mean} \leq \text{Geometric Mean} \leq \text{Arithmetic Mean},$$

with equality only when the two inputs are equal. (Here: $0.18 \leq \sqrt{0.09} = 0.3 \leq 0.5$.)

Lecture examples

Case	TP	FN	FP	TN	Precision	Recall	F1
Balanced	50	10	10	50	0.833	0.833	0.833
High P, low R	10	40	2	48	0.833	0.200	0.320
High R, low P	45	5	40	10	0.529	0.900	0.666

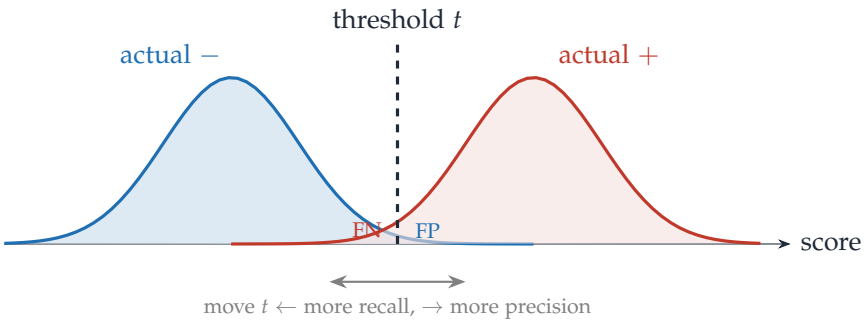
These lecture examples show why looking at precision or recall alone can be misleading. F1 drops when one side is much weaker than the other.

2.3.6 The Decision Threshold

Added note
The course outline includes the decision threshold, but the revised lecture PDF does not contain a dedicated threshold subsection. This explanation is therefore an added clarification.

A probabilistic binary classifier often outputs a score such as $P(y = 1 \mid \mathbf{x}) = 0.73$. A threshold t converts that score into a class:

$$\text{score} \geq t \;\rightarrow\; \text{predict positive} \qquad \text{score} < t \;\rightarrow\; \text{predict negative}$$



Change	What happens	Typical effect
Lower threshold	more samples called positive	Recall tends to rise; Precision tends to fall
Higher threshold	fewer samples called positive	Precision tends to rise; Recall tends to fall

Key idea Changing the threshold changes TP, FP, TN, and FN, so it changes accuracy, precision, recall, and F1 **without retraining the model**.

2.3.7 Weighted F-beta

F-beta Generalizes F1 so that precision and recall do not have to receive equal emphasis.

$$F_\beta = \frac{(1+\beta^2)(\text{Precision} \cdot \text{Recall})}{\beta^2 \,\text{Precision} + \text{Recall}}$$

- $\beta = 1$ gives F1.
- $\beta > 1$ emphasizes **recall** (e.g. disease screening: missing a positive is worse).
- $\beta < 1$ emphasizes **precision** (e.g. spam filter: a false alarm is worse).

Lecture example

$TP = 45$, $FN = 40$, $FP = 5$, $TN = 10$ gives

$$\text{Precision} = \frac{45}{50} = 0.9, \qquad \text{Recall} = \frac{45}{85} = 0.529, \qquad F_1 = 0.666.$$

With $\beta = 2$:

$$F_2 = \frac{5(0.9)(0.529)}{4(0.9) + 0.529} = \frac{2.38}{4.13} = 0.577.$$

Because recall is the weaker metric, emphasizing recall pulls the score downward.

2.3.8 Regression Metrics: MSE and MAE

For regression we compare a continuous prediction $\hat{y}_i$ with its true value y_i. The lecture focuses on MSE and MAE.

Mean Squared Error (MSE)

MSE

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

MSE **squares** errors. Large mistakes therefore receive disproportionately large penalties. It is smooth and convenient for gradient-based optimization, but highly sensitive to outliers.

Mean Absolute Error (MAE)

MAE

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

MAE grows **linearly** with error. It is more robust to outliers and remains in the **original units** of the target, making it easier to interpret.

Metric	Penalty	Outliers	Units
MSE	squared	very sensitive	squared target units
MAE	absolute / linear	more robust	same as target

This is the L_2 vs L_1 story again (§1.1.7, §2.2.2): squaring lets one big error dominate.

Lecture example 1

$y = [3,\ 0.5,\ 2,\ 7]$, $\hat{y} = [2.5,\ 0,\ 2,\ 8]$. Errors: $[0.5,\ 0.5,\ 0,\ -1]$.

$$\text{MSE} = \frac{0.25 + 0.25 + 0 + 1}{4} = 0.375 \qquad \text{MAE} = \frac{0.5 + 0.5 + 0 + 1}{4} = 0.5$$

They should not be compared directly because MSE is in squared units. RMSE $= \sqrt{\text{MSE}} \approx 0.61$ is in the original units.

Lecture example 2: add one outlier

$y = [3,\ 0.5,\ 2,\ 7,\ 100]$, $\hat{y} = [2.5,\ 0,\ 2,\ 8,\ 8]$. The final error is 92.

$$\text{MSE} = \frac{1.5 + 92^2}{5} = \frac{1.5 + 8464}{5} = 1693.1 \qquad \text{MAE} = \frac{2 + 92}{5} = 18.8$$

Squaring 92 produces 8464, which explains why MSE reacts so dramatically.

Which one to use

Use MSE when large mistakes deserve especially strong punishment or smooth gradient-based optimization is important.
Use MAE when robustness to extreme errors and interpretability in the original units matter more.

Same idea as mean vs median (§2.1.2): if you predict one constant for all points, MSE is minimized by the **mean** and MAE by the **median**, which is why MAE shrugs off outliers.

2.3.9 Quick Reference

Concept	Formula / rule	Meaning
Training	fit model parameters	data the model learns from
Validation	tune / select	development-time generalization check
Test	final evaluation	unseen final benchmark
Underfit	train error high; val error high	model too simple / insufficiently trained
Overfit	train error low; val error high	fits training data too specifically
K-fold CV	rotate held-out fold; average	more stable validation estimate
Accuracy	$(TP + TN)/\text{all}$	overall fraction correct
Precision	$TP/(TP + FP)$	correctness of positive predictions
Recall	$TP/(TP + FN)$	coverage of actual positives
F1	$2PR/(P + R)$	harmonic balance of P and R
Threshold	score $\geq t \to +$	trades precision vs recall, no retraining
F-beta	$\beta > 1$ recall; $\beta < 1$ precision	weighted P/R balance
MSE	$\frac{1}{N}\sum(y_i - \hat{y}_i)^2$	strongly punishes large errors
MAE	$\frac{1}{N}\sum\lvert y_i - \hat{y}_i\rvert$	robust, original units

Source note. The core content, examples, figures, and ordering in these notes follow the MLR570 Chapter 2 revised lecture notes. Amber boxes mark study clarifications or analogies that were added during our walkthrough so they remain distinguishable from source-derived material.

CHAPTER 3

3 · Supervised Learning

The models themselves, taken apart one by one: what each one optimizes, where its formula comes from, and the geometry behind it.

PART 3.1

Hard-Margin SVM

The flow: perceptron limitation → maximum margin → line/hyperplane geometry → point-to-hyperplane distance → margin $2/\|\mathbf{w}\|$ → hard-margin primal → Lagrangian → dual problem → support vectors → worked example.

Added note

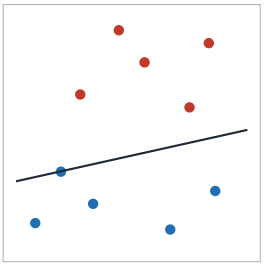
Scope for this lecture. These notes cover the hard-margin SVM material used in class. Soft-margin SVM and the hyperparameter C, as well as KKT conditions, are intentionally omitted.

3.1.1 Why SVM? From "a separator" to the best separator

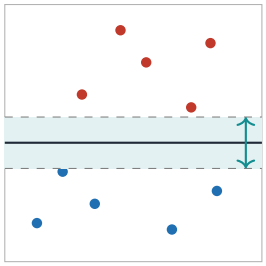
The perceptron can find a separating hyperplane when the data are linearly separable, but it may converge to **any** valid separator. It does not ask whether the boundary passes dangerously close to training points. SVM adds one central idea:

Central idea Among all separating hyperplanes, choose the one with the **largest geometric margin**.

Valid, but small safety gap



Maximum-margin idea



Mental model

Perceptron: "Did I separate the classes?"
SVM: "Can I separate them while leaving the **largest possible safety corridor** between the classes?"

3.1.2 Rebuilding the Line Geometry from First Principles

Before the SVM formulas, we need to understand exactly what

$$\boxed{\mathbf{w}^T\mathbf{x} + b = 0}$$

means geometrically.

Start with an A-level line: $x + y = 4$

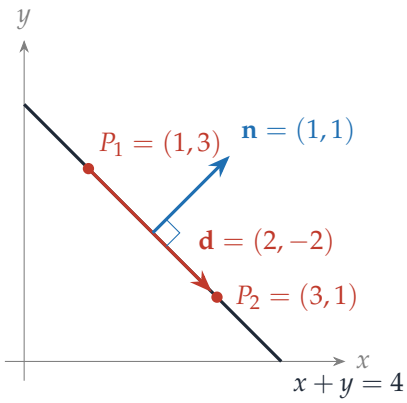
Points such as $(1,3)$ and $(3,1)$ lie on the line because

$$1 + 3 = 4, \qquad 3 + 1 = 4.$$

Subtracting the two points gives the displacement from one point to the other:

$$\underbrace{(3,1)}_{P_2} - \underbrace{(1,3)}_{P_1} = (2,-2).$$

This vector points **along** the line: move 2 units right and 2 units down.



The coefficients of the line are $(1,1)$. Take their dot product with the direction along the line:

$$(1,1) \cdot (2,-2) = 2 - 2 = 0.$$

A zero dot product means perpendicular vectors, so $(1,1)$ is **normal** (perpendicular) to the line.

Why the coefficients of $Ax + By + C = 0$ form the normal vector

Take two arbitrary points $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ on

$$Ax + By + C = 0.$$

They both satisfy the line equation:

$$Ax_1 + By_1 + C = 0, \qquad Ax_2 + By_2 + C = 0.$$

Subtract the equations:

$$A(x_2 - x_1) + B(y_2 - y_1) = 0.$$

But $(x_2 - x_1,\ y_2 - y_1)$ is exactly the movement vector along the line. Therefore

$$(A, B) \cdot (x_2 - x_1,\ y_2 - y_1) = 0.$$

Hence

$$\boxed{(A, B) \perp \text{the line}}$$

ML simply writes the same idea as

$$\mathbf{w}^T\mathbf{x} + b = 0,$$

where $\mathbf{w}$ contains the line/hyperplane coefficients. Therefore

Normal vector $\mathbf{w}$ is **perpendicular** to the decision boundary.

Why $\mathbf{w}^T\mathbf{x} + b$ tells us which side a point is on

For any point $\mathbf{x}$, compute the scalar

$$s(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b.$$

The boundary itself contains all points with $s(\mathbf{x}) = 0$. Moving in the direction of $\mathbf{w}$ **increases** the score because

$$s(\mathbf{x} + \epsilon\mathbf{w}) = \mathbf{w}^T(\mathbf{x} + \epsilon\mathbf{w}) + b = s(\mathbf{x}) + \epsilon\|\mathbf{w}\|^2 > s(\mathbf{x})$$

for $\epsilon > 0$. So $\mathbf{w}$ points toward the positive side.

Signed side-test

$$\begin{aligned} \mathbf{w}^T\mathbf{x} + b > 0 &\Rightarrow \text{positive side} \\ \mathbf{w}^T\mathbf{x} + b < 0 &\Rightarrow \text{negative side} \\ \mathbf{w}^T\mathbf{x} + b = 0 &\Rightarrow \text{on the boundary} \end{aligned}$$

Think of $\mathbf{w}^T\mathbf{x} + b$ as a **signed side-test** along the normal direction.

E.g. for $x + y - 4 = 0$: the point $(3,3)$ gives $3 + 3 - 4 = 2 > 0$ (the side $\mathbf{n}$ points to), and $(0,0)$ gives $-4 < 0$.

3.1.3 Distance from a Point to a Hyperplane

For a point $\mathbf{x}_0$ and the hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$, the perpendicular distance is

Point-to-hyperplane distance

$$\text{dist}(\mathbf{x}_0,\ \mathbf{w}^T\mathbf{x} + b = 0) = \frac{|\mathbf{w}^T\mathbf{x}_0 + b|}{\|\mathbf{w}\|}$$

Connection to the familiar 2D formula

For $Ax + By + C = 0$ and point (x_0, y_0),

$$\frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$$

is exactly the same formula. In ML notation (A, B) becomes $\mathbf{w}$, and $\sqrt{A^2 + B^2}$ is the Pythagorean length $\|\mathbf{w}\|$ (the L_2 norm, §1.1.7).

Example

Distance from $(0,0)$ to $x+y-4=0$:

$$\frac{|0+0-4|}{\sqrt{1^2+1^2}} = \frac{4}{\sqrt{2}} = 2\sqrt{2} \approx 2.83.$$

Why the denominator is necessary

The raw value $\mathbf{w}^T\mathbf{x}_0 + b$ is **not** a true geometric distance because the same line can be written with different scales. For example,

$$x+y-1=0 \qquad \text{and} \qquad 10x+10y-10=0$$

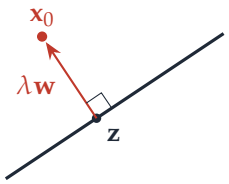
are the same line, but the raw score of every point is multiplied by 10 in the second equation. The norm $\|\mathbf{w}\|$ is also multiplied by 10, so the ratio stays unchanged.

Key idea

The numerator measures separation from the boundary in the normal direction, but it is scaled by the size of $\mathbf{w}$. Dividing by $\|\mathbf{w}\|$ removes that arbitrary scaling and gives **actual geometric distance**.

Short derivation using the normal direction

Let $\mathbf{z}$ be the closest point on the hyperplane to $\mathbf{x}_0$. The shortest route is perpendicular to the hyperplane, so it must be parallel to $\mathbf{w}$:



$$\mathbf{x}_0 - \mathbf{z} = \lambda\mathbf{w}.$$

Because $\mathbf{z}$ lies on the hyperplane,

$$\mathbf{w}^T\mathbf{z} + b = 0.$$

Now

$$\mathbf{w}^T\mathbf{x}_0 + b = \mathbf{w}^T(\mathbf{z}+\lambda\mathbf{w}) + b = \underbrace{(\mathbf{w}^T\mathbf{z}+b)}_{0} + \lambda\mathbf{w}^T\mathbf{w} = \lambda\|\mathbf{w}\|^2.$$

Thus

$$|\lambda| = \frac{|\mathbf{w}^T\mathbf{x}_0 + b|}{\|\mathbf{w}\|^2}.$$

The geometric distance is

$$\|\mathbf{x}_0 - \mathbf{z}\| = \|\lambda\mathbf{w}\| = |\lambda|\,\|\mathbf{w}\| = \boxed{\frac{|\mathbf{w}^T\mathbf{x}_0 + b|}{\|\mathbf{w}\|}}$$

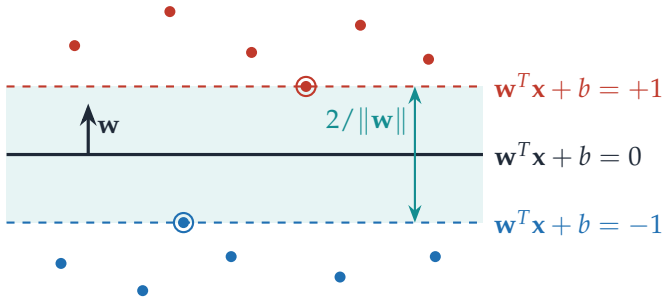
3.1.4 SVM Margin Geometry

SVM chooses a convenient scaling in which the two **margin hyperplanes** are

$$\boxed{\mathbf{w}^T\mathbf{x} + b = +1}, \qquad \boxed{\mathbf{w}^T\mathbf{x} + b = -1},$$

with the **decision boundary** in the middle:

$$\boxed{\mathbf{w}^T\mathbf{x} + b = 0}$$



A point on $\mathbf{w}^T\mathbf{x} + b = +1$ has distance

$$\frac{|1|}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

from the center boundary. Likewise, the -1 margin is $1/\|\mathbf{w}\|$ away. Therefore

Margin width

$$\text{total margin} = \frac{2}{\|\mathbf{w}\|}$$

Why specifically $+1$ and -1?

The classifier only depends on the **sign** of $\mathbf{w}^T\mathbf{x} + b$. Multiplying $\mathbf{w}$ and b by any positive constant does not change the geometric boundary (same as $x + y - 1 = 0$ vs $10x + 10y - 10 = 0$). Therefore the scale of $\mathbf{w}, b$ is arbitrary until we fix it. SVM chooses the convenient normalization in which the **closest** positive and negative points have scores $+1$ and -1.

The number 1 is not magical

We could use another positive constant, but every parameter would simply be rescaled. Choosing ± 1 makes the margin formulas and constraints especially clean.

From maximum margin to the SVM objective

We want to maximize

$$\frac{2}{\|\mathbf{w}\|}.$$

A fraction becomes larger when its positive denominator becomes smaller, so

$$\boxed{\max \frac{2}{\|\mathbf{w}\|} \iff \min \|\mathbf{w}\|}$$

Instead of minimizing $\|\mathbf{w}\|$ directly, SVM uses

$$\boxed{\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2}$$

because it has the same minimizer and a simpler derivative (§1.3.5):

$$\frac{\partial}{\partial \mathbf{w}}\left(\frac{1}{2}\mathbf{w}^T\mathbf{w}\right) = \mathbf{w}.$$

($\|\mathbf{w}\|$ itself has a square root, which is messier to differentiate and not smooth at $\mathbf{w} = \mathbf{0}$. Squaring is monotone for nonnegative numbers, so the minimizer doesn't move.)

3.1.5 Why We Need the Hard-Margin Constraint

If the objective were only

$$\min \frac{1}{2}\|\mathbf{w}\|^2,$$

the smallest vector is trivially $\mathbf{w} = \mathbf{0}$. But then

$$\mathbf{w}^T\mathbf{x} + b = b,$$

so the prediction no longer depends on $\mathbf{x}$ at all. There is no meaningful oriented decision boundary.

SVM therefore says: make $\|\mathbf{w}\|$ as small as possible *while still correctly separating every training point outside the margin.*

For a positive point ($y_i = +1$), require

$$\mathbf{w}^T\mathbf{x}_i + b \geq 1.$$

For a negative point ($y_i = -1$), require

$$\mathbf{w}^T\mathbf{x}_i + b \leq -1.$$

Both cases combine into one compact inequality:

Hard-margin constraint

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$$

Check the two cases

$$y_i = +1: \quad (+1)(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \;\Rightarrow\; \mathbf{w}^T\mathbf{x}_i + b \geq 1.$$

$$y_i = -1: \quad (-1)(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \;\Rightarrow\; \mathbf{w}^T\mathbf{x}_i + b \leq -1.$$

The same expression therefore places both classes on the correct side of their corresponding margin boundary.

Why $\mathbf{w} = 0$ cannot survive the constraints

If $\mathbf{w} = \mathbf{0}$, then every score is just b. A positive sample would require $b \geq 1$, while a negative sample would require $b \leq -1$. With both classes present, both conditions cannot hold at once.

3.1.6 Hard-Margin SVM Primal Problem

Putting the objective and constraints together gives the **primal** optimization problem:

Primal

$$\begin{aligned} \min_{\mathbf{w},b} \quad & \frac{1}{2}\|\mathbf{w}\|^2 \\ \text{subject to} \quad & y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \qquad i = 1, \ldots, n. \end{aligned}$$

Interpretation

The **objective** wants the widest possible margin. The **constraints** prevent the optimizer from achieving that by destroying the classifier; every training point must remain correctly separated and outside the margin.

This is a convex problem (§1.3.6): a quadratic bowl in $\mathbf{w}$ with linear constraints, so there is one global optimum and no bad local minima.

3.1.7 Lagrange Multipliers: Putting the Constraints into One Function

The primal is constrained. A **Lagrange multiplier** gives each constraint a nonnegative weight $\alpha_i \geq 0$ and combines objective and constraints into a single **Lagrangian**.

Rewrite each constraint as

$$g_i(\mathbf{w}, b) = y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 \geq 0.$$

Then

Lagrangian

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \Big[y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 \Big] \qquad \text{with } \alpha_i \geq 0.$$

Expand the Lagrangian slowly

First distribute y_i inside the brackets:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 = y_i\mathbf{w}^T\mathbf{x}_i + y_i b - 1.$$

Therefore

$$\mathcal{L} = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_i \alpha_i\left(y_i\mathbf{w}^T\mathbf{x}_i + y_i b - 1\right).$$

Now distribute the minus sign:

$$\boxed{\mathcal{L} = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_i \alpha_i y_i \mathbf{w}^T\mathbf{x}_i - \sum_i \alpha_i y_i b + \sum_i \alpha_i}$$

No new SVM idea happened here; this is simply the algebraic identity $-(A+B-C) = -A-B+C$ applied inside the sum.

Differentiate with respect to $\mathbf{w}$ and b

For $\mathbf{w}$ (using $\nabla_{\mathbf{w}}\frac{1}{2}\mathbf{w}^T\mathbf{w} = \mathbf{w}$ and $\nabla_{\mathbf{w}}\mathbf{w}^T\mathbf{x}_i = \mathbf{x}_i$, §1.3.5),

$$\frac{\partial\mathcal{L}}{\partial\mathbf{w}} = \mathbf{w} - \sum_i \alpha_i y_i \mathbf{x}_i.$$

Set it to zero:

$$\boxed{\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i}$$

For b,

$$\frac{\partial\mathcal{L}}{\partial b} = -\sum_i \alpha_i y_i.$$

Set it to zero:

$$\boxed{\sum_i \alpha_i y_i = 0}$$

> **Important meaning**
> The final normal vector $\mathbf{w}$ is a **weighted combination of training samples**. This is the first hint that we may be able to solve the problem using relationships between training samples rather than constructing $\mathbf{w}$ directly.

3.1.8 From the Lagrangian to the Dual Problem

The **primal** solves directly for $\mathbf{w}, b$. The **dual** rewrites the same optimization problem so that we solve for the constraint weights $\alpha_1, \ldots, \alpha_n$ instead.

Where do the indices i and j come from?

We have

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i.$$

To compute $\|\mathbf{w}\|^2 = \mathbf{w}^T\mathbf{w}$, we multiply one copy of this sum by another copy:

$$\Big(\sum_i \alpha_i y_i \mathbf{x}_i\Big)^T \Big(\sum_j \alpha_j y_j \mathbf{x}_j\Big).$$

We use i for the sample chosen from the first sum and j for the sample chosen from the second sum. With two samples, the double sum simply represents the four pairings $(1,1), (1,2), (2,1), (2,2)$.

Why is only $\mathbf{x}_i$ visibly transposed?

α_i and y_i are scalars, so transposing them changes nothing. $\mathbf{x}_i$ is a column vector, so its transpose is a row vector. Therefore

$$(\alpha_i y_i \mathbf{x}_i)^T = \alpha_i y_i \mathbf{x}_i^T.$$

Hence

$$\boxed{\|\mathbf{w}\|^2 = \sum_i \sum_j \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^T \mathbf{x}_j}$$

Substitute w back into the Lagrangian

From the expanded Lagrangian,

$$\mathcal{L} = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_i \alpha_i y_i \mathbf{w}^T \mathbf{x}_i - b \sum_i \alpha_i y_i + \sum_i \alpha_i.$$

The b term vanishes because

$$\sum_i \alpha_i y_i = 0.$$

For the middle term, substitute $\mathbf{w} = \sum_j \alpha_j y_j \mathbf{x}_j$:

$$\sum_i \alpha_i y_i \mathbf{w}^T \mathbf{x}_i = \sum_i \sum_j \alpha_i \alpha_j y_i y_j \, \mathbf{x}_j^T \mathbf{x}_i = \|\mathbf{w}\|^2.$$

So the Lagrangian becomes $\frac{1}{2}\|\mathbf{w}\|^2 - \|\mathbf{w}\|^2 + \sum_i \alpha_i = \sum_i \alpha_i - \frac{1}{2}\|\mathbf{w}\|^2$. Collecting the matching double sums, we obtain the **dual function**

Dual function

$$g(\boldsymbol{\alpha}) = \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^T \mathbf{x}_j$$

3.1.9 Why Does the Dual Maximize?

This is the conceptual step that makes the dual formulation make sense.

The original primal **minimizes** an objective $f(\mathbf{w}, b)$ subject to constraints $g_i(\mathbf{w}, b) \geq 0$. Its Lagrangian is

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = f(\mathbf{w}, b) - \sum_i \alpha_i g_i(\mathbf{w}, b), \qquad \alpha_i \geq 0.$$

For any **feasible** point, $g_i(\mathbf{w}, b) \geq 0$. Since $\alpha_i \geq 0$,

$$-\alpha_i g_i(\mathbf{w}, b) \leq 0.$$

Therefore

$$\boxed{\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) \leq f(\mathbf{w}, b)}$$

for feasible solutions. Now define

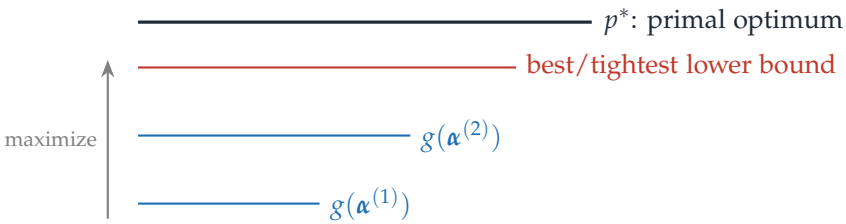
$$\boxed{g(\boldsymbol{\alpha}) = \min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})}$$

Because we minimize the Lagrangian over $\mathbf{w}, b$, $g(\boldsymbol{\alpha})$ gives a **lower bound** on the primal optimum p^*:

$$\boxed{g(\boldsymbol{\alpha}) \leq p^*}$$

Different choices of $\boldsymbol{\alpha}$ give different lower bounds. We want the one that comes as close as possible to the true primal optimum, so we choose the **largest** lower bound:

$$\boxed{\max_{\boldsymbol{\alpha}} \; g(\boldsymbol{\alpha})}$$



Toy optimization to see the max appear

Consider

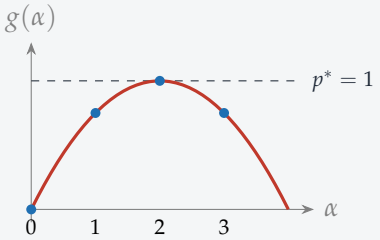
$$\min_x \; x^2 \quad \text{subject to} \quad x \geq 1.$$

The primal optimum is obviously $x = 1$ with $p^* = 1$. Write the constraint as $x - 1 \geq 0$ and form

$$\mathcal{L}(x, \alpha) = x^2 - \alpha(x - 1), \qquad \alpha \geq 0.$$

For fixed α, minimizing over x gives $2x - \alpha = 0$, so $x = \alpha/2$. Substitution gives

$$g(\alpha) = \frac{\alpha^2}{4} - \alpha\left(\frac{\alpha}{2} - 1\right) = \alpha - \frac{\alpha^2}{4}.$$



Then

$$g(0) = 0, \qquad g(1) = 0.75, \qquad g(2) = 1, \qquad g(3) = 0.75.$$

Every value is a lower bound on $p^* = 1$, and the best one is obtained by **maximizing** $g(\alpha)$. Here the max, $g(2) = 1$, hits p^* exactly: for convex problems like this one (and the SVM), the best lower bound is not just close, it equals the primal optimum.

Memory line

The primal **minimizes** the original objective. The dual **maximizes** the tightest lower bound on that minimum.

3.1.10 Final Hard-Margin Dual Problem

SVM dual

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to} \quad & \sum_{i=1}^{n} \alpha_i y_i = 0, \\ & \alpha_i \geq 0, \qquad i = 1, \ldots, n. \end{aligned}$$

Why the dual matters later

The training data appear **only through dot products** $\mathbf{x}_i^T \mathbf{x}_j$. This is the doorway to **kernel methods**: later, $\mathbf{x}_i^T \mathbf{x}_j$ can be replaced by a kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ without explicitly constructing the high-dimensional feature map.

Once the α_i are found, $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$, so a new point is classified by

$$\operatorname{sign}(\mathbf{w}^T \mathbf{x} + b) = \operatorname{sign}\Big(\sum_i \alpha_i y_i \, \mathbf{x}_i^T \mathbf{x} + b \Big),$$

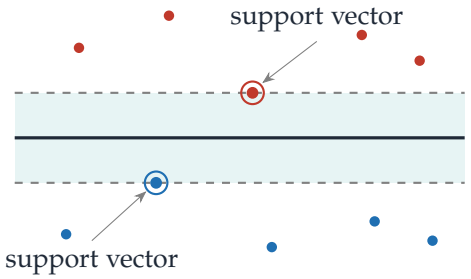
which again only needs dot products between the new point and training points.

3.1.11 Support Vectors and the Geometry of the Final Classifier

Support vectors The training points that **touch the margin hyperplanes**. Geometrically, they satisfy

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1.$$

They are the closest points to the decision boundary and therefore limit how wide the margin can become.



Points far from the boundary are **not** what limits the margin. Moving a non-support-vector point slightly usually leaves the maximum-margin separator unchanged. In the dual representation, the support vectors are the training samples that retain **nonzero coefficients** ($\alpha_i > 0$); every other point has $\alpha_i = 0$ and drops out of $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$. We use this result geometrically here without the KKT derivation.

Mental model

SVM largely **ignores the "easy" points** far away from the boundary. The difficult, closest points determine the separator.

3.1.12 Lecture Worked Example

The lecture uses ten linearly separable points:

$$D_+ = \{(2,2),\ (3,1.6),\ (2.6,2.6),\ (3,2.6),\ (2,3)\},$$

$$D_- = \{(-1,-1),\ (0,-2),\ (-2,0),\ (-1.4,-1.2),\ (0.6,-1.6)\}.$$

One valid separator is $x_1 + x_2 = 0.5$, but SVM seeks the **maximum-margin** separator rather than merely any separator.

The exact hard-margin solution given in the lecture is

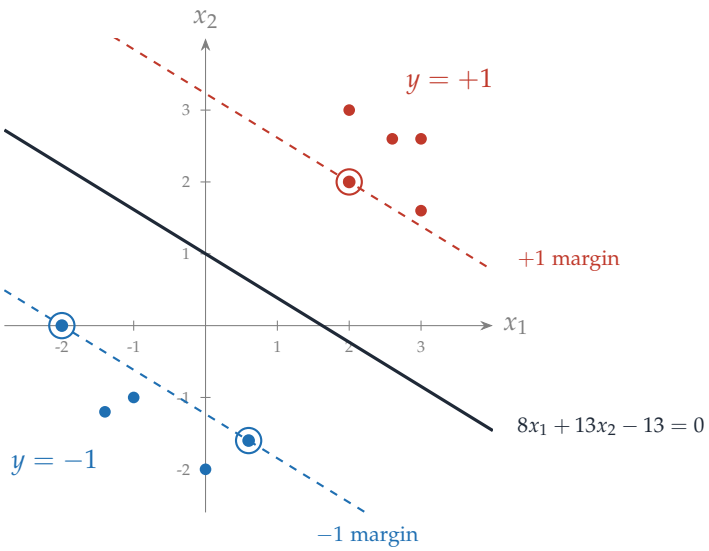
$$\boxed{\mathbf{w}^* = \left(\frac{8}{29}, \frac{13}{29}\right) \approx (0.276,\ 0.448), \qquad b^* = -\frac{13}{29} \approx -0.448}$$

Thus the decision boundary is (multiplying through by 29)

$$\boxed{8x_1 + 13x_2 - 13 = 0}$$

The geometric margin is

$$\|\mathbf{w}^*\| = \frac{\sqrt{8^2 + 13^2}}{29} = \frac{\sqrt{233}}{29} \approx 0.526, \qquad \boxed{\frac{2}{\|\mathbf{w}^*\|} \approx 3.80}$$



Exactly three points lie on the margins and are the **support vectors** (circled):

$$\boxed{(2,2), \qquad (-2,0), \qquad (0.6,-1.6)}$$

They satisfy $y_i(\mathbf{w}^{*T}\mathbf{x}_i + b^*) = 1$ and determine the geometry of the final separator.

Check: $\mathbf{w}^{*T}\mathbf{x} + b^* = (8x_1 + 13x_2 - 13)/29$

Point	y	$8x_1 + 13x_2 - 13$	$y(\mathbf{w}^{*T}\mathbf{x} + b^*)$
$(2,2)$	$+1$	$16 + 26 - 13 = 29$	**1** (support vector)
$(-2,0)$	-1	$-16 + 0 - 13 = -29$	**1** (support vector)
$(0.6,-1.6)$	-1	$4.8 - 20.8 - 13 = -29$	**1** (support vector)
$(3,1.6)$	$+1$	$24 + 20.8 - 13 = 31.8$	1.10
$(-1,-1)$	-1	$-8 - 13 - 13 = -34$	1.17

Support vectors give exactly 1; every other point gives more than 1 (it sits beyond its margin). The remaining points $(2.6, 2.6)$, $(3, 2.6)$, $(2, 3)$, $(0, -2)$, $(-1.4, -1.2)$ also give values > 1.

3.1.13 Quick Reference

Concept	What to remember
Decision boundary	$\mathbf{w}^T\mathbf{x} + b = 0$. The vector $\mathbf{w}$ is perpendicular to the hyperplane and points toward the positive side.
Signed side-test	$\mathbf{w}^T\mathbf{x} + b > 0$ positive side; < 0 negative side; $= 0$ on the boundary.
Point-to-hyperplane distance	$\lvert\mathbf{w}^T\mathbf{x} + b\rvert / \lVert\mathbf{w}\rVert$. The denominator removes arbitrary scaling of the line equation.
Margin planes	$\mathbf{w}^T\mathbf{x} + b = +1$ and $\mathbf{w}^T\mathbf{x} + b = -1$. The choice ± 1 is a convenient normalization.
Margin width	$2/\lVert\mathbf{w}\rVert$. Therefore maximum margin is equivalent to minimizing $\lVert\mathbf{w}\rVert$.
Primal objective	$\min \frac{1}{2}\lVert\mathbf{w}\rVert^2$. The $\frac{1}{2}$ and square simplify differentiation without changing the minimizer.
Hard-margin constraint	$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$. Positive points stay beyond $+1$ and negative points beyond -1.
Lagrangian	$\mathcal{L} = \frac{1}{2}\lVert\mathbf{w}\rVert^2 - \sum_i \alpha_i[y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1]$, $\alpha_i \geq 0$.
Stationarity results	$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ and $\sum_i \alpha_i y_i = 0$.
Dual	$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2}\sum_i \sum_j \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$, s.t. $\sum_i \alpha_i y_i = 0$, $\alpha_i \geq 0$.
Dual idea	Solve for α_i instead of $\mathbf{w}, b$; maximize the tightest lower bound on the primal minimum.
Kernel bridge	The dual uses samples only through dot products $\mathbf{x}_i^T\mathbf{x}_j$.
Support vectors	Closest points on the margins ($\alpha_i > 0$); they determine the maximum-margin separator.

One-line summary SVM chooses the separating hyperplane whose closest training points are as far from the boundary as possible, and the dual reformulates this geometry in terms of pairwise dot products between training samples.

PART 3.2

Linear Regression: OLS, WLS & Ridge

The flow: linear model → residual sum of squares → OLS closed form → projection onto Col(X) → normal equations from geometry → homoscedasticity / heteroscedasticity → WLS → multicollinearity → Ridge regularization.

Added note

Scope for this lecture. These notes cover OLS, WLS, and Ridge Regression. Lasso and Elastic Net are intentionally omitted because they were marked as skipped material.

3.2.1 Ordinary Least Squares (OLS): What Are We Solving?

In regression the target is **continuous**. The linear model is

$$\boxed{\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\epsilon}}$$

with $X \in \mathbb{R}^{n\times p}$, $\boldsymbol{\beta} \in \mathbb{R}^p$, $\mathbf{y} \in \mathbb{R}^n$, and noise $\boldsymbol{\epsilon} \in \mathbb{R}^n$.

The prediction vector is

$$\hat{\mathbf{y}} = X\boldsymbol{\beta}.$$

For sample i, the **residual** is

$$e_i = y_i - \hat{y}_i = y_i - \mathbf{x}_i^T\boldsymbol{\beta}.$$

OLS chooses $\boldsymbol{\beta}$ to minimize the **residual sum of squares (RSS)**:

OLS objective

$$L(\boldsymbol{\beta}) = \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 = \sum_{i=1}^{n}(y_i - \mathbf{x}_i^T\boldsymbol{\beta})^2$$

This is exactly the squared-residual loss from the identities table (§1.3.5), and RSS$/n$ is the MSE from §2.3.8.

Why square the residuals?

If we simply summed residuals, positive and negative mistakes could cancel. Squaring makes every error nonnegative and penalizes large errors more strongly.

From the norm to an expression we can differentiate

Use $\|\mathbf{v}\|^2 = \mathbf{v}^T\mathbf{v}$:

$$L(\boldsymbol{\beta}) = (\mathbf{y} - X\boldsymbol{\beta})^T(\mathbf{y} - X\boldsymbol{\beta}).$$

Expand just like ordinary algebra:

$$L(\boldsymbol{\beta}) = \mathbf{y}^T\mathbf{y} - \mathbf{y}^TX\boldsymbol{\beta} - \boldsymbol{\beta}^TX^T\mathbf{y} + \boldsymbol{\beta}^TX^TX\boldsymbol{\beta}.$$

The two middle terms are equal because each is a scalar:

$$\mathbf{y}^TX\boldsymbol{\beta} = \left(\mathbf{y}^TX\boldsymbol{\beta}\right)^T = \boldsymbol{\beta}^TX^T\mathbf{y}.$$

Therefore

$$\boxed{L(\boldsymbol{\beta}) = \mathbf{y}^T\mathbf{y} - 2\boldsymbol{\beta}^T X^T\mathbf{y} + \boldsymbol{\beta}^T X^T X\boldsymbol{\beta}}$$

Added note

Fast way to see that $\mathbf{y}^T X\boldsymbol{\beta}$ is a scalar. Read it as $\mathbf{y}^T(X\boldsymbol{\beta})$. The quantity $X\boldsymbol{\beta}$ is a vector in $\mathbb{R}^n$, so $\mathbf{y}^T(X\boldsymbol{\beta})$ is a vector dot product. A vector-transpose times a vector gives one number.

3.2.2 Deriving the OLS Gradient from First Principles

Differentiate the expanded loss term by term.

Term 1: $\mathbf{y}^T\mathbf{y}$

$\mathbf{y}$ does not depend on $\boldsymbol{\beta}$, so

$$\frac{\partial}{\partial\boldsymbol{\beta}}(\mathbf{y}^T\mathbf{y}) = \mathbf{0}.$$

Term 2: $-2\boldsymbol{\beta}^T X^T\mathbf{y}$

Since $X^T\mathbf{y}$ is constant with respect to $\boldsymbol{\beta}$ (this is the $\mathbf{a}^T\mathbf{x}$ identity from §1.3.5),

$$\frac{\partial}{\partial\boldsymbol{\beta}}(\boldsymbol{\beta}^T\mathbf{a}) = \mathbf{a} \qquad \Longrightarrow \qquad \frac{\partial}{\partial\boldsymbol{\beta}}(-2\boldsymbol{\beta}^T X^T\mathbf{y}) = -2X^T\mathbf{y}.$$

Term 3: the quadratic form $\boldsymbol{\beta}^T A\boldsymbol{\beta}$

A useful matrix-calculus identity (the quadratic form from §1.3.5) is

$$\frac{\partial}{\partial\boldsymbol{\beta}}(\boldsymbol{\beta}^T A\boldsymbol{\beta}) = (A + A^T)\boldsymbol{\beta}.$$

Rather than memorizing it blindly, expand a two-dimensional case. Let

$$\boldsymbol{\beta} = \begin{bmatrix}\beta_1\\ \beta_2\end{bmatrix}, \qquad A = \begin{bmatrix}a & b\\ c & d\end{bmatrix}.$$

Then

$$\boldsymbol{\beta}^T A\boldsymbol{\beta} = \begin{bmatrix}\beta_1 & \beta_2\end{bmatrix}\begin{bmatrix}a & b\\ c & d\end{bmatrix}\begin{bmatrix}\beta_1\\ \beta_2\end{bmatrix} = a\beta_1^2 + (b+c)\beta_1\beta_2 + d\beta_2^2.$$

Ordinary partial differentiation gives

$$\nabla_{\boldsymbol{\beta}}(\boldsymbol{\beta}^T A\boldsymbol{\beta}) = \begin{bmatrix}2a\beta_1 + (b+c)\beta_2\\ (b+c)\beta_1 + 2d\beta_2\end{bmatrix} = (A + A^T)\boldsymbol{\beta}.$$

For OLS, $A = X^T X$. This matrix is **symmetric** because

$$(X^T X)^T = X^T X.$$

Hence

$$\frac{\partial}{\partial\boldsymbol{\beta}}(\boldsymbol{\beta}^T X^T X\boldsymbol{\beta}) = 2X^T X\boldsymbol{\beta}.$$

Putting the three derivatives together:

$$\nabla_\beta L = -2X^T\mathbf{y} + 2X^TX\boldsymbol{\beta} = -2X^T(\mathbf{y} - X\boldsymbol{\beta}).$$

Therefore

$$\boxed{\frac{\partial L}{\partial \boldsymbol{\beta}} = -2X^T(\mathbf{y} - X\boldsymbol{\beta})}$$

which is exactly the least-squares gradient $2A^T(A\mathbf{x} - \mathbf{b})$ from §1.3.5 with $A \to X$, $\mathbf{b} \to \mathbf{y}$.

Set the gradient to zero

At the least-squares minimum,

$$-2X^T(\mathbf{y} - X\boldsymbol{\beta}) = \mathbf{0}.$$

Divide by -2 and distribute X^T:

$$\begin{aligned} X^T(\mathbf{y} - X\boldsymbol{\beta}) &= \mathbf{0} \\ X^T\mathbf{y} - X^TX\boldsymbol{\beta} &= \mathbf{0} \\ X^TX\boldsymbol{\beta} &= X^T\mathbf{y}. \end{aligned}$$

Normal equations

$$X^TX\boldsymbol{\beta} = X^T\mathbf{y}$$

If X^TX is invertible, multiply by $(X^TX)^{-1}$ on the left. Since $(X^TX)^{-1}(X^TX) = I$,

$$\boxed{\hat{\boldsymbol{\beta}}_{\text{OLS}} = (X^TX)^{-1}X^T\mathbf{y}}$$

The loss is a convex bowl in $\boldsymbol{\beta}$ (its Hessian $2X^TX$ is positive semidefinite), so this flat point is the global minimum (§1.3.6), not just a stationary point.

Added note

Why did X^TX not just "cancel" earlier? Matrices are not ordinary fractions. Cancellation is justified only by multiplying with the appropriate inverse (§1.1.1). The matrix analogue of $\frac{1}{a}a = 1$ is $A^{-1}A = I$.

3.2.3 The Geometry of OLS: the Normal Equations Come from the Picture

For a single predictor with an intercept,

$$\hat{\mathbf{y}} = \beta_0\mathbf{1} + \beta_1\mathbf{x}.$$

This means every possible prediction vector is a linear combination of the two vectors $\mathbf{1}$ and $\mathbf{x}$.

Key geometric statement Because $\hat{\mathbf{y}} = \beta_0\mathbf{1} + \beta_1\mathbf{x}$, the fitted vector $\hat{\mathbf{y}}$ must lie in

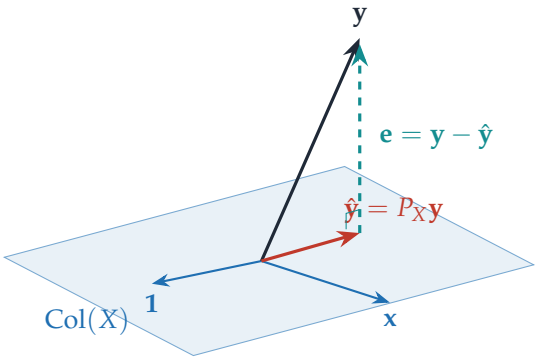
$$\text{Col}(X) = \text{span}\{\mathbf{1}, \mathbf{x}\}.$$

The true target vector $\mathbf{y}$ generally does **not** lie in that plane — this is exactly the "$\mathbf{b} \notin \text{Col}(A)$, no exact solution" case from §1.1.2. OLS therefore chooses the point $\hat{\mathbf{y}}$ in the column space that is **closest** to $\mathbf{y}$.

The shortest route from a point to a plane is **perpendicular** to the plane. Therefore the residual

$$\mathbf{e} = \mathbf{y} - \hat{\mathbf{y}}$$

is perpendicular to the entire column space.



Because the plane is generated by $\mathbf{1}$ and $\mathbf{x}$, perpendicular to the plane means perpendicular to **both** generating vectors:

$$\boxed{(\mathbf{y} - \hat{\mathbf{y}}) \cdot \mathbf{1} = 0} \qquad \boxed{(\mathbf{y} - \hat{\mathbf{y}}) \cdot \mathbf{x} = 0}$$

Equivalently, $\mathbf{1}^T(\mathbf{y} - \hat{\mathbf{y}}) = 0$ and $\mathbf{x}^T(\mathbf{y} - \hat{\mathbf{y}}) = 0$.

Derive the normal equations directly from the geometry

Substitute $\hat{\mathbf{y}} = \beta_0\mathbf{1} + \beta_1\mathbf{x}$.

First orthogonality equation:

$$\begin{aligned} \mathbf{1}^T(\mathbf{y} - \beta_0\mathbf{1} - \beta_1\mathbf{x}) &= 0 \\ \mathbf{1}^T\mathbf{y} - \beta_0\mathbf{1}^T\mathbf{1} - \beta_1\mathbf{1}^T\mathbf{x} &= 0 \\ (\mathbf{1}^T\mathbf{1})\beta_0 + (\mathbf{1}^T\mathbf{x})\beta_1 &= \mathbf{1}^T\mathbf{y}. \end{aligned}$$

Second orthogonality equation:

$$\begin{aligned} \mathbf{x}^T(\mathbf{y} - \beta_0\mathbf{1} - \beta_1\mathbf{x}) &= 0 \\ (\mathbf{x}^T\mathbf{1})\beta_0 + (\mathbf{x}^T\mathbf{x})\beta_1 &= \mathbf{x}^T\mathbf{y}. \end{aligned}$$

Stack the two equations:

$$\begin{bmatrix} \mathbf{1}^T\mathbf{1} & \mathbf{1}^T\mathbf{x} \\ \mathbf{x}^T\mathbf{1} & \mathbf{x}^T\mathbf{x} \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = \begin{bmatrix} \mathbf{1}^T\mathbf{y} \\ \mathbf{x}^T\mathbf{y} \end{bmatrix}.$$

Now define

$$X = \begin{bmatrix} \mathbf{1} & \mathbf{x} \end{bmatrix}, \qquad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}.$$

Then the entire system collapses to

$$\boxed{X^TX\boldsymbol{\beta} = X^T\mathbf{y}}$$

The important connection Calculus gave $X^T(\mathbf{y} - X\hat{\boldsymbol{\beta}}) = \mathbf{0}$. Geometry gives exactly the same equation because $\mathbf{y} - X\hat{\boldsymbol{\beta}}$ is perpendicular to every column of X. These are not two different facts; they are the **same least-squares condition written in two languages**.

Projection matrix viewpoint

Using $\hat{\boldsymbol{\beta}} = (X^T X)^{-1} X^T \mathbf{y}$,

$$\hat{\mathbf{y}} = X\hat{\boldsymbol{\beta}} = X(X^T X)^{-1} X^T \mathbf{y}.$$

Define

$$\boxed{P_X = X(X^T X)^{-1} X^T}$$

Then $\hat{\mathbf{y}} = P_X \mathbf{y}$, so P_X is the matrix that **projects** $\mathbf{y}$ onto $\mathrm{Col}(X)$.

3.2.4 OLS Numerical Example from the Lecture

Example

Take

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}.$$

Then

$$X^T X = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}, \qquad X^T \mathbf{y} = \begin{bmatrix} 5 \\ 11 \end{bmatrix}$$

$$\hat{\boldsymbol{\beta}} = (X^T X)^{-1} X^T \mathbf{y} = \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} \begin{bmatrix} 5 \\ 11 \end{bmatrix} \approx \begin{bmatrix} 0.6667 \\ 0.5 \end{bmatrix}.$$

Hence the fitted line is

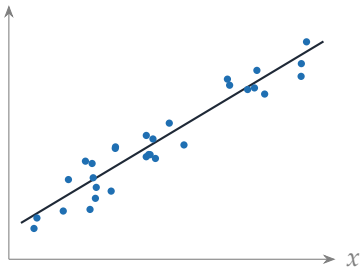
$$\boxed{\hat{y} = 0.6667 + 0.5x}$$

3.2.5 Homoscedasticity vs. Heteroscedasticity

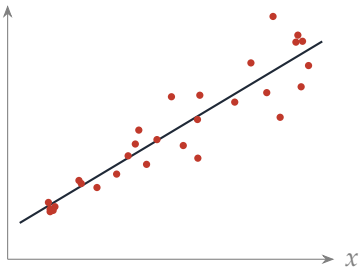
OLS assumes that the residual variance is **constant** across observations:

$$\boxed{\mathrm{Var}(\epsilon_i) = \sigma^2}$$

This is **homoscedasticity**. If the error spread changes with x, the data are **heteroscedastic**.

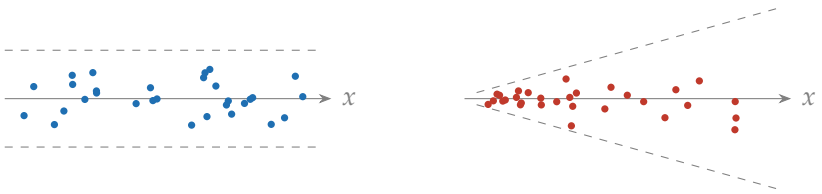


homoscedastic:
constant spread



heteroscedastic:
spread grows with x

A **residual plot** (residuals vs x) makes the difference especially clear: a constant band vs a funnel that widens.



residuals: flat band residuals: widening funnel

Mental model

Homoscedasticity means "the model is about equally uncertain everywhere." Heteroscedasticity means "some regions of the data are much noisier than others."

3.2.6 Weighted Least Squares (WLS)

If some observations are noisier than others, OLS still treats every squared residual equally. WLS changes the objective so **more reliable observations can matter more**.

Instead of $\sum_i e_i^2$, WLS minimizes

WLS objective

$$\sum_{i=1}^{n} w_i e_i^2 = \sum_{i=1}^{n} w_i (y_i - \mathbf{x}_i^T \boldsymbol{\beta})^2$$

A standard choice is

$$w_i \propto \frac{1}{\sigma_i^2},$$

so large variance $\Rightarrow$ small weight, and small variance $\Rightarrow$ large weight.

Define the diagonal weight matrix (§1.1.4: each residual handled on its own, nothing mixed)

$$W = \operatorname{diag}(w_1, w_2, \ldots, w_n).$$

Then the loss becomes

$$\boxed{L(\boldsymbol{\beta}) = (\mathbf{y} - X\boldsymbol{\beta})^T W (\mathbf{y} - X\boldsymbol{\beta})}$$

Why is W in the middle?

Let

$$\mathbf{e} = \mathbf{y} - X\boldsymbol{\beta} = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix}, \qquad W = \begin{bmatrix} w_1 & 0 & 0 \\ 0 & w_2 & 0 \\ 0 & 0 & w_3 \end{bmatrix}.$$

Then

$$W\mathbf{e} = \begin{bmatrix} w_1 e_1 \\ w_2 e_2 \\ w_3 e_3 \end{bmatrix} \qquad \text{and therefore} \qquad \mathbf{e}^T W \mathbf{e} = \begin{bmatrix} e_1 & e_2 & e_3 \end{bmatrix} \begin{bmatrix} w_1 e_1 \\ w_2 e_2 \\ w_3 e_3 \end{bmatrix} = w_1 e_1^2 + w_2 e_2^2 + w_3 e_3^2.$$

So the middle matrix **scales each residual** before the final dot product. (Same mechanics as $\mathbf{x}^T A \mathbf{x}$ in §1.1.7: the matrix in the middle decides how much each direction costs.)

OLS is a special case of WLS If every observation has the same weight, $W = I$, then $\mathbf{e}^T W \mathbf{e} = \mathbf{e}^T \mathbf{e}$, which is ordinary least squares.

WLS derivation

Differentiate (same shape as the OLS gradient, with W riding along):

$$\frac{\partial L}{\partial \boldsymbol{\beta}} = -2X^T W(\mathbf{y} - X\boldsymbol{\beta}).$$

Set to zero:

$$\begin{aligned} X^T W(\mathbf{y} - X\boldsymbol{\beta}) &= \mathbf{0} \\ X^T W\mathbf{y} - X^T WX\boldsymbol{\beta} &= \mathbf{0} \\ X^T WX\boldsymbol{\beta} &= X^T W\mathbf{y}. \end{aligned}$$

If $X^T WX$ is invertible,

$$\boxed{\hat{\boldsymbol{\beta}}_{\text{WLS}} = (X^T WX)^{-1} X^T W\mathbf{y}}$$

In the lecture example with strongly non-constant variance, WLS changes the fitted line because high-variance observations are given less influence.

3.2.7 Ridge Regression: a Different Problem

WLS addressed unequal noise **across observations**. Ridge addresses a different issue: **multicollinearity**, where predictor **columns** of X are strongly correlated.

Suppose two feature columns are almost the same:

$$\mathbf{x}_1 \approx \mathbf{x}_2.$$

Then

$$\hat{\mathbf{y}} = \beta_1 \mathbf{x}_1 + \beta_2 \mathbf{x}_2.$$

If they were exactly equal, $\mathbf{x}_1 = \mathbf{x}_2 = \mathbf{x}$, then

$$\hat{\mathbf{y}} = (\beta_1 + \beta_2)\mathbf{x}.$$

The model can identify the **sum** $\beta_1 + \beta_2$, but not how that total should be split between the two coefficients. Many coefficient pairs can produce almost the same predictions.

Concrete intuition

If $\mathbf{x}_1 = \mathbf{x}_2 = \mathbf{x}$, then $(\beta_1, \beta_2) = (1, 0)$ and $(100, -99)$ both give $\hat{\mathbf{y}} = \mathbf{x}$. OLS can therefore become numerically unstable and assign large coefficients with opposite signs even though the predictions barely change.

The OLS formula contains

$$(X^T X)^{-1}.$$

Highly correlated columns make $X^T X$ **nearly singular** — this is the redundancy-lowers-rank story from §1.1.2, happening to $X^T X$ — so its inverse becomes unstable. Tiny changes in the data can then cause large swings in $\hat{\boldsymbol{\beta}}$.

3.2.8 Ridge Objective and Closed-Form Solution

Ridge keeps the least-squares fit term but adds a penalty on coefficient magnitude — the L_2 regularization from §1.1.7:

Ridge objective

$$L(\boldsymbol{\beta}) = \underbrace{\|\mathbf{y} - X\boldsymbol{\beta}\|_2^2}_{\text{fit the data}} + \underbrace{\lambda\|\boldsymbol{\beta}\|_2^2}_{\text{discourage large coefficients}}, \qquad \lambda \geq 0.$$

Since $\|\boldsymbol{\beta}\|^2 = \boldsymbol{\beta}^T\boldsymbol{\beta}$, its derivative is

$$\frac{\partial}{\partial \boldsymbol{\beta}}(\lambda \boldsymbol{\beta}^T\boldsymbol{\beta}) = 2\lambda\boldsymbol{\beta}.$$

Therefore

$$\nabla_\beta L = -2X^T(\mathbf{y} - X\boldsymbol{\beta}) + 2\lambda\boldsymbol{\beta}.$$

Set equal to zero:

$$\begin{aligned} -2X^T(\mathbf{y} - X\boldsymbol{\beta}) + 2\lambda\boldsymbol{\beta} &= \mathbf{0} \\ -X^T\mathbf{y} + X^TX\boldsymbol{\beta} + \lambda\boldsymbol{\beta} &= \mathbf{0} \\ X^TX\boldsymbol{\beta} + \lambda\boldsymbol{\beta} &= X^T\mathbf{y}. \end{aligned}$$

To combine the two terms multiplying $\boldsymbol{\beta}$, write $\lambda\boldsymbol{\beta} = \lambda I\boldsymbol{\beta}$. Then

$$(X^TX + \lambda I)\boldsymbol{\beta} = X^T\mathbf{y}$$

and hence

$$\boxed{\hat{\boldsymbol{\beta}}_{\text{ridge}} = (X^TX + \lambda I)^{-1}X^T\mathbf{y}}$$

Why does adding λI help?

Adding λI increases the diagonal entries of X^TX. In the lecture framing, this makes the matrix invertible even when predictors are collinear and improves numerical stability. For example,

$$X^TX = \begin{bmatrix} 4 & 3.99 \\ 3.99 & 4 \end{bmatrix} \qquad \longrightarrow \qquad X^TX + I = \begin{bmatrix} 5 & 3.99 \\ 3.99 & 5 \end{bmatrix}.$$

In eigenvalue language (§1.1.5): $X^TX + \lambda I$ has eigenvalues $\lambda_i + \lambda$, so every near-zero eigenvalue that made the inverse explode gets lifted safely away from 0.

Ridge tradeoff

Ridge deliberately introduces some **bias** in exchange for lower **variance** and more stable coefficients (the same trade-off as §2.3.2). It may accept a slightly worse fit to the training data in order to generalize more reliably.

3.2.9 Ridge Worked Example from the Lecture

The lecture uses

$$X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0.9 \\ 1 & 0.9 & 1 \\ 1 & 0.9 & 0.9 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 2.8 \\ 2.5 \\ 2.6 \\ 2.4 \end{bmatrix}.$$

The non-intercept predictor columns are extremely similar, so $X^T X$ is nearly singular:

$$X^T X = \begin{bmatrix} 4 & 3.8 & 3.8 \\ 3.8 & 3.62 & 3.61 \\ 3.8 & 3.61 & 3.62 \end{bmatrix}, \qquad X^T \mathbf{y} = \begin{bmatrix} 10.3 \\ 9.8 \\ 9.81 \end{bmatrix}.$$

(Its determinant is 0.0004 — practically the collapsed-dimension case from §1.1.1.) The lecture reports the OLS estimate

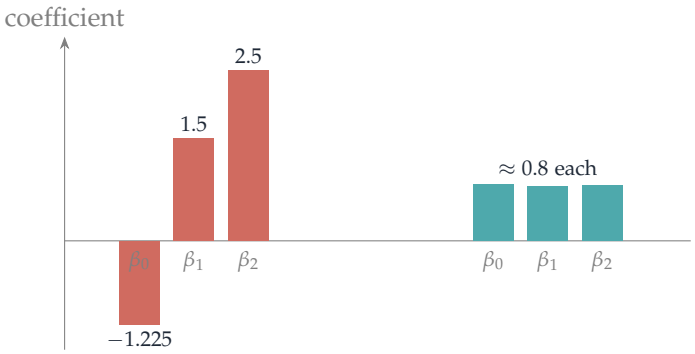
$$\hat{\boldsymbol{\beta}}_{\text{OLS}} \approx \begin{bmatrix} -1.225 \\ 1.500 \\ 2.500 \end{bmatrix}.$$

The coefficients are comparatively large and uneven despite the predictor columns carrying nearly the same information.

Now set $\lambda = 1$:

$$X^T X + I = \begin{bmatrix} 5 & 3.8 & 3.8 \\ 3.8 & 4.62 & 3.61 \\ 3.8 & 3.61 & 4.62 \end{bmatrix} \quad \Rightarrow \quad \hat{\boldsymbol{\beta}}_{\text{ridge}} \approx \begin{bmatrix} 0.835 \\ 0.801 \\ 0.811 \end{bmatrix}.$$

The coefficients are smaller and much more similar, reflecting the fact that the columns contain similar information.



OLS: large, uneven **Ridge ($\lambda = 1$): small, balanced**

What happens as λ changes?

$$\boxed{\lambda \to 0 \;\Rightarrow\; \text{Ridge} \to \text{OLS}} \qquad \boxed{\lambda \to \infty \;\Rightarrow\; \hat{\boldsymbol{\beta}}_{\text{ridge}} \to \mathbf{0}}$$

A moderate λ balances fit and stability. This is why L_2 regularization keeps weights *small but not zero* (§1.1.7): the circle has no corners, so Ridge shrinks rather than deletes.

3.2.10 Quick Reference: OLS vs. WLS vs. Ridge

	OLS	WLS	Ridge
Problem addressed	baseline linear regression	unequal residual variance / reliability	multicollinearity and unstable coefficients
Objective	$\|\mathbf{y} - X\boldsymbol{\beta}\|^2$	$(\mathbf{y} - X\boldsymbol{\beta})^T W(\mathbf{y} - X\boldsymbol{\beta})$	$\|\mathbf{y} - X\boldsymbol{\beta}\|^2 + \lambda\|\boldsymbol{\beta}\|^2$
Normal equations	$X^T X\boldsymbol{\beta} = X^T\mathbf{y}$	$X^T WX\boldsymbol{\beta} = X^T W\mathbf{y}$	$(X^T X + \lambda I)\boldsymbol{\beta} = X^T\mathbf{y}$
Closed form	$(X^T X)^{-1}X^T\mathbf{y}$	$(X^T WX)^{-1}X^T W\mathbf{y}$	$(X^T X + \lambda I)^{-1}X^T\mathbf{y}$
Key idea	orthogonal projection onto $\mathrm{Col}(X)$	weight observations by reliability	penalize large coefficients

One chain to remember

$$\text{OLS: closest point in } \mathrm{Col}(X) \;\Rightarrow\; \mathbf{e} \perp \mathrm{Col}(X) \;\Rightarrow\; X^T\mathbf{e} = \mathbf{0} \;\Rightarrow\; X^T X\boldsymbol{\beta} = X^T\mathbf{y}.$$

WLS: unequal uncertainty $\Rightarrow$ W changes each residual's influence.
Ridge: correlated predictors $\Rightarrow$ $+\lambda I$ stabilizes the solution.

PART 3.3

Nonlinear Models, Kernels & k-NN

The flow: linear limitation → feature map ϕ → polynomial features → polynomial kernel → kernel trick → kernel perceptron → RBF → local vs global → valid kernels → kernel ridge regression → k-NN → bias–variance in k-NN → choosing k → cost & curse of dimensionality.

Added note

Scope. These notes integrate the professor's Chapter 3 Revised PDF and the derivations developed in class. Soft-margin SVM, KKT, Lasso, and Elastic Net are intentionally outside this part.

3.3.1 Why Linear Models Need Help

A linear classifier such as the perceptron or SVM uses a hyperplane

$$\mathbf{w}^T\mathbf{x} + b = 0$$

while ordinary least squares uses a linear prediction rule. These models work well when the relevant geometry is approximately linear, but many datasets have structure that **cannot be captured by one line or hyperplane**.

The enclosed-class example is worth setting up carefully. Suppose a sample is

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2.$$

The inner class lies near the origin while the outer class forms a ring. A straight line divides the plane into only two half-spaces, so no single line can put the entire inner class on one side and the entire ring on the other.

A useful feature can change the geometry

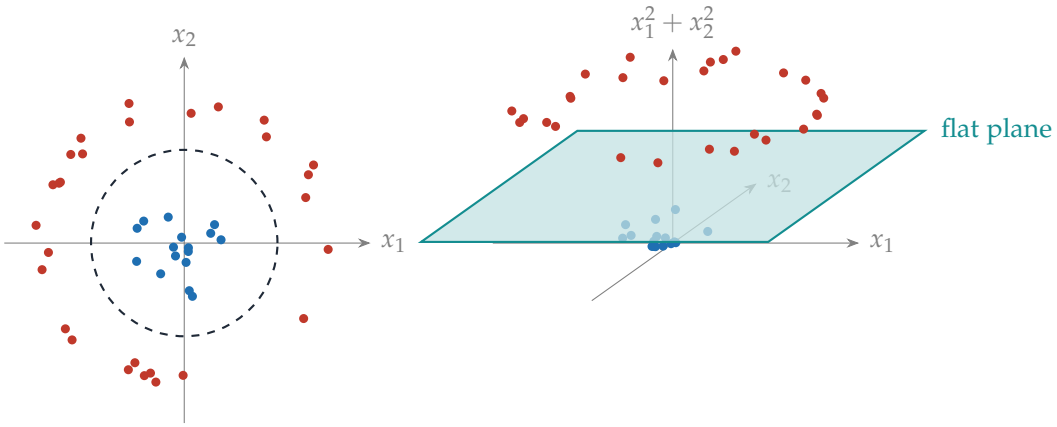
A natural feature for this geometry is the squared distance from the origin:

$$x_1^2 + x_2^2.$$

Define

$$\boxed{\phi(\mathbf{x}) = \begin{bmatrix} x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{bmatrix}} \quad \text{so that} \quad \mathbb{R}^2 \to \mathbb{R}^3.$$

Points near the center have a small third coordinate; points on the outer ring have a larger third coordinate. In the lifted space, a **flat plane** can separate them.



original space: no line works, the true boundary is a circle

lifted space: ring rises, center stays low, a flat plane separates them

Key result A boundary can be **nonlinear in the original input coordinates** while still being a **linear hyperplane in the transformed features**. For example, a plane involving the third feature $x_1^2 + x_2^2$ maps back to a circular boundary in the original 2D space.

E.g. the plane $x_1^2 + x_2^2 = 2.1$ in the lifted space is exactly the circle of radius $\sqrt{2.1} \approx 1.45$ drawn dashed on the left.

Why this matters

The goal is not "higher dimensions for their own sake." The goal is to find a **representation** in which the structure becomes easier for a linear algorithm to separate or fit.

Why not simply lift each class to a different height?

For the training set, one could cheat and define a new coordinate using the label itself, for example

$$\tilde{\phi}(\mathbf{x}) = \begin{bmatrix} x_1 \\ x_2 \\ y \end{bmatrix}.$$

Then class 0 and class 1 would sit at different heights and a plane would separate them perfectly. But this is **not a valid predictive feature map**, because for a new test sample the label y is exactly what we are trying to predict. A legitimate feature map must be **computable from the input alone**. The circular mapping

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{bmatrix}$$

is valid because $x_1^2 + x_2^2$ can be computed for any unseen point without knowing its class. (Same rule as §2.3.1: nothing the test set hides may leak into the model.)

3.3.2 Nonlinear Feature Mappings

In general, define a mapping

$$\phi : \mathbb{R}^{d_1} \to \mathbb{R}^{d_2}, \qquad d_2 > d_1.$$

A linear classifier in feature space is

$$\boxed{h_\phi(\mathbf{x}) = \operatorname{sign}\left(\mathbf{w}^T \phi(\mathbf{x}) + b\right)}$$

The mapping must be **meaningful**: moving to a larger space is useful only if the transformed coordinates expose a structure that a linear model can exploit. For the circular example, $\phi(\mathbf{x}) = [x_1,\ x_2,\ x_1^2 + x_2^2]^T$ is useful because **radius** is exactly the property distinguishing the inner and outer classes.

> **Intuition**
> A feature map is like **changing the camera angle** on the same dataset. Nothing about the labels changes; only the representation changes. From the right angle, a pattern that looked tangled can become linearly separable.

3.3.3 Polynomial Features Before Polynomial Kernels

Before defining a polynomial kernel, first understand polynomial *features*. For $\mathbf{x} = [x_1,\ x_2]^T$, a linear model only sees x_1 and x_2. If we add polynomial features, the model may also use

$$1, \quad x_1, \quad x_2, \quad x_1 x_2, \quad x_1^2, \quad x_2^2, \quad \ldots$$

A model that is linear in these transformed features can represent a **curved boundary** in the original coordinates.

What are x and x′?

Kernel formulas compare **two ordinary input samples**. We will write

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \qquad \mathbf{x}' = \begin{bmatrix} x_1' \\ x_2' \end{bmatrix}.$$

The prime in $\mathbf{x}'$ does **not** mean transformed. It simply means "a second sample." The transformed versions are $\phi(\mathbf{x})$ and $\phi(\mathbf{x}')$.

> **Clarification**
> Keep these symbols separate:
>
> $$\begin{aligned} \mathbf{x} &= \text{first original sample}, \\ \mathbf{x}' &= \text{second original sample}, \\ \phi(\mathbf{x}) &= \text{transformed version of } \mathbf{x}. \end{aligned}$$

3.3.4 The Polynomial Kernel, Built from First Principles

The degree-d polynomial kernel used in the lecture is

Polynomial kernel

$$K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^T\mathbf{x}')^d$$

For $d = 2$: $K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^T\mathbf{x}')^2$.

Why is there a $1+$?

Without the constant, $(\mathbf{x}^T\mathbf{x}')^2$ contains only degree-2 interactions. Adding 1 gives $(1 + \mathbf{x}^T\mathbf{x}')^2$, which produces degree 0, degree 1, and degree 2 terms. Thus the 1 allows **lower-order terms and a constant/bias feature** to be included.

Key result

$$(\mathbf{x}^T\mathbf{x}')^d \;\Rightarrow\; \text{degree } d \text{ terms only}, \qquad (1 + \mathbf{x}^T\mathbf{x}')^d \;\Rightarrow\; \text{degrees } 0, 1, \ldots, d.$$

Exact $n = 2$, $d = 2$ derivation

Since $\mathbf{x}^T\mathbf{x}' = x_1x_1' + x_2x_2'$, we have

$$\begin{aligned} K(\mathbf{x}, \mathbf{x}') &= (1 + x_1x_1' + x_2x_2')^2 \\ &= 1 + 2x_1x_1' + 2x_2x_2' + x_1^2(x_1')^2 + 2x_1x_2x_1'x_2' + x_2^2(x_2')^2. \end{aligned}$$

Now ask a very specific question:

Can this whole expression be written as an ordinary dot product between two transformed vectors?

Match each kernel term with a product of one feature from $\mathbf{x}$ and the corresponding feature from $\mathbf{x}'$:

$$1 = (1)(1), \qquad 2x_1x_1' = (\sqrt{2}\,x_1)(\sqrt{2}\,x_1'), \qquad 2x_1x_2x_1'x_2' = (\sqrt{2}\,x_1x_2)(\sqrt{2}\,x_1'x_2'),$$

and similarly for the remaining terms. This produces

$$\phi(\mathbf{x}) = \begin{bmatrix} 1 \\ \sqrt{2}\,x_1 \\ \sqrt{2}\,x_2 \\ \sqrt{2}\,x_1x_2 \\ x_1^2 \\ x_2^2 \end{bmatrix}, \qquad \phi(\mathbf{x}') = \begin{bmatrix} 1 \\ \sqrt{2}\,x_1' \\ \sqrt{2}\,x_2' \\ \sqrt{2}\,x_1'x_2' \\ (x_1')^2 \\ (x_2')^2 \end{bmatrix}.$$

Therefore

$$\begin{aligned} \phi(\mathbf{x})^T\phi(\mathbf{x}') &= (1)(1) + (\sqrt{2}x_1)(\sqrt{2}x_1') + (\sqrt{2}x_2)(\sqrt{2}x_2') + (\sqrt{2}x_1x_2)(\sqrt{2}x_1'x_2') + x_1^2(x_1')^2 + x_2^2(x_2')^2 \\ &= 1 + 2x_1x_1' + 2x_2x_2' + 2x_1x_2x_1'x_2' + x_1^2(x_1')^2 + x_2^2(x_2')^2 \\ &= (1 + \mathbf{x}^T\mathbf{x}')^2. \end{aligned}$$

Hence

$$\boxed{K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T \phi(\mathbf{x}')}$$

Intuition

We could transform every 2D point into the 6D vector $\phi(\mathbf{x})$ and compute there. The kernel gives the **same high-dimensional dot product directly from the original inputs**: behave as if we entered feature space without explicitly constructing it.

Check with numbers

$\mathbf{x} = (1, 2)$, $\mathbf{x}' = (3, 1)$: $\mathbf{x}^T\mathbf{x}' = 3 + 2 = 5$, so $K = (1 + 5)^2 = 36$.
The long way: $\phi(\mathbf{x}) = (1,\ \sqrt{2},\ 2\sqrt{2},\ 2\sqrt{2},\ 1,\ 4)$, $\phi(\mathbf{x}') = (1,\ 3\sqrt{2},\ \sqrt{2},\ 3\sqrt{2},\ 9,\ 1)$, so

$$\phi(\mathbf{x})^T \phi(\mathbf{x}') = 1 + 6 + 4 + 12 + 9 + 4 = 36. \checkmark$$

Same number, but the kernel needed one 2D dot product instead of building two 6D vectors.

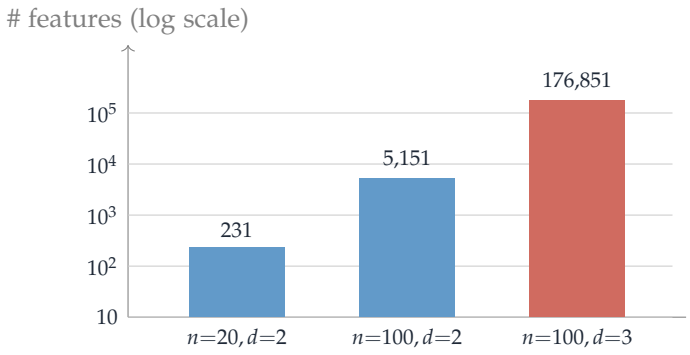
Why explicit polynomial maps become expensive

For an input $\mathbf{x} \in \mathbb{R}^n$, the lecture gives the number of polynomial monomial features up to degree d as

$$\boxed{|\phi(\mathbf{x})| = \binom{n+d}{d} = \frac{(n+d)!}{n!\, d!}}$$

(Check: $n = 2$, $d = 2$ gives $\binom{4}{2} = 6$, exactly the 6D vector above.) Examples from the lecture:

$$n = 20,\ d = 2 \Rightarrow 231, \qquad n = 100,\ d = 2 \Rightarrow 5{,}151, \qquad n = 100,\ d = 3 \Rightarrow 176{,}851.$$



Feature explosion is the motivation for computing $K(\mathbf{x}, \mathbf{x}')$ directly instead of constructing every component of $\phi(\mathbf{x})$: the kernel always costs one n-dimensional dot product, no matter how big ϕ is.

3.3.5 Why Dot Products Let Us Kernelize the Perceptron

Start with the ordinary perceptron (using learning rate $\eta = 1$ for clarity). If sample $(\mathbf{x}_i, y_i)$ is misclassified,

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \leq 0,$$

then

$$\mathbf{w} \leftarrow \mathbf{w} + y_i\mathbf{x}_i, \qquad b \leftarrow b + y_i.$$

Rewrite w as a sum of past updates

Every time training sample j causes an update, the vector $y_j\mathbf{x}_j$ is added to $\mathbf{w}$. Define

$$\boxed{\alpha_j = \text{number of times sample } j \text{ caused an update}}$$

Then after training

$$\mathbf{w} = \sum_{j=1}^{n} \alpha_j y_j \mathbf{x}_j, \qquad b = \sum_{j=1}^{n} \alpha_j y_j.$$

This is **not a new model assumption**; it is a bookkeeping rewrite of the original perceptron updates. (Same shape as the SVM result $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$ from §3.1.7 — there it came out of the Lagrangian, here out of counting updates.)

Substitute the expansion of $\mathbf{w}$ into the score for sample $\mathbf{x}_i$:

$$\mathbf{w}^T\mathbf{x}_i + b = \Big(\sum_j \alpha_j y_j \mathbf{x}_j\Big)^T \mathbf{x}_i + b = \sum_j \alpha_j y_j \underbrace{\mathbf{x}_j^T\mathbf{x}_i}_{\text{pairwise dot product}} + b.$$

The crucial fact is that the algorithm can now be written using **only dot products between samples**.

Kernelized perceptron

In feature space, the pairwise dot product would be $\phi(\mathbf{x}_j)^T\phi(\mathbf{x}_i)$. But by definition of a kernel,

$$\phi(\mathbf{x}_j)^T\phi(\mathbf{x}_i) = K(\mathbf{x}_j, \mathbf{x}_i).$$

Therefore the classifier can be written

Kernel perceptron

$$f(\mathbf{x}_i) = \operatorname{sign}\Big(\sum_{j=1}^{n} \alpha_j y_j\, K(\mathbf{x}_j, \mathbf{x}_i) + b\Big)$$

If $\mathbf{x}_i$ is misclassified, we simply increment

$$\alpha_i \leftarrow \alpha_i + 1.$$

Concrete counter example

Suppose there are five samples. Initialize $\alpha_1 = \cdots = \alpha_5 = 0$.
During the first pass, suppose samples 1, 2, and 5 cause updates. Then $\alpha_1 = 1$, $\alpha_2 = 1$, $\alpha_5 = 1$, and

$$\mathbf{w} = y_1\mathbf{x}_1 + y_2\mathbf{x}_2 + y_5\mathbf{x}_5.$$

If sample 1 causes another update on the next pass, $\alpha_1 = 2$, so

$$\mathbf{w} = 2y_1\mathbf{x}_1 + y_2\mathbf{x}_2 + y_5\mathbf{x}_5.$$

Thus α_j literally records how many times that training sample has contributed its update vector.

Key result The reason the kernel trick can be inserted into the perceptron is not magic: after rewriting $\mathbf{w}$, the prediction depends on $\mathbf{x}_j^T\mathbf{x}_i$. Replace the transformed-space dot product $\phi(\mathbf{x}_j)^T\phi(\mathbf{x}_i)$ by $K(\mathbf{x}_j, \mathbf{x}_i)$, and the explicit feature vector disappears.

This is exactly the doorway promised in the SVM dual (§3.1.10): the dual only touches $\mathbf{x}_i^T\mathbf{x}_j$, so swapping in $K(\mathbf{x}_i, \mathbf{x}_j)$ gives a **kernel SVM** the same way.

3.3.6 RBF / Gaussian Kernel

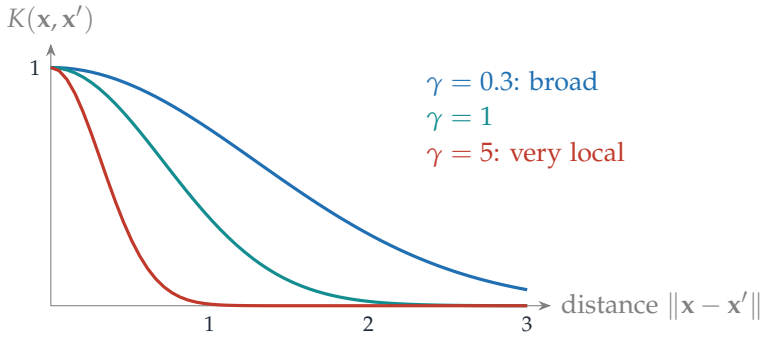
RBF kernel

$$K(\mathbf{x}, \mathbf{x}') = \exp\big(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2\big), \qquad \gamma > 0.$$

It measures similarity using Euclidean distance (§2.2.2).

If $\mathbf{x} = \mathbf{x}'$, then $\|\mathbf{x} - \mathbf{x}'\|^2 = 0$ and $K(\mathbf{x}, \mathbf{x}') = e^0 = 1$. As the distance grows, the exponent becomes more negative and the similarity **decays toward zero**.

E.g. with $\gamma = 1$: distance 1 gives $e^{-1} \approx 0.37$, distance 2 gives $e^{-4} \approx 0.018$. Doubling the distance kills the similarity almost entirely.



Small γ gives broad influence; large γ makes similarity decay quickly and produces very local influence.

Full derivation of the implicit infinite-dimensional map

Start from

$$K(\mathbf{x}, \mathbf{x}') = e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}.$$

Use the vector identity

$$\|\mathbf{x} - \mathbf{x}'\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - 2\mathbf{x}^T\mathbf{x}'.$$

Therefore

$$\begin{aligned} K(\mathbf{x}, \mathbf{x}') &= e^{-\gamma(\|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - 2\mathbf{x}^T\mathbf{x}')} \\ &= e^{-\gamma\|\mathbf{x}\|^2} e^{-\gamma\|\mathbf{x}'\|^2} e^{2\gamma\mathbf{x}^T\mathbf{x}'}. \end{aligned}$$

The first two factors each depend on only one sample; all the interaction between $\mathbf{x}$ and $\mathbf{x}'$ lives in the third. Now use the Taylor/Maclaurin series

$$e^z = \sum_{k=0}^{\infty} \frac{z^k}{k!} = 1 + z + \frac{z^2}{2!} + \frac{z^3}{3!} + \cdots .$$

Set $z = 2\gamma \mathbf{x}^T \mathbf{x}'$:

$$e^{2\gamma \mathbf{x}^T \mathbf{x}'} = \sum_{k=0}^{\infty} \frac{(2\gamma \mathbf{x}^T \mathbf{x}')^k}{k!} = 1 + 2\gamma \mathbf{x}^T \mathbf{x}' + \frac{(2\gamma)^2 (\mathbf{x}^T \mathbf{x}')^2}{2!} + \frac{(2\gamma)^3 (\mathbf{x}^T \mathbf{x}')^3}{3!} + \cdots .$$

Each $(\mathbf{x}^T \mathbf{x}')^k$ is a polynomial kernel of degree k (§3.3.4). So the expansion contains polynomial interactions of degree

$$0,\ 1,\ 2,\ 3,\ \ldots$$

with **no finite last degree**. This is why the RBF kernel corresponds to an implicit feature representation with **infinitely many components**.

> **Clarification**
>
> **Do we literally need infinite dimensions?** No. A particular finite dataset might have some finite clever mapping that separates it. The point of RBF is that we **do not need to guess that mapping in advance**. The kernel gives access to an extremely rich implicit feature space while still requiring only the simple quantity $e^{-\gamma \|\mathbf{x} - \mathbf{x}'\|^2}$.

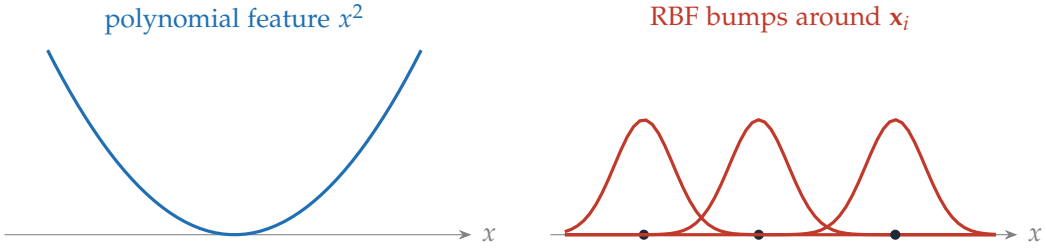
Circle versus complex local patterns

For the circular dataset, one extra feature such as $x_1^2 + x_2^2$ may be enough to create a clean 3D separation. For a pattern with many alternating local class regions, there may be no obvious single feature to invent. RBF avoids committing to one low-degree polynomial map and provides a rich family of nonlinear interactions automatically.

3.3.7 Local versus Global Geometry

The words *local* and *global* describe how an adjustment or training point influences the input space.

- **Global polynomial feature:** a feature such as x^2 or x^3 is defined everywhere. Changing its coefficient changes the fitted curve **across the whole domain**.
- **Local RBF influence:** $K(\mathbf{x}, \mathbf{x}_i) = e^{-\gamma \|\mathbf{x} - \mathbf{x}_i\|^2}$ is strongest near $\mathbf{x}_i$ and fades toward zero as we move away from $\mathbf{x}_i$.



global: alive everywhere local: fades away from each point

Intuition

Think of a polynomial model as **bending one large sheet**: changing the global shape affects distant regions too. Think of RBF as **combining many local hills**: a training point has strong influence nearby and little influence far away.

This explains the role of γ:

$$\gamma \downarrow \Rightarrow \text{ wide neighborhoods, smoother boundary,}$$

$$\gamma \uparrow \Rightarrow \text{ narrow neighborhoods, more detailed boundary, greater overfitting risk.}$$

Same bias–variance dial as §2.3.2: small γ leans toward underfitting, large γ toward overfitting.

3.3.8 Polynomial versus RBF: What Each Kernel Assumes

The polynomial kernel

$$K_{\text{poly}}(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^T\mathbf{x}')^d$$

uses a **finite** set of polynomial interactions up to degree d and imposes a more **global** geometry. Increasing d increases flexibility, but high degree can produce unstable or oscillatory global boundaries.

The RBF kernel

$$K_{\text{RBF}}(\mathbf{x}, \mathbf{x}') = e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}$$

provides an **infinite** expansion with **local** distance-based similarity. This makes it natural for complicated smooth patterns whose decision geometry varies from region to region.

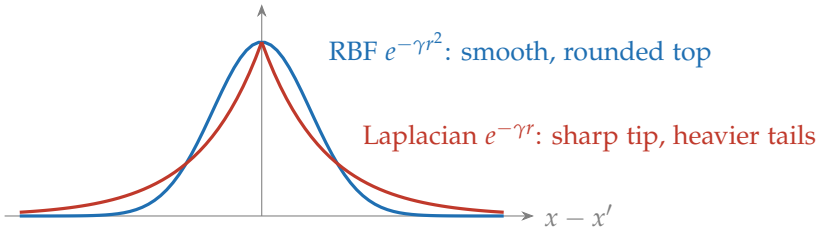
Kernel	Main geometric idea	Typical failure mode from the lecture
Linear	One hyperplane; no nonlinear mapping	Cannot capture curved/XOR-like structure
Polynomial	Global polynomial interactions up to degree d	Small d underfits; large d may overfit/oscillate; poor locality
RBF	Smooth localized similarity; very rich implicit feature space	γ too small underfits; γ too large memorizes/local-overfits
Laplacian	Local similarity with a sharper, less smooth peak	Can become overly sharp/noise-sensitive
Sigmoid	Neural-activation-like similarity	Sensitive to parameters/scaling; not valid for all parameter choices in the lecture notes

3.3.9 Exponential / Laplacian Kernel

The exponential (Laplacian) kernel uses distance **without squaring it**:

$$\boxed{K_{\text{Lap}}(\mathbf{x}, \mathbf{x}') = e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|}}$$

In one dimension this resembles $e^{-\gamma|x-x'|}$. The absolute value produces a **sharper peak** than the RBF's smooth Gaussian profile.



Same squared-vs-not-squared story as L_2 vs L_1 (§1.1.7) and MSE vs MAE (§2.3.8). The lecture relates the exponential kernel to the broader **Matérn family**: $\nu = \frac{1}{2}$ gives the exponential form, while the RBF is recovered in the infinitely smooth limit $\nu \to \infty$.

3.3.10 What Makes a Kernel Valid?

A kernel must behave like an inner product in some feature space:

$$K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T \phi(\mathbf{x}').$$

The lecture summarizes this through **positive semi-definiteness**: for any set of samples, the matrix of pairwise values $K_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ must be symmetric with no negative eigenvalues.

Why that is the right test: if K really is a dot product, then for any weights $\mathbf{c}$,

$$\mathbf{c}^T K \mathbf{c} = \sum_i \sum_j c_i c_j \, \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = \Big\| \sum_i c_i \phi(\mathbf{x}_i) \Big\|^2 \geq 0.$$

A squared length can never be negative, so a genuine dot product can never produce a matrix with a negative direction.

According to the lecture notes, valid kernels can be combined through operations such as **addition, positive scalar multiplication, products, and certain powers**. This permits composite kernels, for example

$$K = K_{\text{poly}} + K_{\text{RBF}},$$

which can mix different notions of similarity.

> **Key result** Choosing a kernel means choosing a **notion of similarity**. That notion of similarity defines the geometry available to the model and therefore the kinds of decision boundaries it can form.

3.3.11 Kernel Least Squares / Kernel Ridge Regression

Ordinary least squares (§3.2) predicts $\hat{\mathbf{y}} = X\boldsymbol{\beta}$ with $\hat{\boldsymbol{\beta}} = (X^TX)^{-1}X^T\mathbf{y}$. For nonlinear data, map each sample first:

$$\mathbf{x}_i \to \phi(\mathbf{x}_i).$$

Define the feature matrix

$$\Phi = \phi(X) = \begin{bmatrix} \phi(\mathbf{x}_1)^T \\ \phi(\mathbf{x}_2)^T \\ \vdots \\ \phi(\mathbf{x}_n)^T \end{bmatrix}.$$

Then $\hat{\mathbf{y}} = \Phi\boldsymbol{\beta}$. Explicitly computing Φ can again be expensive or impossible for a huge/infinite feature map.

The kernel matrix

Define

$$\boxed{K = \Phi\Phi^T \in \mathbb{R}^{n\times n}} \qquad \text{so that} \qquad \boxed{K_{ij} = \phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j) = K(\mathbf{x}_i, \mathbf{x}_j)}$$

K is a table of **pairwise similarities** between training samples.

What does $\boldsymbol{\alpha}$ mean here?

In kernel regression, $\boldsymbol{\alpha}$ is a vector of learned weights attached to the training samples:

$$\boldsymbol{\alpha} = [\,\alpha_1 \;\cdots\; \alpha_n\,]^T.$$

This is **not the same semantic role** as the perceptron update counter, even though the same symbol is reused. Write

$$\boxed{\boldsymbol{\beta} = \Phi^T\boldsymbol{\alpha}}$$

(again: the weight vector is a combination of training samples). Then

$$\hat{\mathbf{y}} = \Phi\boldsymbol{\beta} = \Phi\Phi^T\boldsymbol{\alpha} = K\boldsymbol{\alpha}.$$

Thus a high-dimensional parameter vector $\boldsymbol{\beta}$ is replaced by **one coefficient per training example**.

Regularized kernel least-squares objective

The lecture uses

Kernel ridge objective

$$L(\boldsymbol{\alpha}) = \|\mathbf{y} - K\boldsymbol{\alpha}\|^2 + \lambda\boldsymbol{\alpha}^TK\boldsymbol{\alpha}$$

The penalty is not arbitrary: it is the ridge penalty $\lambda\|\boldsymbol{\beta}\|^2$ from §3.2.8 rewritten in terms of $\boldsymbol{\alpha}$, since

$$\|\boldsymbol{\beta}\|^2 = \boldsymbol{\beta}^T\boldsymbol{\beta} = \boldsymbol{\alpha}^T\Phi\Phi^T\boldsymbol{\alpha} = \boldsymbol{\alpha}^TK\boldsymbol{\alpha}.$$

Differentiate the first term using the same OLS pattern:

$$\frac{\partial}{\partial\boldsymbol{\alpha}}\|\mathbf{y} - K\boldsymbol{\alpha}\|^2 = -2K^T(\mathbf{y} - K\boldsymbol{\alpha}).$$

Since a kernel matrix is symmetric, $K^T = K$, so this is $-2K(\mathbf{y} - K\boldsymbol{\alpha})$. For the regularization term, use the quadratic-form identity (§1.3.5). Since $K^T = K$,

$$\frac{\partial}{\partial \boldsymbol{\alpha}}(\boldsymbol{\alpha}^T K \boldsymbol{\alpha}) = 2K\boldsymbol{\alpha},$$

so

$$\nabla_{\boldsymbol{\alpha}} L = -2K(\mathbf{y} - K\boldsymbol{\alpha}) + 2\lambda K\boldsymbol{\alpha}.$$

Set the gradient to zero:

$$\begin{aligned} -2K(\mathbf{y} - K\boldsymbol{\alpha}) + 2\lambda K\boldsymbol{\alpha} &= \mathbf{0} \\ -K\mathbf{y} + K^2\boldsymbol{\alpha} + \lambda K\boldsymbol{\alpha} &= \mathbf{0}. \end{aligned}$$

Every term has a K in front, so factor it out:

$$K\big(K\boldsymbol{\alpha} + \lambda\boldsymbol{\alpha} - \mathbf{y}\big) = \mathbf{0}.$$

When K is invertible, multiply by K^{-1} and the bracket itself must vanish. Under the regularized kernel-ridge solution used in the lecture, this yields

$$\boxed{(K + \lambda I)\boldsymbol{\alpha} = \mathbf{y}}$$

and therefore

$$\boxed{\hat{\boldsymbol{\alpha}} = (K + \lambda I)^{-1}\mathbf{y}} \qquad \boxed{\hat{\mathbf{y}} = K(K + \lambda I)^{-1}\mathbf{y}}$$

Compare ridge in §3.2.8: $(X^T X + \lambda I)^{-1}$ is $p \times p$ (one row per *feature*); here $(K + \lambda I)^{-1}$ is $n \times n$ (one row per *sample*). That swap is the whole reason the feature space can be infinite.

Prediction at a new point

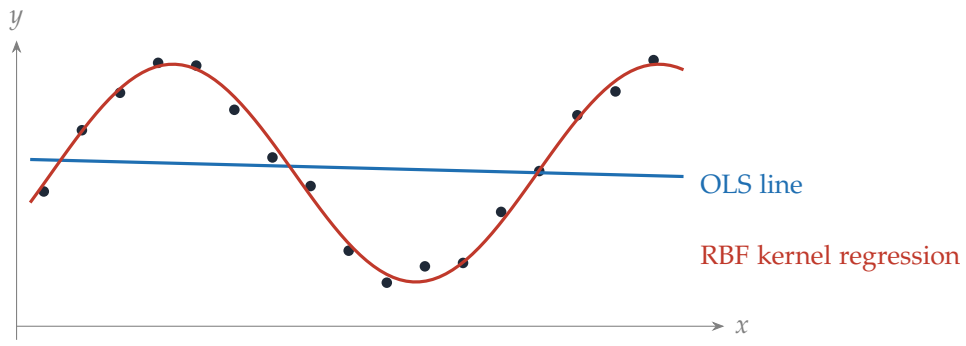
For a new input $\mathbf{x}_*$,

Kernel regression prediction

$$\hat{y}(\mathbf{x}_*) = \sum_{i=1}^{n} \alpha_i \, K(\mathbf{x}_i, \mathbf{x}_*)$$

Interpretation:

- α_i says how strongly training sample i contributes.
- $K(\mathbf{x}_i, \mathbf{x}_*)$ says how similar the new point is to training sample i.
- The prediction is the **weighted sum of these similarity contributions**.



Why this matters

For RBF regression, nearby training points have large kernel values and distant points have values near zero. The model can therefore **follow local curvature**, unlike a single global straight OLS line.

Kernel-regression limitations

The lecture emphasizes:

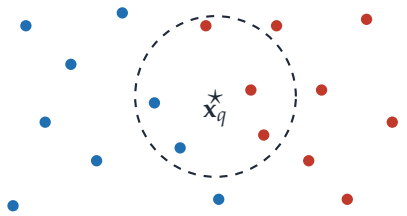
- K is $n \times n$, so storage is $O(n^2)$.
- Matrix inversion is approximately $O(n^3)$ in the basic formulation.
- Flexible kernels can overfit.
- Performance is sensitive to hyperparameters such as γ, polynomial degree d, and regularization λ.
- Similarity can become less discriminative in high dimensions (§2.2.3).

3.3.12 k-Nearest Neighbors (k-NN)

k-NN is conceptually different from the preceding models. It does **not** learn an explicit $\mathbf{w}$, β, or kernel coefficient function during a conventional training stage. It **stores the training set** and postpones most work until a query arrives. This is why it is called **instance-based** or **lazy learning**.

For a query point $\mathbf{x}_q$:

1. compute its distance to training samples;
2. identify the k closest samples;
3. classify by vote or regress by averaging their target values.



$k = 5$ nearest: 3 red, 2 blue
$\Rightarrow$ predict red

Distance metrics

These are the distances from §2.2.2. Euclidean distance:

$$d(\mathbf{x}_q, \mathbf{x}_i) = \sqrt{\sum_{j=1}^{d} (x_{qj} - x_{ij})^2}.$$

Manhattan distance:

$$d(\mathbf{x}_q, \mathbf{x}_i) = \sum_{j=1}^{d} |x_{qj} - x_{ij}|.$$

The **Minkowski** family combines both:

$$\boxed{d_p(\mathbf{x}_q, \mathbf{x}_i) = \left(\sum_{j} |x_{qj} - x_{ij}|^p \right)^{1/p}}$$

with $p = 1$ giving Manhattan and $p = 2$ giving Euclidean distance.

Why feature scaling is critical

Distance is numerical. If one feature ranges from 0 to 200,000 and another from 0 to 100, the first feature can dominate the distance simply because of its units. Standard fixes (§2.1.4) are min–max normalization

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

or standardization

$$x' = \frac{x - \mu}{\sigma}.$$

Key result In k-NN, "nearest" is meaningful **only after the feature scales make sense**. Otherwise the largest-scale feature may silently define the neighborhood.

Classification, ties, and weighted voting

For classification,

$$\boxed{\hat{y}_q = \text{mode}(y_1, \ldots, y_k)}$$

Example: $[0, 1, 1, 0, 1] \Rightarrow \hat{y}_q = 1$.

A tie such as $[0, 1, 1, 0]$ needs a tie-breaking rule. The lecture lists random selection, preference for the single nearest neighbor, and **weighted voting**. A common distance weight is

$$\boxed{w_i = \frac{1}{d(\mathbf{x}_q, \mathbf{x}_i) + \epsilon}}$$

where ϵ prevents division by zero. Closer points receive larger voting influence.

Weighting can flip the vote

$k = 3$ neighbors at distances $1, 2, 4$ with labels $1, 0, 0$.
Plain vote: two 0's vs one $1 \Rightarrow$ predict 0.
Weighted: $w = 1,\ 0.5,\ 0.25$. Class 1 gets 1; class 0 gets $0.5 + 0.25 = 0.75 \Rightarrow$ predict **1**. The one very close neighbor outweighs two far ones.

k-NN regression

For regression, average the targets of the k nearest samples:

$$\hat{y}_q = \frac{1}{k} \sum_{i=1}^{k} y_i.$$

Or use a weighted average:

$$\hat{y}_q = \frac{\sum_{i=1}^{k} w_i y_i}{\sum_{i=1}^{k} w_i}.$$

This weighted average is kernel-regression-flavoured: closer samples count more, just with a hard cutoff at k neighbors instead of a smooth decay.

3.3.13 The Meaning of Bias and Variance in k-NN

The parameter k controls how local or smooth the model is. This is a classic bias–variance tradeoff (§2.3.2).

Variance: sensitivity to the exact training sample

Variance means how strongly the learned predictions would change if the training dataset changed slightly. For $k = 1$, the entire prediction depends on one nearest point. Remove that point, move it slightly, or replace it with a noisy/mislabeled point, and the prediction may change dramatically. Therefore

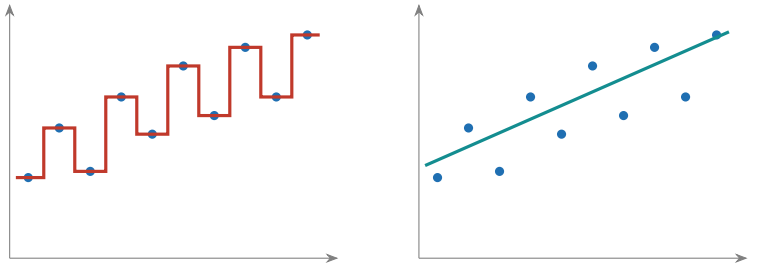
$$\boxed{k \text{ small} \Rightarrow \text{high variance}}$$

This is why small k can overfit noise.

Bias: systematic error from too much smoothing

Bias means systematic error caused by a model being too rigid or too smooth to represent real structure. For large k, predictions average over a broad neighborhood. Local bumps, class pockets, and genuine small-scale structure are washed out. In the extreme $k \to n$, regression approaches the **global average** target and classification approaches the **globally dominant class**. Thus

$$\boxed{k \text{ large} \Rightarrow \text{high bias}}$$



$k = 1$: follows every point

k moderate: follows the trend

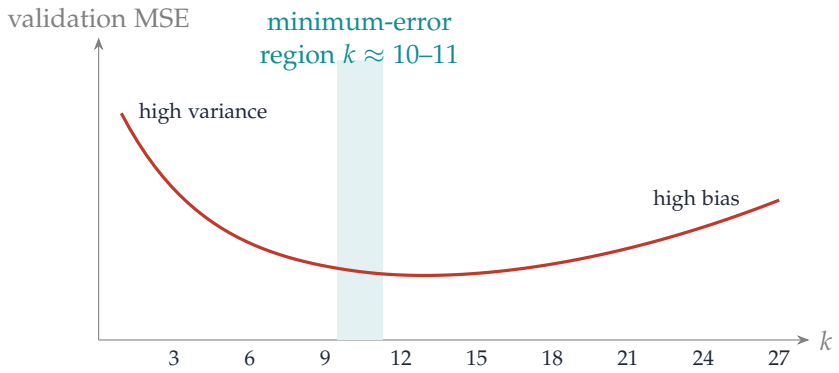
Key result

$$k \text{ small} \Rightarrow \text{low bias, high variance (overfitting risk)}$$

$$k \text{ large} \Rightarrow \text{high bias, low variance (underfitting risk)}$$

3.3.14 Choosing k: the Elbow / Minimum-Error Region

The lecture plots validation error or regression MSE as a function of k.



Read the graph from left to right:

1. At very small k, error is high because the model has high variance and follows local noise.
2. As k increases, averaging reduces noise and MSE falls.
3. Around $k \approx$ 10–11, this example reaches a good bias–variance balance.
4. Increasing k further makes the model too smooth; bias rises and error begins to increase.

Clarification

The word "elbow" refers to the region where improvement starts flattening or reversing. In this lecture figure, the practically important feature is the **minimum-error region** around $k \approx$ 10–11; k should be chosen from **validation performance**, not by habit (K-fold cross-validation, §2.3.3, is the standard way).

3.3.15 k-NN Computational Cost and the Curse of Dimensionality

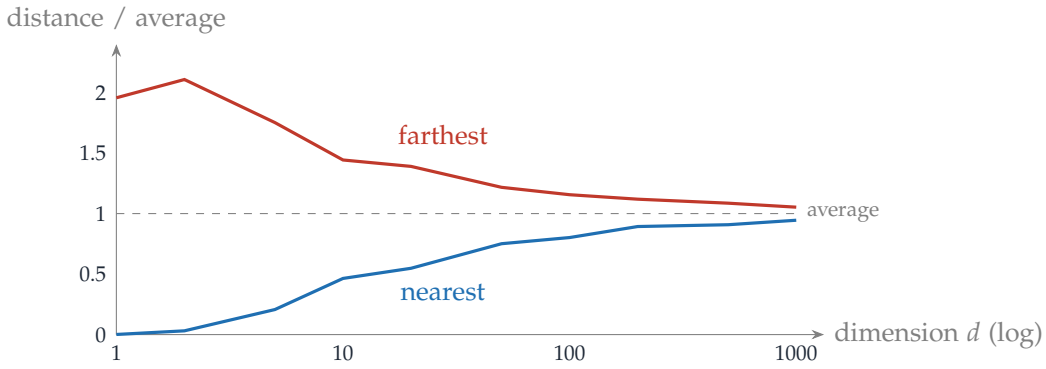
For one query, brute-force k-NN computes distance to all n training samples. With d features,

$$\boxed{\text{query time} = O(nd)} \qquad \boxed{\text{memory} = O(nd)}$$

(memory because the entire dataset is stored). Note the flip compared with the other models: training is free, prediction is expensive. The lecture lists **KD-trees** for low-dimensional data, **ball trees** for moderate dimensions, and **approximate-nearest-neighbor** methods for large-scale search.

Curse of dimensionality

As dimension grows, Euclidean distances tend to become less distinguishable (§2.2.3). A nearest point, an average point, and a farthest point can all become relatively similar in distance.



Simulation: 1000 uniform points, one random query. As d grows, nearest and farthest distances squeeze toward the average.

This hurts k-NN because its entire logic depends on the nearest neighbors being **meaningfully nearer** than the rest. The lecture suggests dimensionality reduction such as PCA/LDA or feature selection (§2.1.7–2.1.9) before k-NN when needed.

Class imbalance

If one class dominates the dataset, the local neighborhood can also be dominated by that class. Distance-weighted voting or class-specific weighting can reduce this effect (and see the sampling fixes in §2.1.3).

3.3.16 Quick Reference: One-Page Mental Model

Concept	Core idea	Formula / cue
Feature mapping	Re-express data so a linear model can solve a nonlinear-looking problem	$\mathbf{x} \mapsto \phi(\mathbf{x})$
Polynomial kernel	Dot product in a finite polynomial feature space without building it	$(1 + \mathbf{x}^T\mathbf{x}')^d$
Feature count	Explicit polynomial maps explode	$\binom{n+d}{d}$
Kernel trick	Replace $\phi(\mathbf{x})^T\phi(\mathbf{x}')$ with direct $K(\mathbf{x}, \mathbf{x}')$	"high-dimensional dot product for free"
Kernel perceptron	Rewrite $\mathbf{w}$ as weighted training samples, then kernelize the dot products	$\sum_j \alpha_j y_j K(\mathbf{x}_j, \mathbf{x})$
RBF	Local distance-based similarity with an infinite polynomial expansion	$e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}$
γ	Width of each local hill	$\gamma \uparrow$: narrow, detailed, overfit risk
Laplacian	Sharper local distance similarity	$e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|}$
Valid kernel	Behaves like a dot product	K symmetric PSD
Kernel ridge	One weight per training sample	$\hat{\boldsymbol{\alpha}} = (K + \lambda I)^{-1}\mathbf{y}$
Kernel regression	Predict from weighted kernel similarities to training samples	$\hat{y}(\mathbf{x}_*) = \sum_i \alpha_i K(\mathbf{x}_i, \mathbf{x}_*)$
k-NN	No explicit fitted model; vote/average the nearest stored examples	distance $\rightarrow$ k neighbors $\rightarrow$ vote/mean
Minkowski	One family for both distances	$p = 1$ Manhattan, $p = 2$ Euclidean
Weighted vote	Closer neighbors count more	$w_i = 1/(d + \epsilon)$
Bias–variance in k-NN	k controls locality versus smoothing	small k: high variance; large k: high bias
Cost	Lazy: free training, expensive queries	query $O(nd)$, memory $O(nd)$

Final conceptual bridge Across the whole section, the recurring question is how **similarity and representation control geometry**. Feature maps change the representation; kernels compute transformed-space similarity implicitly; k-NN skips a parametric model and uses distance directly. The mathematics differs, but the common theme is deciding **which points should look similar** and what structure that similarity makes learnable.